



---

# Plants\_app

---

Proyecto Realizado por Andrés Ojeda Rodríguez



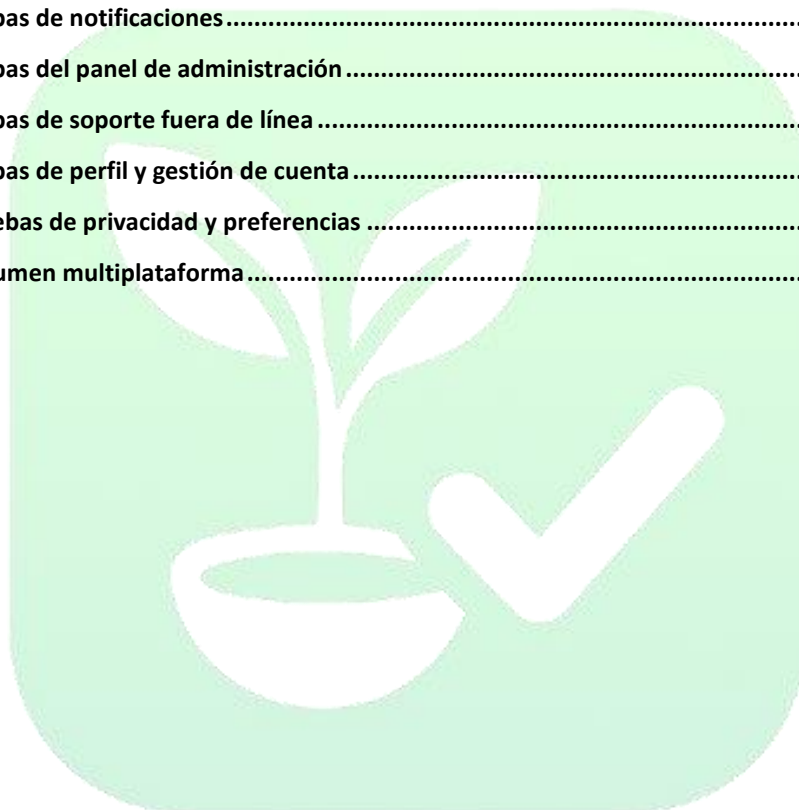
**C.F.G.S. DESARROLLO DE APLICACIONES MULTIPLATAFORMA**

IES NERVIÓN, 2025-26

## Tabla de contenido

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>1. Objetivos del EVS .....</b>                                            | <b>4</b>  |
| <b>1.1. Descripción general del sistema .....</b>                            | <b>4</b>  |
| 1.1.1. Diagrama de contexto del sistema.....                                 | 5         |
| <b>1.2. Descripción de la situación actual .....</b>                         | <b>6</b>  |
| <b>1.3. Catálogo de requisitos previos .....</b>                             | <b>6</b>  |
| 1.3.1. Requisitos funcionales.....                                           | 7         |
| 1.3.2. Requisitos de datos .....                                             | 9         |
| 1.3.3. Requisitos de interfaz .....                                          | 9         |
| 1.3.4. Requisitos no funcionales.....                                        | 10        |
| <b>1.4. Alternativas a la solución .....</b>                                 | <b>11</b> |
| 1.4.1. Alternativa I — Android nativo con Kotlin y Firebase .....            | 11        |
| 1.4.2. Alternativa II — React Native con Node.js (Express) y PostgreSQL..... | 12        |
| <b>1.5. Justificación de la alternativa seleccionada .....</b>               | <b>13</b> |
| <b>2. Gestión del proyecto.....</b>                                          | <b>15</b> |
| 2.1. Metodología.....                                                        | 15        |
| 2.2. Backlog.....                                                            | 15        |
| 2.3. Sprints.....                                                            | 18        |
| <b>3. Análisis de los sistemas de información .....</b>                      | <b>19</b> |
| 3.1. Descripción general del entorno tecnológico del sistema .....           | 19        |
| 3.2. Catálogo de usuarios .....                                              | 20        |
| 3.3. Modelo de casos de uso .....                                            | 21        |
| 3.4. Especificación de casos de uso .....                                    | 22        |
| 3.5. Modelo de clases de análisis .....                                      | 40        |
| 3.6. Interfaces de usuario.....                                              | 41        |
| 3.6.1. Aspectos comunes de la interfaz de usuario .....                      | 42        |
| 3.6.2. Especificación de pantallas y ventanas .....                          | 42        |
| <b>4. Diseño de los sistemas de información .....</b>                        | <b>45</b> |
| 4.1. Diseño de la arquitectura del sistema .....                             | 45        |
| 4.1.1. Descripción general del entorno tecnológico del sistema .....         | 46        |
| 4.1.2. Catálogo de requisitos de diseño.....                                 | 48        |
| 4.2. Modelo de clases de diseño .....                                        | 49        |
| 4.2.1. Modelo de clases de diseño (Frontend Flutter).....                    | 49        |
| 4.2.2. Modelo de clases de diseño (Backend NestJS) .....                     | 49        |

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>4.3. Modelo físico de datos .....</b>                     | <b>50</b> |
| <b>4.4. Diseño de la interfaz de usuario .....</b>           | <b>59</b> |
| <b>4.5. Plan de migración y carga inicial de datos .....</b> | <b>82</b> |
| <b>4.6. Plan de pruebas.....</b>                             | <b>83</b> |
| <b>4.6.1. Pruebas de autenticación y registro.....</b>       | <b>83</b> |
| <b>4.6.2. Pruebas de inventario de plantas .....</b>         | <b>83</b> |
| <b>4.6.3. Pruebas de recordatorios y calendario .....</b>    | <b>84</b> |
| <b>4.6.4. Pruebas del feed de la comunidad .....</b>         | <b>84</b> |
| <b>4.6.5. Pruebas de chat en tiempo real .....</b>           | <b>85</b> |
| <b>4.6.6. Pruebas de notificaciones.....</b>                 | <b>86</b> |
| <b>4.6.7. Pruebas del panel de administración.....</b>       | <b>86</b> |
| <b>4.6.8. Pruebas de soporte fuera de línea .....</b>        | <b>87</b> |
| <b>4.6.9. Pruebas de perfil y gestión de cuenta .....</b>    | <b>87</b> |
| <b>4.6.10. Pruebas de privacidad y preferencias .....</b>    | <b>88</b> |
| <b>4.6.11. Resumen multiplataforma .....</b>                 | <b>88</b> |



## 1. Objetivos del EVS

El presente Estudio de Viabilidad del Sistema tiene como objetivo determinar si es razonable abordar el desarrollo de Plants\_app, una aplicación multiplataforma orientada a la gestión inteligente de plantas en el ámbito doméstico. Para ello se describe de forma narrativa el sistema que se pretende construir, se analizan las soluciones existentes en el mercado, se enumeran los requisitos previos que deberá cumplir y se evalúan dos alternativas técnicas adicionales antes de justificar la solución que finalmente se adopta.

Con esta sección se persigue, además, fijar un marco común de entendimiento entre el alumno y el tribunal evaluador: qué problema se aborda, a qué tipo de usuario se dirige la solución y qué decisiones tecnológicas determinarán el resto del documento. La conclusión del EVS habilita la entrada en la fase de análisis (sección 3) y diseño (sección 4) con un alcance acotado y respaldado por un estudio de riesgos.

### 1.1. Descripción general del sistema

Plants\_app es una aplicación multiplataforma (web y dispositivos móviles Android e iOS) que asiste al usuario aficionado a la jardinería en el cuidado diario de sus plantas. El producto se dirige a personas con plantas en casa, balcón o jardín que desean disponer de una herramienta única para no olvidar el riego, recibir alertas cuando las condiciones meteorológicas afecten a los cuidados habituales y compartir su experiencia con otros usuarios. El objetivo del sistema es transformar el cuidado doméstico de plantas, hoy generalmente disperso entre la intuición personal y aplicaciones comerciales limitadas, en una práctica acompañada y sistematizada por la propia aplicación.

Para hacer uso del sistema se requiere un registro previo con nombre, correo electrónico y contraseña. Una vez autenticado, el usuario dispone de un conjunto de capacidades funcionales que se describen a continuación de forma independiente del modo concreto en el que se materialicen en la interfaz.

La primera capacidad es la gestión del inventario personal de plantas. El usuario puede registrar cada una de sus plantas indicando un nombre personalizado, una especie procedente de un catálogo común, una foto opcional, la ubicación de la planta y la frecuencia de riego deseada. Al seleccionar la especie, el sistema pre-rellena automáticamente la necesidad de luz y la frecuencia base de riego sugeridas, ambas modificables manualmente. El inventario es consultable y filtrable por nombre, especie o ciudad, y cada planta dispone de una ficha detallada con su histórico, sus próximos cuidados previstos y, cuando aplique, un desglose explicativo del cálculo de riego.

A partir del inventario, el sistema genera recordatorios inteligentes adaptados al clima local del usuario. Los recordatorios de riego, poda y cosecha se planifican según la frecuencia configurada y las características de cada especie. La planificación de riego no se trata como un valor fijo aislado, sino que se ajusta dinámicamente en función de las condiciones meteorológicas reales: cuando la previsión anticipa precipitación significativa, el sistema pospone el próximo riego y, al día siguiente, contrasta el dato observado con la previsión para confirmar el ajuste o revertirlo si la lluvia no se produjo. Todos los recordatorios planificados pueden consultarse de forma unificada con visibilidad a noventa días.

El sistema incorpora también una capa social y comunitaria. Los usuarios pueden publicar contenido textual y gráfico accesible al resto de la comunidad, expresar su interés mediante "me gusta", comentar publicaciones de otros usuarios, navegar a los perfiles públicos de otros miembros y reportar contenido que consideren inadecuado. Cada reporte genera un ticket de incidencia con un número de seguimiento único, que pasa a ser gestionado por el equipo administrador.

Junto a la capa social pública, los usuarios disponen de un canal de comunicación privada en tiempo real. Cualquier usuario puede establecer una conversación uno a uno con otro miembro de la comunidad; los mensajes se entregan inmediatamente cuando ambos interlocutores se encuentran conectados y, en caso contrario, se complementan con notificaciones push agrupadas que conducen al usuario a la conversación correspondiente al pulsarlas. Cada mensaje informa de su estado (pendiente, enviado, entregado o leído), de forma equivalente a las soluciones de mensajería habituales.

Todas las notificaciones internas generadas por el sistema (recordatorios, alertas climáticas, avisos administrativos y mensajes informativos) se centralizan en un único punto de consulta, donde el usuario puede revisarlas, marcarlas como leídas o eliminarlas, individualmente o por lotes. Un contador permanente refleja el número de notificaciones pendientes, evitando que el usuario pase por alto eventos relevantes.

La aplicación dispone, finalmente, de un bloque de gestión del perfil, de la cuenta y de las preferencias personales. El usuario puede personalizar su perfil público (foto, banner, biografía, ubicación), cambiar su contraseña, exportar todos sus datos personales en un formato portable y eliminar su cuenta voluntariamente. Las preferencias de privacidad (perfil público o privado) y de notificaciones (activación o desactivación de notificaciones push) se aplican de forma efectiva tanto en la presentación como en el comportamiento del servidor.

Un subconjunto de usuarios dispone, adicionalmente, de privilegios administrativos que le permiten gestionar los reportes de incidencia generados por la comunidad, moderar contenido eliminando publicaciones y comentarios que infrinjan las normas, banear temporalmente a usuarios reincidentes, enviar avisos formales, restaurar elementos eliminados por moderación y, desde un panel de diagnóstico, ejecutar manualmente el proceso de recordatorios o simular eventos meteorológicos para verificar el sistema.

---

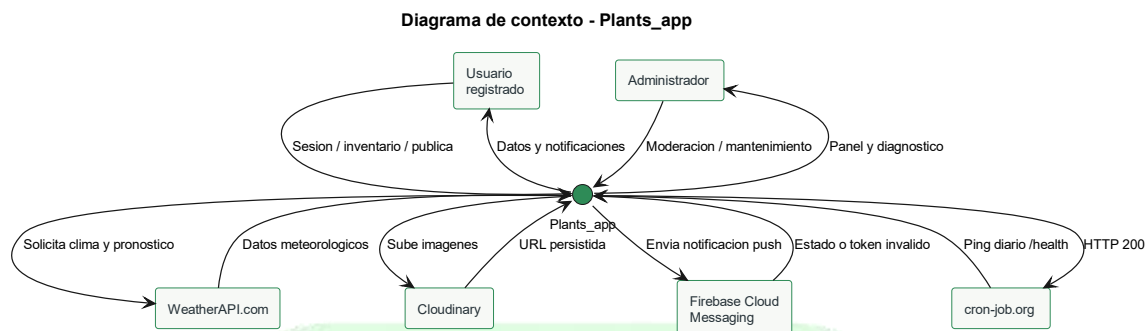
### 1.1.1. Diagrama de contexto del sistema

El diagrama de contexto representa *Plants\_app* como un sistema único y muestra todos los actores externos (humanos y servicios) con los que se intercambia información. Este nivel de abstracción no detalla la implementación interna; únicamente identifica qué entidades interactúan con la aplicación y qué flujos circulan entre ellas en cada sentido.

Los actores externos representados son:

- **Usuario registrado:** el rol principal. Envía credenciales, datos de plantas y publicaciones; consume el feed, recibe notificaciones y mantiene conversaciones privadas.
- **Administrador:** usuario registrado con privilegios elevados. Además de las acciones de un usuario común, gestiona reportes, modera contenido, banea usuarios y ejecuta acciones de mantenimiento.
- **WeatherAPI.com:** servicio externo que proporciona datos meteorológicos (clima actual, pronóstico a 48 horas e historial de precipitaciones). El sistema lo consulta mediante un proxy interno para evitar exponer la clave de acceso al cliente.
- **Cloudinary:** servicio externo de almacenamiento de imágenes. La aplicación delega en él la subida y entrega de fotografías de plantas, posts, perfiles y banners.
- **Firestore Cloud Messaging (FCM):** pasarela de notificaciones push del fabricante. El backend invoca su API para entregar notificaciones al sistema operativo del dispositivo cuando la aplicación está cerrada o en segundo plano.

- **cron-job.org**: servicio externo de programación de tareas. Realiza una petición HTTP diaria al endpoint de salud del backend para mantener el contenedor de hosting activo justo antes de la ejecución del cron interno.



## 1.2. Descripción de la situación actual

En el mercado existen diversas aplicaciones móviles orientadas al cuidado de plantas. Las más populares son Planta, PictureThis, Greg y Blossom. Estas soluciones se enfocan principalmente en dos aspectos: la identificación visual de especies a partir de fotografías y la generación de recordatorios de riego basados en una frecuencia fija seleccionada al añadir la planta. Salvo excepciones puntuales, su modelo de negocio se basa en suscripciones de pago que limitan el número de plantas registrables en la versión gratuita o restringen el acceso a funciones avanzadas como el reconocimiento por imagen o el diagnóstico de enfermedades.

Frente a estas opciones, Plants\_app introduce tres elementos diferenciales. En primer lugar, la frecuencia de riego no se trata como un valor fijo aislado, sino que se ajusta de forma automática en función del clima local del usuario: si la previsión meteorológica indica precipitación significativa en las próximas horas, el sistema pospone el próximo riego y notifica al usuario; al día siguiente se contrasta el dato observado con la previsión para confirmar el ajuste o revertirlo si la lluvia no llegó a producirse. En segundo lugar, la aplicación incorpora una capa social completa (feed de publicaciones, comentarios, "me gusta" y chat privado en tiempo real) que ninguna de las herramientas comerciales ofrece de forma nativa. Por último, todo el conjunto se entrega como una aplicación multiplataforma única, con un código compartido entre web, Android e iOS, sin barreras de pago ni limitaciones en el número de plantas registrables.

## 1.3. Catálogo de requisitos previos

El catálogo de requisitos recoge, en cuatro tablas separadas, las capacidades que el sistema deberá ofrecer. Se clasifican en cuatro categorías clásicas:

- **Requisitos funcionales (RQF#)**: acciones concretas que el sistema realiza y comportamientos observables desde el punto de vista del usuario.
- **Requisitos de datos (RQD#)**: información que el sistema debe persistir entre sesiones, independientemente del motor de base de datos elegido.
- **Requisitos de interfaz (RQI#)**: qué información debe presentarse al usuario y cómo debe estructurarse la navegación.

- **Requisitos no funcionales (RQNF#):** características transversales del sistema (rendimiento, seguridad, multiplataforma, escalabilidad, accesibilidad).

Cada requisito incluye un código único, una descripción concisa redactada en infinitivo, una prioridad en escala 1 (alta) – 2 (media) – 3 (baja) y la fecha de creación. La prioridad refleja la importancia funcional para el alcance del proyecto: prioridad 1 indica funcionalidad imprescindible del producto mínimo viable; prioridad 2, funcionalidad importante pero no esencial; prioridad 3, funcionalidad complementaria que aporta valor pero no condiciona la entrega.

El detalle de cada categoría se desarrolla en los apartados siguientes.

### 1.3.1. Requisitos funcionales

| Código | Descripción                                                                                                                                            | Prioridad | Fecha       |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------------|
| RQF1   | Debe permitir registrar nuevos usuarios mediante nombre, correo electrónico y contraseña.                                                              | 1         | 26 feb 2026 |
| RQF2   | Debe permitir iniciar sesión a los usuarios registrados mediante correo y contraseña.                                                                  | 1         | 26 feb 2026 |
| RQF3   | Debe mantener la sesión del usuario entre arranques de la aplicación sin requerir credenciales si la sesión sigue siendo válida.                       | 1         | 26 feb 2026 |
| RQF4   | Debe permitir cerrar sesión de forma manual desde la pantalla de perfil.                                                                               | 1         | 26 feb 2026 |
| RQF5   | Debe permitir registrar plantas con nombre personalizado, especie, foto, ubicación, frecuencia de riego y necesidad de luz.                            | 1         | 1 mar 2026  |
| RQF6   | Debe permitir editar y eliminar las plantas que el usuario ha registrado previamente.                                                                  | 1         | 1 mar 2026  |
| RQF7   | Debe permitir buscar y filtrar el inventario de plantas por nombre, por especie y por ciudad.                                                          | 2         | 5 mar 2026  |
| RQF8   | Debe permitir registrar manualmente el riego de una planta y recalcular el próximo recordatorio aplicando el ajuste estacional definido en la especie. | 1         | 8 mar 2026  |
| RQF9   | Debe generar recordatorios automáticos de riego, poda y cosecha en función de la frecuencia configurada y de las características de la especie.        | 1         | 10 mar 2026 |
| RQF10  | Debe ajustar la planificación de riego en función del clima local, posponiendo el próximo riego cuando se prevé precipitación significativa.           | 1         | 14 mar 2026 |
| RQF11  | Debe consultar al día siguiente la precipitación realmente caída y revertir el ajuste si la lluvia prevista no se confirmó.                            | 2         | 18 mar 2026 |
| RQF12  | Debe presentar un calendario unificado con todos los recordatorios planificados para los próximos noventa días.                                        | 2         | 25 mar 2026 |
| RQF13  | Debe permitir publicar mensajes en el feed de la comunidad, opcionalmente con imagen adjunta; y eliminarlos.                                           | 1         | 25 mar 2026 |

| Código | Descripción                                                                                                                                                                                                                                              | Prioridad | Fecha       |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------------|
| RQF14  | Debe permitir a otros usuarios dar y quitar "me gusta" a una publicación.                                                                                                                                                                                | 1         | 25 mar 2026 |
| RQF15  | Debe permitir añadir y eliminar comentarios sobre cualquier publicación del feed.                                                                                                                                                                        | 2         | 29 mar 2026 |
| RQF16  | Debe permitir reportar publicaciones o comentarios considerados inadecuados, generando un ticket de incidencia.                                                                                                                                          | 1         | 1 abr 2026  |
| RQF17  | Debe permitir el envío y la recepción de mensajes privados entre dos usuarios en tiempo real.                                                                                                                                                            | 2         | 5 abr 2026  |
| RQF18  | Debe mostrar el estado de cada mensaje del chat (pendiente, enviado, entregado, leído).                                                                                                                                                                  | 2         | 7 abr 2026  |
| RQF19  | Debe permitir buscar otros usuarios por nombre y acceder a su perfil público.                                                                                                                                                                            | 2         | 10 abr 2026 |
| RQF20  | Debe permitir editar el perfil del usuario (foto, banner, biografía, ubicación, preferencias).                                                                                                                                                           | 2         | 10 abr 2026 |
| RQF21  | Debe permitir al usuario cambiar su contraseña tras introducir la anterior.                                                                                                                                                                              | 3         | 12 abr 2026 |
| RQF22  | Debe permitir exportar todos los datos personales del usuario en formato JSON descargable.                                                                                                                                                               | 2         | 12 abr 2026 |
| RQF23  | Debe permitir al usuario eliminar su cuenta, con opción de mantener su contenido público de forma anónima.                                                                                                                                               | 2         | 12 abr 2026 |
| RQF24  | Debe enviar notificaciones push al dispositivo del usuario cuando este se encuentre fuera de la aplicación.                                                                                                                                              | 2         | 15 abr 2026 |
| RQF25  | Debe centralizar todas las notificaciones internas (recordatorios, alertas climáticas, avisos administrativos) en una pestaña dedicada con selección múltiple.                                                                                           | 2         | 15 abr 2026 |
| RQF26  | Debe permitir al administrador moderar la comunidad: eliminar publicaciones y comentarios, banear usuarios temporalmente y enviar avisos.                                                                                                                | 1         | 18 abr 2026 |
| RQF27  | Debe permitir al administrador resolver, desestimar o reabrir reportes de incidencia.                                                                                                                                                                    | 2         | 18 abr 2026 |
| RQF28  | Debe permitir al administrador restaurar elementos eliminados por moderación.                                                                                                                                                                            | 3         | 20 abr 2026 |
| RQF29  | Debe ofrecer un panel de diagnóstico con información del sistema y un botón para ejecutar manualmente el proceso de recordatorios; y un botón para simular eventos climáticos sobre las plantas registradas para verificar el sistema de notificaciones. | 3         | 20 abr 2026 |
| RQF30  | Debe gestionar de forma transparente la pérdida temporal de conexión, informando al usuario y reintentando automáticamente al recuperarla.                                                                                                               | 2         | 22 abr 2026 |

### 1.3.2. Requisitos de datos

| Código | Descripción                                                                                                                                                        | Prioridad | Fecha       |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------------|
| RQD1   | Deben guardarse los datos de perfil de cada usuario (nombre, correo, contraseña cifrada, foto, banner, biografía, ubicación).                                      | 1         | 26 feb 2026 |
| RQD2   | Deben guardarse las preferencias de cada usuario (notificaciones push, perfil público o privado).                                                                  | 2         | 27 feb 2026 |
| RQD3   | Deben guardarse las plantas registradas por cada usuario con todos sus atributos y su histórico de riegos.                                                         | 1         | 1 mar 2026  |
| RQD4   | Debe guardarse un catálogo de especies de plantas con sus características de cuidado, climatología compatible y consejos.                                          | 1         | 1 mar 2026  |
| RQD5   | Deben guardarse los recordatorios programados y un histórico de los recordatorios procesados para garantizar la idempotencia.                                      | 1         | 10 mar 2026 |
| RQD6   | Deben guardarse las publicaciones del feed, los "me gusta" emitidos y los comentarios añadidos a cada publicación.                                                 | 1         | 25 mar 2026 |
| RQD7   | Deben guardarse las conversaciones privadas, los mensajes intercambiados y el estado de cada mensaje.                                                              | 1         | 1 abr 2026  |
| RQD8   | Deben guardarse las notificaciones in-app generadas por el sistema con su estado de lectura.                                                                       | 2         | 15 abr 2026 |
| RQD9   | Deben guardarse los reportes de incidencia recibidos, con número de ticket secuencial y traza del administrador que los resuelve.                                  | 2         | 18 abr 2026 |
| RQD10  | Debe guardarse una caché de datos meteorológicos por ubicación para reducir el número de peticiones a servicios externos.                                          | 2         | 14 mar 2026 |
| RQD11  | Las contraseñas se almacenarán siempre cifradas mediante un algoritmo de hash de un solo sentido, nunca en claro.                                                  | 1         | 26 feb 2026 |
| RQD12  | Las imágenes (plantas, perfiles, publicaciones) se almacenarán en un servicio externo de gestión de imágenes; en la base de datos solo se guardará la URL pública. | 2         | 1 mar 2026  |

### 1.3.3. Requisitos de interfaz

| Código | Descripción                                                                                                                                                          | Prioridad | Fecha       |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------------|
| RQI1   | Debe mostrarse una barra de navegación inferior en dispositivos móviles en posición vertical y una barra lateral en pantallas anchas (web y tabletas en horizontal). | 1         | 28 feb 2026 |
| RQI2   | Debe mostrarse el inventario de plantas en formato de tarjetas con foto, nombre, especie y próximo riego previsto.                                                   | 1         | 1 mar 2026  |

| Código | Descripción                                                                                                                                                         | Prioridad | Fecha       |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------------|
| RQI3   | Debe mostrarse el detalle de cada planta con foto a tamaño completo, información de la especie y desglose del cálculo de riego cuando aplique un ajuste estacional. | 2         | 5 mar 2026  |
| RQI4   | Debe mostrarse un calendario mensual con marcadores de color diferenciados por tipo de tarea (riego, poda, cosecha).                                                | 2         | 22 mar 2026 |
| RQI5   | Debe mostrarse el feed de la comunidad con desplazamiento infinito y publicaciones que incluyan foto, autor, fecha, "me gusta" y comentarios.                       | 1         | 25 mar 2026 |
| RQI6   | Debe mostrarse la lista de conversaciones con un indicador visual cuando contenga mensajes sin leer.                                                                | 1         | 1 abr 2026  |
| RQI7   | Debe mostrarse el estado de cada mensaje del chat con iconos distintos (reloj, una marca, dos marcas, dos marcas en color).                                         | 2         | 5 abr 2026  |
| RQI8   | Debe mostrarse el contador de notificaciones sin leer sobre el icono de la pestaña de avisos.                                                                       | 2         | 15 abr 2026 |
| RQI9   | Debe mostrarse de forma persistente un indicador en la parte superior cuando el dispositivo pierda conexión a internet.                                             | 2         | 22 abr 2026 |
| RQI10  | Debe ofrecerse un tema visual coherente con la paleta corporativa, garantizando contraste suficiente para cumplir el nivel AA de las pautas WCAG.                   | 2         | 25 abr 2026 |

#### 1.3.4. Requisitos no funcionales

| Código | Descripción                                                                                                                               | Prioridad | Fecha       |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------------|
| RQNF1  | Debe funcionar correctamente en dispositivos móviles Android 8 o superior.                                                                | 1         | 26 feb 2026 |
| RQNF2  | Debe funcionar correctamente en navegadores web modernos (Chrome, Firefox, Edge, Safari) en sus versiones recientes.                      | 1         | 26 feb 2026 |
| RQNF3  | Debe garantizar la confidencialidad de las contraseñas mediante un algoritmo de hash de un solo sentido con sal.                          | 1         | 26 feb 2026 |
| RQNF4  | Debe proteger el acceso a recursos del usuario mediante un token JWT firmado y verificado en cada petición.                               | 1         | 26 feb 2026 |
| RQNF5  | Debe enmascarar los datos personales identificables (correo, ubicación) en los registros del servidor.                                    | 2         | 28 feb 2026 |
| RQNF6  | Debe limitar la tasa de peticiones a los endpoints de autenticación para mitigar ataques de fuerza bruta.                                 | 2         | 25 abr 2026 |
| RQNF7  | El tiempo de respuesta de los endpoints REST debe situarse por debajo de los 500 milisegundos en el percentil 95 en condiciones normales. | 2         | 26 feb 2026 |

| Código | Descripción                                                                                                                                                             | Prioridad | Fecha       |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------------|
| RQNF8  | Debe ser tolerante a la pérdida temporal de conectividad: la aplicación seguirá mostrando los datos cacheados y reintentará automáticamente al recuperar la conexión.   | 2         | 22 abr 2026 |
| RQNF9  | Debe escalarse para añadir nuevas especies, nuevos tipos de recordatorio y nuevos tipos de notificación sin rediseñar las capas inferiores.                             | 2         | 1 mar 2026  |
| RQNF10 | Debe entregarse como una aplicación instalable en Android (archivo APK firmado) y como aplicación web alojada en un servicio de hosting accesible públicamente.         | 1         | 1 may 2026  |
| RQNF11 | Las preferencias de privacidad del usuario (perfil público o privado, notificaciones push) deben aplicarse de forma efectiva en el backend, no solo en la presentación. | 2         | 8 may 2026  |
| RQNF12 | El sistema debe respetar la zona horaria Europe/Madrid para la planificación de tareas programadas y la presentación de fechas al usuario final.                        | 2         | 24 abr 2026 |

## 1.4. Alternativas a la solución

Antes de fijar la pila tecnológica definitiva se evaluaron dos alternativas técnicas adicionales a la finalmente seleccionada. Se describen a continuación, cada una con su descripción general, los requisitos previos, las tecnologías implicadas y un estudio de riesgo.

### 1.4.1. Alternativa I — Android nativo con Kotlin y Firebase

#### Descripción

Aplicación nativa para Android desarrollada en lenguaje Kotlin sobre el SDK oficial de Google. La persistencia, autenticación y mensajería se delegan en Firebase, utilizando Firestore como base de datos en la nube, Firebase Authentication para la gestión de cuentas, Firebase Cloud Functions para la lógica de servidor y Firebase Cloud Messaging para las notificaciones push.

#### Requisitos

- Dispositivo o emulador Android para desarrollo.
- Cuenta de Google con proyecto Firebase configurado.
- Conocimientos de Kotlin y del SDK de Android.
- Plan Spark de Firebase (gratuito) inicialmente; potencial migración a planes de pago en función del volumen de uso.

#### Tecnologías

- Lenguaje: Kotlin.
- SDK: Android SDK.

- Base de datos: Firebase Firestore.
- Autenticación: Firebase Authentication.
- Backend: Firebase Cloud Functions.
- Notificaciones push: Firebase Cloud Messaging.

---

## Estudio de riesgo

| Riesgo                                                                                                    | Porcentaje | Importancia |
|-----------------------------------------------------------------------------------------------------------|------------|-------------|
| Aislamiento de plataforma (solo Android, sin iOS ni web)                                                  | 90%        | 1           |
| Vendor lock-in fuerte con Google Firebase                                                                 | 70%        | 1           |
| Escalado de costes con el volumen de usuarios                                                             | 40%        | 2           |
| Modelo de datos limitado por las restricciones de Firestore (consultas no relacionales, índices manuales) | 50%        | 2           |
| Incumplimiento de los plazos por curva de aprendizaje de Kotlin nativo avanzado                           | 30%        | 3           |

---

### 1.4.2. Alternativa II — React Native con Node.js (Express) y PostgreSQL

---

#### Descripción

Aplicación móvil multiplataforma (Android e iOS) desarrollada con React Native, sobre un backend propio en Node.js con el framework Express y base de datos relacional PostgreSQL. La mensajería en tiempo real se implementaría mediante Socket.IO sobre el mismo servidor HTTP.

---

#### Requisitos

- Entorno Node.js LTS.
- Toolchain de React Native (Xcode para iOS, Android SDK para Android).
- Servidor PostgreSQL accesible (local o en la nube).
- Conocimientos previos de JavaScript moderno, React y SQL relacional.

---

#### Tecnologías

- Lenguaje frontend: JavaScript/TypeScript.
- Framework frontend: React Native.
- Lenguaje backend: JavaScript/TypeScript con Node.js.
- Framework backend: Express.
- Base de datos: PostgreSQL.
- Comunicación en tiempo real: Socket.IO.
- Autenticación: JWT.
- Servicios externos similares a la alternativa seleccionada (almacenamiento de imágenes, notificaciones push, datos meteorológicos).

## Estudio de riesgo

| Riesgo                                                                                                                           | Porcentaje | Importancia |
|----------------------------------------------------------------------------------------------------------------------------------|------------|-------------|
| Cobertura limitada de la plataforma web (React Native no genera build web nativo)                                                | 70%        | 1           |
| Necesidad de mantener dos clientes separados (móvil con React Native y web con React) para cubrir todas las plataformas objetivo | 60%        | 1           |
| Encaje deficiente entre el modelo relacional rígido de PostgreSQL y el carácter heterogéneo de los datos del feed comunitario    | 50%        | 2           |
| Gestión manual de bindings nativos cuando un plugin específico no esté disponible                                                | 40%        | 2           |
| Experiencia visual desigual entre Android, iOS y web (cada plataforma con su propio cliente)                                     | 50%        | 2           |
| Mayor coste de mantenimiento por la duplicación de bases de código entre cliente móvil y web                                     | 60%        | 2           |

### 1.5. Justificación de la alternativa seleccionada

Tras evaluar las alternativas anteriores se opta por la combinación Flutter (frontend) + NestJS (backend) + MongoDB + Render.com, complementada con WeatherAPI.com, Cloudinary y Firebase Cloud Messaging para los servicios externos. Las razones que respaldan esta decisión son:

- **Cobertura multiplataforma real con una única base de código:** Flutter compila el mismo proyecto Dart a Android, iOS y aplicación web desde un único repositorio, sin necesidad de mantener clientes separados. Esto se contrapone tanto a la Alternativa I (limitada a Android nativo) como a la Alternativa II, en la que la cobertura web es particularmente problemática por dos motivos que conviene explicitar:
  - **Cobertura limitada de la plataforma web en React Native:** React Native fue diseñado desde su origen para compilar a binarios nativos de iOS (mediante la cadena Xcode/LLVM) y Android (mediante Gradle y el NDK), traduciendo los componentes de JavaScript a vistas nativas del sistema operativo. El framework no incluye un objetivo de compilación oficial a HTML, CSS y JavaScript ejecutables en navegador. Existen soluciones de terceros como `react-native-web`, pero presentan tres limitaciones que justifican calificar este aspecto como crítico. En primer lugar, su cobertura de la API es parcial: no todos los componentes ni los módulos nativos (cámara, almacenamiento seguro, notificaciones push, ciclos de vida de aplicación) tienen un equivalente en web, y algunos requieren reimplementación específica. En segundo lugar, el comportamiento de un mismo componente puede divergir entre web y móvil, obligando a ramificar manualmente el código para casos como la navegación, los gestos o la adaptación responsive. En tercer lugar, `react-native-web` no forma parte del repositorio principal de React Native, sino que es una librería externa mantenida por la comunidad, lo que implica que las actualizaciones del framework principal pueden romper la compatibilidad y exigir esfuerzo adicional de mantenimiento. Frente a esto, Flutter incluye la plataforma web como objetivo de compilación oficial desde su

versión 2.0, con el mismo código fuente y un nivel de paridad muy elevado. Dado que el catálogo de requisitos del proyecto (en concreto el requisito no funcional RQNF2) exige funcionamiento correcto en navegadores web modernos, esta diferencia resulta decisiva.

- **Necesidad de mantener dos clientes separados en la Alternativa II:** Como consecuencia directa del punto anterior, el escenario realista en un proyecto profesional con React Native sería mantener un cliente móvil con React Native (Android e iOS desde el mismo código) y un cliente web independiente con React clásico o un framework derivado como Next.js. Aunque ambos compartirían el lenguaje (JavaScript/TypeScript) y la librería base (React), las consecuencias prácticas son significativas: duplicación de capas (cada cliente con su propia gestión de estado, su propio sistema de navegación, sus propios componentes visuales y sus propias pruebas), doble coste de desarrollo (cada funcionalidad debe implementarse dos veces, con riesgo de divergencia a lo largo del tiempo), doble coste de pruebas y aseguramiento de calidad (cada flujo crítico se valida de forma independiente en ambos clientes) y riesgo de inconsistencia funcional cuando una corrección aplicada a un cliente no se traslada al otro. En un proyecto unipersonal con plazo acotado a un curso académico, este sobrecoste de mantenimiento condicionaría seriamente la viabilidad. Flutter, en cambio, permite mantener una única base de código que compila a Android, iOS y web desde el primer momento, eliminando por completo este problema.
- **Encaje con la naturaleza de los datos del dominio:** Las plantas, las publicaciones, los mensajes y las notificaciones presentan esquemas con campos opcionales y estructuras anidadas (ajustes estacionales por especie, metadatos de mensajes, claves de idempotencia de notificaciones). MongoDB, al ser una base de datos orientada a documentos, modela estas estructuras de forma directa sin tablas auxiliares ni esquemas rígidos. Esto evita la fricción que aparecería al modelarlas en PostgreSQL (Alternativa II) o en Firestore con sus limitaciones de consultas (Alternativa I).
- **Arquitectura limpia desde el primer momento:** NestJS impone una estructura modular basada en inyección de dependencias que se alinea de forma natural con el patrón Clean Architecture aplicado en el backend, y con la arquitectura MVVM con el patrón Provider en el frontend. Esto facilita la separación entre lógica de negocio, persistencia y presentación, y simplifica la incorporación de pruebas automatizadas.
- **Coste cero durante la fase de prototipo:** El proyecto se aloja en el plan gratuito de Render.com (backend) y en el cluster M0 gratuito de MongoDB Atlas (base de datos), con un ping diario programado en cron-job.org (también gratuito) que mantiene el servicio operativo de cara a la ejecución de tareas programadas. Esta configuración se considera adecuada para una fase de prototipo y validación funcional. Una hipotética puesta en producción a gran escala requeriría su migración a un plan de pago o a una infraestructura más robusta capaz de soportar escalado horizontal.
- **Soporte sólido y comunidad amplia:** Flutter, NestJS y MongoDB son tecnologías ampliamente adoptadas en la industria, con documentación extensa, ecosistema maduro de paquetes y una comunidad activa. Esto reduce el riesgo asociado a la curva de aprendizaje frente a tecnologías más nicho o de adopción más reciente.
- **Independencia del fabricante:** A diferencia de la Alternativa I, donde Firebase actúa simultáneamente como base de datos, servidor de funciones y servicio de autenticación, en la alternativa seleccionada el único servicio del ecosistema Google empleado es Firebase Cloud Messaging, y se utiliza exclusivamente para el envío de notificaciones push. La autenticación, la base de datos y la lógica de negocio son completamente independientes y migrables a cualquier otro proveedor sin reescritura sustancial.

- **Experiencia previa del autor:** El alumno parte con conocimientos consolidados en JavaScript moderno y experiencia previa con frameworks de tipo MVC/Clean Architecture, lo que reduce el riesgo de incumplimiento de plazos por curva de aprendizaje frente a la opción Kotlin nativo de la Alternativa I.

## 2. Gestión del proyecto

### 2.1. Metodología

El desarrollo del proyecto se ha organizado siguiendo una adaptación de la metodología Scrum a un equipo unipersonal. Scrum es una metodología ágil que prioriza la entrega frecuente de incrementos funcionales en ciclos cortos (sprints), favoreciendo la inspección continua del producto y la adaptación a los hallazgos que surgen durante el desarrollo. Sus pilares son la transparencia (todo el trabajo planificado y ejecutado se documenta), la inspección (cada incremento se valida antes de seguir avanzando) y la adaptación (el plan se ajusta cuando los hallazgos lo aconsejan).

En el contexto unipersonal del proyecto se han mantenido los elementos esenciales del marco: una pila de producto (backlog) organizada por temas funcionales, una división del trabajo en sprints sucesivos con un objetivo claro por sprint, y un incremento de producto al cierre de cada uno verificable mediante la ejecución de la propia aplicación. Se han omitido las ceremonias colectivas (planning, daily, review, retrospective) por la naturaleza individual del trabajo, sustituyéndolas por una revisión propia al cierre de cada sprint con verificación funcional en navegador web y posteriormente en dispositivo Android físico.

La gestión de tareas se ha apoyado en una hoja de ruta interna que registra cada tarea con su código identificador, su descripción, sus dependencias previas y la fecha de finalización. Esta lista se ha mantenido sincronizada con el repositorio de código a lo largo del desarrollo. El control de versiones se ha realizado mediante Git con un repositorio remoto centralizado, con la rama principal `main` desplegada de forma continua en el entorno de hosting.

### 2.2. Backlog

La pila de producto recoge todas las historias de usuario que han dado lugar al desarrollo de la aplicación, agrupadas por tema funcional. Cada historia sigue el formato canónico "Como X, quiero Y para Z" desde la perspectiva del rol que la solicita. Las historias se han ordenado dentro de cada tema priorizando primero las que componen el flujo principal y dejando al final las que aportan valor complementario.

| Tema          | ID  | Historia de usuario                                                                                                                                        |
|---------------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Autenticación | LG1 | Como usuario no registrado, quiero crear una cuenta con mi correo electrónico y una contraseña para poder utilizar la aplicación.                          |
| Autenticación | LG2 | Como usuario registrado, quiero iniciar sesión introduciendo mi correo y contraseña para acceder a mi inventario personal.                                 |
| Autenticación | LG3 | Como usuario registrado, quiero que la aplicación recuerde mi sesión entre arranques para no tener que introducir credenciales cada vez que la abro.       |
| Autenticación | LG4 | Como usuario registrado, quiero cerrar sesión voluntariamente desde mi perfil para asegurarme de que mis datos no son accesibles si presto el dispositivo. |
| Autenticación | LG5 | Como usuario nuevo, quiero ver una pantalla de bienvenida que me explique las funciones principales para entender qué puedo hacer con la aplicación.       |

| Tema                  | ID   | Historia de usuario                                                                                                                                                                      |
|-----------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Inventario de plantas | INV1 | Como usuario registrado, quiero registrar una planta indicando su nombre, especie, foto y ubicación para llevar el control de mi colección.                                              |
| Inventario de plantas | INV2 | Como usuario registrado, quiero que al elegir la especie se rellenen automáticamente la luz y la frecuencia de riego sugeridas para no tener que buscarlos por mí mismo.                 |
| Inventario de plantas | INV3 | Como usuario registrado, quiero editar los datos de una planta ya registrada para mantenerlos actualizados.                                                                              |
| Inventario de plantas | INV4 | Como usuario registrado, quiero eliminar una planta de mi inventario cuando ya no la tenga para no recibir más recordatorios suyos.                                                      |
| Inventario de plantas | INV5 | Como usuario registrado, quiero buscar y filtrar mis plantas por nombre, especie y ciudad para localizarlas rápidamente cuando tengo muchas.                                             |
| Inventario de plantas | INV6 | Como usuario registrado, quiero ver el detalle de cada planta con su foto a tamaño completo y toda su información para conocer su estado.                                                |
| Inventario de plantas | INV7 | Como usuario registrado, quiero consultar información ampliada de la especie de una de mis plantas para aprender sobre sus cuidados.                                                     |
| Recordatorios y riego | RIE1 | Como usuario registrado, quiero recibir un recordatorio cuando una planta necesite riego para no olvidarme.                                                                              |
| Recordatorios y riego | RIE2 | Como usuario registrado, quiero marcar una planta como regada para que el sistema actualice el siguiente recordatorio.                                                                   |
| Recordatorios y riego | RIE3 | Como usuario registrado, quiero que el sistema retrase los recordatorios de riego si se prevé lluvia significativa en mi zona para no malgastar agua.                                    |
| Recordatorios y riego | RIE4 | Como usuario registrado, quiero que al día siguiente el sistema verifique si la lluvia realmente cayó y revierta el ajuste si no se produjo, para que mis plantas no se queden sin agua. |
| Recordatorios y riego | RIE5 | Como usuario registrado, quiero recibir avisos de poda y cosecha cuando corresponda según la especie de mi planta para aprovecharlos en el momento adecuado.                             |
| Recordatorios y riego | RIE6 | Como usuario registrado, quiero ver todos mis recordatorios planificados en un calendario unificado para tener una visión global de las próximas tareas.                                 |
| Comunidad             | COM1 | Como usuario registrado, quiero publicar mensajes en el feed de la comunidad, opcionalmente con foto, para compartir mis experiencias.                                                   |
| Comunidad             | COM2 | Como usuario registrado, quiero dar "me gusta" a las publicaciones de otros usuarios para mostrar mi interés sin necesidad de escribir.                                                  |
| Comunidad             | COM3 | Como usuario registrado, quiero comentar las publicaciones para iniciar una conversación con su autor.                                                                                   |
| Comunidad             | COM4 | Como usuario registrado, quiero eliminar mis propias publicaciones y comentarios cuando lo decida para gestionar mi presencia.                                                           |
| Comunidad             | COM5 | Como usuario registrado, quiero buscar otros usuarios por nombre para conocer a personas con intereses similares.                                                                        |
| Comunidad             | COM6 | Como usuario registrado, quiero ver el perfil público de cualquier usuario con sus publicaciones para conocer su actividad.                                                              |

| Tema           | ID   | Historia de usuario                                                                                                                                                   |
|----------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Comunidad      | COM7 | Como usuario registrado, quiero reportar publicaciones o comentarios que considere inadecuados para mantener la comunidad saludable.                                  |
| Chat           | CHT1 | Como usuario registrado, quiero abrir una conversación privada con otro usuario para hablar directamente con él.                                                      |
| Chat           | CHT2 | Como usuario registrado, quiero enviar y recibir mensajes en tiempo real para mantener una conversación fluida.                                                       |
| Chat           | CHT3 | Como usuario registrado, quiero ver el estado de mis mensajes (enviado, entregado, leído) para saber si la otra persona los ha visto.                                 |
| Chat           | CHT4 | Como usuario registrado, quiero recibir una notificación cuando me llegue un mensaje y la aplicación esté cerrada para no perderme conversaciones importantes.        |
| Perfil         | PER1 | Como usuario registrado, quiero editar mi foto, banner, biografía y ubicación para personalizar mi presencia ante los demás.                                          |
| Perfil         | PER2 | Como usuario registrado, quiero cambiar mi contraseña desde la propia aplicación para reforzar la seguridad de mi cuenta.                                             |
| Perfil         | PER3 | Como usuario registrado, quiero exportar todos mis datos personales en un archivo descargable para tener una copia local.                                             |
| Perfil         | PER4 | Como usuario registrado, quiero eliminar mi cuenta cuando lo decida, con la opción de mantener mis publicaciones de forma anónima, para ejercer mi derecho al olvido. |
| Notificaciones | NOT1 | Como usuario registrado, quiero ver en una pestaña centralizada todas las notificaciones generadas por la aplicación para no perderme ninguna.                        |
| Notificaciones | NOT2 | Como usuario registrado, quiero marcar como leídas o eliminar varias notificaciones a la vez para gestionarlas con rapidez.                                           |
| Notificaciones | NOT3 | Como usuario registrado, quiero desactivar las notificaciones push si así lo deseo para evitar interrupciones en mi dispositivo.                                      |
| Privacidad     | PRV1 | Como usuario registrado, quiero marcar mi perfil como privado para que mis publicaciones no aparezcan en el feed público y nadie pueda iniciar conversación conmigo.  |
| Administración | ADM1 | Como administrador, quiero buscar usuarios por nombre o correo para localizarlos rápidamente.                                                                         |
| Administración | ADM2 | Como administrador, quiero consultar todos los reportes de incidencia recibidos para evaluarlos.                                                                      |
| Administración | ADM3 | Como administrador, quiero resolver, desestimar o reabrir reportes de incidencia para gestionar las quejas de los usuarios.                                           |
| Administración | ADM4 | Como administrador, quiero eliminar publicaciones y comentarios que infrinjan las normas de la comunidad.                                                             |
| Administración | ADM5 | Como administrador, quiero banear temporalmente a un usuario para sancionar conductas inadecuadas sin eliminarlo definitivamente.                                     |
| Administración | ADM6 | Como administrador, quiero enviar un aviso personalizado a un usuario como advertencia formal antes de aplicar sanciones mayores.                                     |

| Tema           | ID   | Historia de usuario                                                                                                                                |
|----------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Administración | ADM7 | Como administrador, quiero restaurar elementos eliminados por moderación si la decisión fue errónea.                                               |
| Administración | ADM8 | Como administrador, quiero ejecutar manualmente el proceso de recordatorios para validar su funcionamiento sin esperar a la ejecución programada.  |
| Conectividad   | NET1 | Como usuario registrado, quiero ver un aviso claro cuando pierda conexión a internet para entender por qué algunas acciones fallan.                |
| Conectividad   | NET2 | Como usuario registrado, quiero que la aplicación recupere automáticamente los datos al volver la conexión para no tener que recargar manualmente. |

### 2.3. Sprints

El trabajo se ha organizado en seis sprints sucesivos repartidos a lo largo del curso académico. Cada sprint cierra con un incremento de producto funcional verificable y deja la pila de tareas en un estado consistente para abordar el siguiente. A continuación se detalla, para cada sprint, su fecha de inicio, fecha de finalización, duración y descripción del trabajo realizado.

#### Sprint 1 — Cimientos del proyecto:

- Fecha de inicio: 26 de febrero de 2026
- Fecha de finalización: 11 de marzo de 2026
- Duración: 14 días

Trabajo realizado: preparación del entorno de desarrollo (Node.js, Flutter, MongoDB local), creación de cuentas en los servicios externos (WeatherAPI, Cloudinary, Firebase), inicialización de los proyectos backend (NestJS con InversifyJS) y frontend (Flutter con Provider y GetIt), configuración del contenedor de inyección de dependencias y de los middlewares globales del backend, implementación del flujo de autenticación completo (registro, inicio de sesión, validación de token), configuración del cliente HTTP centralizado en el frontend y definición de la paleta corporativa de colores.

#### Sprint 2 — Dominio principal: plantas y recordatorios:

- Fecha de inicio: 12 de marzo de 2026
- Fecha de finalización: 27 de marzo de 2026
- Duración: 16 días

Trabajo realizado: implementación del módulo de plantas (CRUD completo con propiedad de recursos, carga de fotografías a Cloudinary, búsqueda y filtrado), creación del catálogo de especies con su esquema de cuidado, integración del proxy meteorológico contra WeatherAPI con caché por ubicación, desarrollo del módulo de recordatorios con tarea programada diaria a medianoche y desarrollo de las primeras pantallas de Flutter para gestionar el inventario.

#### Sprint 3 — Comunidad y mensajería:

- Fecha de inicio: 28 de marzo de 2026
- Fecha de finalización: 12 de abril de 2026

- Duración: 16 días

Trabajo realizado: implementación del feed de publicaciones con paginación, "me gusta" y comentarios, desarrollo del módulo de conversaciones y mensajes con persistencia en MongoDB y propagación en tiempo real mediante Socket.IO, integración del servicio de notificaciones push (Firebase Cloud Messaging) y desarrollo de las pantallas del frontend correspondientes (feed, detalle de publicación, lista de conversaciones, conversación individual).

#### **Sprint 4 — Perfiles, administración y pulido:**

- Fecha de inicio: 13 de abril de 2026
- Fecha de finalización: 23 de abril de 2026
- Duración: 11 días

Trabajo realizado: implementación de las funcionalidades de perfil del usuario (edición, cambio de contraseña, exportación RGPD, eliminación de cuenta), desarrollo del panel de administración con gestión de reportes, restauración de elementos eliminados y banner de usuarios, refinamiento visual de la aplicación (banner de perfil, calendario unificado, panel de notificaciones con selección múltiple) y verificación final del enmascaramiento de datos personales en los registros del servidor.

#### **Sprint 5 — Despliegue y verificación móvil:**

- Fecha de inicio: 24 de abril de 2026
- Fecha de finalización: 10 de mayo de 2026
- Duración: 17 días

Trabajo realizado: preparación del backend para despliegue en Render.com (variables de entorno, host explícito, configuración CORS), despliegue efectivo en Render Free Tier con base de datos en MongoDB Atlas Free Tier, configuración del ping diario en cron-job.org para mantener el servicio activo, generación de la APK firmada para Android con keystore propio, batería completa de pruebas físicas en dispositivo Android real y corrección de los hallazgos detectados durante esa verificación.

#### **Sprint 6 — Auditoría y entrega:**

- Fecha de inicio: 11 de mayo de 2026
- Fecha de finalización: 20 de mayo de 2026
- Duración: 10 días

Trabajo realizado: auditoría completa de Clean Code en backend y frontend, refactorización de los puntos identificados como mejora directa, eliminación del código muerto detectado, sincronización final de la documentación técnica interna, generación de la documentación entregable del proyecto y preparación de los artefactos para la entrega (APK, build web, manuales de instalación).

### **3. Análisis de los sistemas de información**

#### **3.1. Descripción general del entorno tecnológico del sistema**

Para el análisis del sistema se ha seleccionado un conjunto de tecnologías orientadas a maximizar la cobertura multiplataforma con una única base de código y a separar de forma estricta las responsabilidades entre cliente

y servidor. La aplicación se compone de un frontend desarrollado con Flutter (lenguaje Dart) que se ejecuta tanto en dispositivos móviles Android e iOS como en navegadores web a partir del mismo proyecto, y de un backend desarrollado con NestJS (lenguaje TypeScript sobre Node.js) que expone una API REST y un canal en tiempo real basado en Socket.IO. La persistencia se delega en MongoDB, una base de datos orientada a documentos accesible a través de un cluster gestionado en MongoDB Atlas. El frontend adopta el patrón MVVM con Provider y GetIt como contenedor de inyección de dependencias, mientras que el backend organiza el código siguiendo el patrón Clean Architecture con InversifyJS, separando entidades de dominio, casos de uso, datos y presentación.

La comunicación entre cliente y servidor se realiza mediante peticiones HTTP REST (formato JSON) para las operaciones transaccionales y mediante WebSockets a través de Socket.IO para la mensajería de chat en tiempo real, la actualización del feed en vivo y la propagación de notificaciones internas. La autenticación se basa en tokens JWT firmados por el servidor y almacenados en el cliente en un almacén seguro específico de cada plataforma. La aplicación se integra con tres servicios externos: WeatherAPI.com como fuente de datos meteorológicos, Cloudinary para el almacenamiento y entrega de imágenes, y Firebase Cloud Messaging para la entrega de notificaciones push al sistema operativo del dispositivo cuando la aplicación se encuentra cerrada o en segundo plano. El backend se aloja en Render.com sobre su plan gratuito, con una base de datos en MongoDB Atlas también en plan gratuito, y un servicio externo de programación de tareas (cron-job.org) que despierta el contenedor con una petición diaria al endpoint de salud justo antes de la ejecución de las tareas programadas.

### 3.2. Catálogo de usuarios

El sistema reconoce tres perfiles de usuario diferenciados. El acceso a cada funcionalidad se determina en función del perfil del usuario que la solicita.

| Código | Nombre                | Descripción                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|--------|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| USU1   | Usuario no registrado | Visitante que aún no ha creado una cuenta. Solo puede acceder a las pantallas de bienvenida, registro e inicio de sesión. No tiene acceso a ningún recurso de la aplicación.                                                                                                                                                                                                                                                                                                                                                                                                            |
| USU2   | Usuario registrado    | Usuario con cuenta válida y sesión iniciada. Dispone de todas las funcionalidades estándar de la aplicación: gestión de su inventario de plantas, recordatorios y calendario, participación en el feed de la comunidad, mensajería privada con otros usuarios registrados, edición de su perfil y gestión de su cuenta.                                                                                                                                                                                                                                                                 |
| USU3   | Administrador         | Usuario registrado al que se le ha concedido el rol administrativo. Conserva todas las capacidades del Usuario registrado y, adicionalmente, accede al panel de administración, donde puede gestionar reportes de incidencia, moderar contenido (eliminar publicaciones y comentarios), enviar avisos formales, banear usuarios temporalmente, restaurar elementos eliminados, simular eventos climáticos para pruebas y ejecutar manualmente el proceso de recordatorios. El administrador es, por diseño, un usuario regular con privilegios elevados, no un tipo de usuario aislado. |

### 3.3. Modelo de casos de uso

El modelo de casos de uso muestra las acciones que cada actor del sistema puede realizar. La frontera del sistema engloba todos los casos de uso identificados; los actores se sitúan fuera de esa frontera y se conectan mediante asociaciones con los casos en los que intervienen. Para mantener la legibilidad del diagrama, los casos de uso se han agrupado visualmente por área funcional (Autenticación, Plantas y recordatorios, Comunidad, Mensajería, Notificaciones, Perfil y administración) pero el listado completo de casos numerados se ofrece a continuación, ordenado por el actor principal que los inicia.

Actor USU1 (Usuario no registrado):

- UC1. Registrarse en la aplicación.
- UC2. Iniciar sesión.

Actor USU2 (Usuario registrado):

- UC3. Cerrar sesión.
- UC4. Registrar una nueva planta.
- UC5. Consultar el detalle de una planta.
- UC6. Editar una planta existente.
- UC7. Eliminar una planta.
- UC8. Buscar y filtrar plantas.
- UC9. Registrar manualmente el riego de una planta.
- UC10. Consultar el calendario de tareas.
- UC11. Publicar en el feed de la comunidad.
- UC12. Dar o quitar "me gusta" a una publicación.
- UC13. Comentar una publicación.
- UC14. Eliminar publicaciones o comentarios propios.
- UC15. Reportar publicación o comentario.
- UC16. Buscar otros usuarios.
- UC17. Consultar el perfil de otro usuario.
- UC18. Iniciar conversación privada con otro usuario.
- UC19. Enviar y recibir mensajes en una conversación.
- UC20. Consultar la lista de notificaciones internas.
- UC21. Gestionar notificaciones (marcar leídas, eliminar).
- UC22. Editar el perfil personal.
- UC23. Cambiar la contraseña.
- UC24. Exportar los datos personales.
- UC25. Eliminar la cuenta.
- UC26. Configurar las preferencias (notificaciones push, perfil privado).

Actor USU3 (Administrador): además de todos los casos del Usuario registrado, los siguientes son específicos de su rol:

- UC27. Acceder al panel de administración.
- UC28. Buscar usuarios desde el panel administrativo.
- UC29. Gestionar reportes de incidencia (resolver, desestimar, reabrir).

- UC30. Eliminar publicaciones y comentarios como moderador.
- UC31. Banear temporalmente a un usuario.
- UC32. Enviar un aviso formal a un usuario.
- UC33. Consultar y restaurar elementos eliminados.
- UC34. Ejecutar manualmente el proceso de recordatorios.
- UC35. Simular eventos climáticos (lluvia o tormenta) sobre las plantas registradas.

[DIAGRAMA: Diagramas/Casos\_de\_Uso.svg]

### 3.4. Especificación de casos de uso

Los requisitos RQF9, RQF10, RQF11, RQF24 y RQF30 describen comportamiento automático del sistema (procesos programados o transversales) y por su naturaleza no se documentan como casos de uso, sino como parte del comportamiento del sistema en el apartado 4.1.1.

A continuación, se especifica en detalle cada uno de los 35 casos de uso identificados en el modelo anterior. Cada caso se documenta con su código, su nombre, el requisito funcional asociado, el actor principal, una descripción concisa, las precondiciones, las postcondiciones, la secuencia normal de pasos, las extensiones o flujos alternativos, la frecuencia esperada y los temas abiertos.

#### UC1. Registrarse:

- **Código:** UC1
- **Nombre:** Registrarse
- **Requisito:** RQF1
- **Versión:** 1.0
- **Actor principal:** USU1 – Usuario no registrado
- **Personal involucrado e intereses:** USU1
- **Descripción:** Un usuario no registrado crea una cuenta para poder acceder a la aplicación.
- **Precondiciones:** El usuario no posee una cuenta previa con el mismo correo electrónico. La contraseña introducida cumple los requisitos mínimos de longitud.
- **Postcondiciones:** Se crea un nuevo registro de usuario en la base de datos y se devuelve un token JWT válido. El usuario queda autenticado en la aplicación.
- **Secuencia normal:**
  1. El usuario accede a la pantalla de registro desde la pantalla de inicio de sesión.
  2. El usuario introduce nombre, correo y contraseña.
  3. El sistema valida el formato de los datos y la disponibilidad del correo.
  4. El sistema cifra la contraseña y crea el registro en la base de datos.
  5. El sistema genera y devuelve un token JWT junto con los datos del usuario.
  6. El usuario es redirigido a la pantalla de bienvenida.
- **Extensiones o flujos alternativos:** Si el correo ya está registrado, el sistema responde con un error de conflicto y solicita usar otro correo. Si la contraseña no cumple la longitud mínima, el sistema muestra un mensaje de validación.
- **Frecuencia esperada:** Alta durante las primeras semanas tras el lanzamiento, baja en régimen permanente.
- **Temas abiertos:** Ninguno

## UC2. Iniciar sesión:

- **Código:** UC2
- **Nombre:** Iniciar sesión
- **Requisito:** RQF2, RQF3
- **Versión:** 1.0
- **Actor principal:** USU1 – Usuario no registrado
- **Personal involucrado e intereses:** USU1
- **Descripción:** Un usuario registrado introduce sus credenciales para iniciar sesión en la aplicación.
- **Precondiciones:** El usuario dispone de una cuenta previamente registrada.
- **Postcondiciones:** El usuario queda autenticado y se le devuelve un token JWT con vigencia de 30 días.
- **Secuencia normal:**
  1. El usuario introduce correo y contraseña.
  2. El sistema busca el usuario por correo y verifica la contraseña frente al hash almacenado.
  3. El sistema genera un token JWT con duración de 30 días.
  4. El sistema devuelve el token y los datos del usuario al cliente.
  5. El cliente persiste el token en el almacén seguro local.
  6. El usuario es redirigido a la pantalla de bienvenida.
- **Extensiones o flujos alternativos:** Si las credenciales son incorrectas, el sistema responde con un error de autenticación. Si se exceden los intentos permitidos en una ventana de tiempo, el sistema bloquea temporalmente la dirección IP.
- **Frecuencia esperada:** Media.
- **Temas abiertos:** Ninguno

## UC3. Cerrar sesión:

- **Código:** UC3
- **Nombre:** Cerrar sesión
- **Requisito:** RQF4
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario cierra su sesión de forma voluntaria.
- **Precondiciones:** El usuario tiene una sesión activa.
- **Postcondiciones:** El token JWT se elimina del almacén seguro local, el socket en tiempo real se desconecta, las cachés locales se purgan y el usuario queda redirigido a la pantalla de inicio de sesión.
- **Secuencia normal:**
  1. El usuario pulsa el botón de cierre de sesión en su perfil.
  2. El sistema muestra un diálogo de confirmación.
  3. El usuario confirma la acción.
  4. El cliente notifica al servidor para desasociar el token de notificaciones push.
  5. El cliente desconecta el socket y purga los datos locales.
  6. El cliente reinicia su árbol de proveedores y navega a la pantalla de inicio de sesión.
- **Extensiones o flujos alternativos:** Si la notificación al servidor falla, la sesión local se cierra igualmente y el token push queda invalidado en el siguiente login.
- **Frecuencia esperada:** Baja.

- **Temas abiertos:** Ninguno

#### UC4. Registrar una nueva planta:

- **Código:** UC4
- **Nombre:** Registrar una nueva planta.
- **Requisito:** RQF5
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario añade una nueva planta a su inventario.
- **Precondiciones:** El usuario tiene sesión activa.
- **Postcondiciones:** Queda registrada una nueva planta asociada al usuario; la frecuencia de riego y la necesidad de luz se rellenan automáticamente a partir de la especie seleccionada.
- **Secuencia normal:**
  1. El usuario pulsa el botón "+" desde la lista de plantas.
  2. El sistema muestra el formulario de creación.
  3. El usuario introduce el nombre, selecciona una especie del catálogo y, opcionalmente, sube una foto y elige la ubicación de la planta.
  4. El sistema completa automáticamente la luz y la frecuencia de riego sugeridas por la especie.
  5. El usuario pulsa Guardar.
  6. El sistema sube la foto al servicio externo de imágenes y persiste la planta en la base de datos.
  7. El sistema redirige al usuario al listado actualizado.
- **Extensiones o flujos alternativos:** Si la subida de la foto falla, el usuario puede reintentar o guardar la planta sin imagen. Si la especie seleccionada no existe en el catálogo, el sistema responde con un error y pide volver a elegir.
- **Frecuencia esperada:** Media.
- **Temas abiertos:** Ninguno

#### UC5. Consultar el detalle de una planta:

- **Código:** UC5
- **Nombre:** Consultar el detalle de una planta
- **Requisito:** RQI3
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario consulta toda la información disponible sobre una de sus plantas.
- **Precondiciones:** El usuario tiene al menos una planta registrada.
- **Postcondiciones:** Ninguna.
- **Secuencia normal:**
  1. El usuario pulsa una planta de su listado.
  2. El sistema carga los datos de la planta y de su especie asociada.
  3. El sistema muestra la foto, el nombre, la especie, la ubicación, los próximos recordatorios y, cuando aplique, un desglose del ajuste estacional de riego.

4. El usuario puede pulsar la especie para abrir una ventana emergente con información ampliada.
- **Extensiones o flujos alternativos:** Si la planta ha sido eliminada en otra sesión, el sistema muestra un error y devuelve al usuario al listado.
- **Frecuencia esperada:** Alta.
- **Temas abiertos:** Ninguno

#### UC6. Editar una planta existente:

- **Código:** UC6
- **Nombre:** Editar una planta existente
- **Requisito:** RQF6
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario modifica los datos de una de sus plantas.
- **Precondiciones:** La planta pertenece al usuario que ejecuta la edición.
- **Postcondiciones:** Los datos modificados quedan persistidos. Si se actualiza la frecuencia o se registra un riego, se recalcula la próxima fecha de riego aplicando el ajuste estacional.
- **Secuencia normal:**
  1. Desde el detalle de la planta, el usuario pulsa Editar.
  2. El sistema muestra el formulario precargado.
  3. El usuario modifica los campos deseados y pulsa Guardar.
  4. El sistema valida los datos, actualiza el registro y recalcula la próxima fecha de riego cuando proceda.
  5. El sistema redirige al detalle actualizado.
- **Extensiones o flujos alternativos:** Si el usuario intenta editar una planta que no le pertenece, el sistema responde con un error de permiso.
- **Frecuencia esperada:** Media.
- **Temas abiertos:** Ninguno

#### UC7. Eliminar una planta:

- **Código:** UC7
- **Nombre:** Eliminar una planta
- **Requisito:** RQF6
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario retira una planta de su inventario.
- **Precondiciones:** La planta pertenece al usuario.
- **Postcondiciones:** La planta queda marcada como eliminada (borrado lógico) y no aparece en futuros listados. Los recordatorios asociados dejan de procesarse.
- **Secuencia normal:**
  1. El usuario pulsa Eliminar desde el detalle de la planta
  2. El sistema solicita confirmación.

3. El usuario confirma.
  4. El sistema marca la planta como eliminada y devuelve al usuario al listado.
- **Extensiones o flujos alternativos:** Si la planta ya estaba eliminada en otra sesión, el sistema muestra un aviso y refresca el listado.
  - **Frecuencia esperada:** Baja.
  - **Temas abiertos:** Ninguno

#### UC8. Buscar y filtrar plantas:

- **Código:** UC8
- **Nombre:** Buscar y filtrar plantas.
- **Requisito:** RQF7
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario reduce el listado de plantas mediante criterios de búsqueda.
- **Precondiciones:** El usuario tiene al menos una planta registrada.
- **Postcondiciones:** El listado mostrado contiene únicamente las plantas que cumplen los filtros activos.
- **Secuencia normal:**
  1. El usuario introduce un texto en el buscador o selecciona un filtro de especie y/o ciudad.
  2. El sistema aplica los criterios sobre el listado cargado.
  3. El sistema muestra los resultados.
- **Extensiones o flujos alternativos:** Si ningún resultado coincide, se muestra un mensaje explícito que distingue entre "sin plantas" y "sin resultados de filtro".
- **Frecuencia esperada:** Media.
- **Temas abiertos:** Ninguno

#### UC9. Registrar manualmente el riego de una planta:

- **Código:** UC9
- **Nombre:** Registrar manualmente el riego de una planta.
- **Requisito:** RQF8
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario indica que ha regado una de sus plantas y solicita que se recalcule el próximo recordatorio.
- **Precondiciones:** La planta pertenece al usuario.
- **Postcondiciones:** La fecha del último riego se actualiza al instante actual. La próxima fecha de riego se recalcula aplicando el ajuste estacional definido en la especie.
- **Secuencia normal:**
  1. El usuario pulsa "Regar hoy" desde el detalle de la planta.
  2. El sistema marca el riego como realizado, recalcula el próximo recordatorio y limpia el ajuste por previsión de lluvia si existía.
  3. El sistema muestra un mensaje de confirmación con la próxima fecha prevista.

- **Extensiones o flujos alternativos:** Si la operación falla por error de red, el usuario es informado mediante un mensaje y puede reintentar.
- **Frecuencia esperada:** Alta.
- **Temas abiertos:** Ninguno

**UC10. Consultar el calendario de tareas:**

- **Código:** UC10
- **Nombre:** Consultar el calendario de tareas
- **Requisito:** RQF12, RQI4
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** Un usuario consulta en un calendario mensual todas las tareas previstas para sus plantas.
- **Precondiciones:** El usuario tiene al menos una planta registrada.
- **Postcondiciones:** Ninguna.
- **Secuencia normal:**
  1. El usuario abre la pestaña Calendario.
  2. El sistema proyecta a noventa días los recordatorios de riego, poda y cosecha a partir de las plantas y las especies del usuario
  3. El sistema renderiza el calendario con marcadores de color diferenciados por tipo.
  4. El usuario selecciona un día y obtiene la lista detallada de eventos.
- **Extensiones o flujos alternativos:** Si no hay eventos para el día seleccionado, se muestra una lista vacía.
- **Frecuencia esperada:** Media.
- **Temas abiertos:** Ninguno

**UC11. Publicar en el feed de la comunidad:**

- **Código:** UC11
- **Nombre:** Publicar en el feed de la comunidad
- **Requisito:** RQF13
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario crea una publicación visible para el resto de usuarios.
- **Precondiciones:** El usuario no está baneado.
- **Postcondiciones:** La publicación queda registrada y aparece en el feed de los demás usuarios.
- **Secuencia normal:**
  1. El usuario pulsa el botón "+" desde el feed.
  2. El usuario, opcionalmente, selecciona una foto y escribe el contenido.
  3. El usuario pulsa Publicar.
  4. El sistema sube la foto al servicio externo y persiste la publicación.
  5. El sistema notifica al resto de clientes la actualización del feed en tiempo real.

- **Extensiones o flujos alternativos:** Si la subida de imagen falla, el usuario es informado y puede reintentar o publicar sin imagen. Si el usuario está baneado, la operación es rechazada.
- **Frecuencia esperada:** Media.
- **Temas abiertos:** Ninguno

#### UC12. Dar o quitar "me gusta" a una publicación:

- **Código:** UC12
- **Nombre:** Dar o quitar "me gusta" a una publicación
- **Requisito:** RQF14
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario marca o desmarca su "me gusta" en una publicación.
- **Precondiciones:** El usuario no está baneado.
- **Postcondiciones:** El contador de "me gusta" de la publicación se incrementa o decrementa en una unidad.
- **Secuencia normal:**
  1. El usuario pulsa el icono de corazón en una publicación.
  2. El cliente aplica el cambio de forma optimista.
  3. El sistema persiste el cambio en la base de datos.
  4. El sistema emite la actualización por el canal en tiempo real para sincronizar a los demás clientes.
- **Extensiones o flujos alternativos:** Si la operación falla, el cliente revierte el cambio optimista y muestra un mensaje de error.
- **Frecuencia esperada:** Alta.
- **Temas abiertos:** Ninguno

#### UC13. Comentar una publicación:

- **Código:** UC13
- **Nombre:** Comentar una publicación
- **Requisito:** RQF15
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario añade un comentario a una publicación existente.
- **Precondiciones:** El usuario no está baneado.
- **Postcondiciones:** El comentario queda registrado y el contador de comentarios de la publicación se incrementa.
- **Secuencia normal:**
  1. El usuario abre el detalle de la publicación.
  2. El usuario escribe el texto en el campo de comentario.
  3. El usuario pulsa Enviar.
  4. El sistema valida y persiste el comentario.
  5. El cliente añade el comentario a la lista local y actualiza el contador.

- **Extensiones o flujos alternativos:** Si el texto está vacío, el sistema no envía la petición. Si el envío falla, el usuario es informado.
- **Frecuencia esperada:** Media.
- **Temas abiertos:** Ninguno

#### UC14. Eliminar publicaciones o comentarios propios:

- **Código:** UC14
- **Nombre:** Eliminar publicaciones o comentarios propios
- **Requisito:** RQF13, RQF15
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario retira contenido publicado por él.
- **Precondiciones:** El contenido pertenece al usuario que ejecuta la acción.
- **Postcondiciones:** El contenido queda marcado como eliminado y deja de mostrarse en el feed. Si era un comentario, el contador de comentarios de la publicación se decrementa.
- **Secuencia normal:**
  1. El usuario abre el menú contextual del contenido y selecciona Eliminar.
  2. El sistema confirma la acción y aplica el borrado lógico.
  3. El sistema notifica al resto de clientes mediante el canal en tiempo real.
- **Extensiones o flujos alternativos:** Si el contenido no pertenece al usuario, la operación es rechazada con un error de permiso.
- **Frecuencia esperada:** Baja.
- **Temas abiertos:** Ninguno

#### UC15. Reportar publicación o comentario:

- **Código:** UC15
- **Nombre:** Reportar publicación o comentario
- **Requisito:** RQF16
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario denuncia un contenido inadecuado.
- **Precondiciones:** El usuario tiene sesión activa.
- **Postcondiciones:** Se genera un ticket de incidencia con un número secuencial único y el contenido reportado queda asociado al ticket.
- **Secuencia normal:**
  1. El usuario selecciona "Reportar" en el menú contextual del contenido.
  2. El sistema abre el formulario de reporte con el tipo precargado.
  3. El usuario escribe la descripción de la incidencia y, opcionalmente, adjunta una imagen.
  4. El usuario pulsa Enviar.
  5. El sistema asigna un número de ticket secuencial y persiste el reporte con estado pendiente.
  6. El sistema muestra confirmación con el número de ticket asignado.

- **Extensiones o flujos alternativos:** Si el campo de descripción está vacío, el sistema rechaza la operación.
- **Frecuencia esperada:** Baja.
- **Temas abiertos:** Ninguno

#### UC16. Buscar otros usuarios:

- **Código:** UC16
- **Nombre:** Buscar otros usuarios
- **Requisito:** RQF19
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario localiza otros usuarios por nombre.
- **Precondiciones:** El usuario tiene sesión activa.
- **Postcondiciones:** Ninguna.
- **Secuencia normal:**
  1. El usuario abre el buscador desde la pestaña Comunidad.
  2. El usuario escribe un fragmento del nombre.
  3. El sistema busca usuarios cuyo nombre coincida y devuelve un listado de hasta 20 resultados (excluyendo perfiles privados para usuarios no administradores).
  4. El usuario puede pulsar un resultado para abrir el perfil.
- **Extensiones o flujos alternativos:** Si no hay resultados, se muestra un mensaje vacío.
- **Frecuencia esperada:** Media.
- **Temas abiertos:** Ninguno

#### UC17. Consultar el perfil de otro usuario:

- **Código:** UC17
- **Nombre:** Consultar el perfil de otro usuario
- **Requisito:** RQF19
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario accede al perfil público de otro usuario.
- **Precondiciones:** El usuario destino existe y no ha sido eliminado.
- **Postcondiciones:** Ninguna.
- **Secuencia normal:**
  1. El usuario pulsa el avatar o el nombre del usuario destino.
  2. El sistema carga el perfil con banner, foto, biografía y publicaciones del usuario.
  3. Si el perfil es privado, se muestra un mensaje explícito sin publicaciones y el botón de mensaje queda oculto.
- **Extensiones o flujos alternativos:** Si el usuario destino ha sido eliminado, se muestra una notificación y no se navega.
- **Frecuencia esperada:** Media.
- **Temas abiertos:** Ninguno

**UC18. Iniciar una conversación privada con otro usuario:**

- **Código:** UC18
- **Nombre:** Iniciar una conversación privada con otro usuario
- **Requisito:** RQF17
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario abre o crea una conversación 1:1 con otro usuario.
- **Precondiciones:** El usuario destino existe y no ha vetado nuevas conversaciones por tener el perfil privado (salvo que ya exista una conversación previa).
- **Postcondiciones:** Existe una conversación entre ambos usuarios y queda navegada en la pantalla de chat.
- **Secuencia normal:**
  1. Desde el perfil del usuario destino, el usuario pulsa "Enviar mensaje".
  2. El sistema busca una conversación previa entre ambos; si no existe, crea una nueva.
  3. El sistema redirige a la pantalla de chat con la conversación cargada.
- **Extensiones o flujos alternativos:** Si el usuario destino tiene perfil privado y no existe conversación previa, el sistema rechaza la operación con un error de permiso.
- **Frecuencia esperada:** Baja.
- **Temas abiertos:** Ninguno

**UC19. Enviar y recibir mensajes en una conversación:**

- **Código:** UC19
- **Nombre:** Enviar y recibir mensajes en una conversación
- **Requisito:** RQF17, RQF18
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario intercambia mensajes con el otro participante de una conversación.
- **Precondiciones:** El usuario es uno de los participantes de la conversación y no está baneado para escritura.
- **Postcondiciones:** El mensaje queda persistido y se entrega al destinatario en tiempo real si está conectado, o mediante notificación push si no lo está.
- **Secuencia normal:**
  1. El usuario abre la conversación.
  2. El sistema carga los últimos mensajes ordenados cronológicamente.
  3. El usuario escribe un mensaje y pulsa Enviar (Enter).
  4. El sistema persiste el mensaje con estado "enviado".
  5. El sistema actualiza el estado a "entregado" cuando el destinatario está conectado, o emite una notificación push si está fuera de la aplicación.
  6. Cuando el destinatario abre la conversación, el sistema marca los mensajes como "leído" y emite la actualización al emisor.

- **Extensiones o flujos alternativos:** Si el usuario está baneado, el campo de entrada se deshabilita y se muestra el periodo del baneo. Si la conversación es con un usuario eliminado, el campo de entrada se sustituye por un aviso de solo lectura.
- **Frecuencia esperada:** Alta.
- **Temas abiertos:** Ninguno

**UC20. Consultar la lista de notificaciones internas:**

- **Código:** UC20
- **Nombre:** Consultar la lista de notificaciones internas
- **Requisito:** RQF25
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario consulta todas las notificaciones internas generadas por la aplicación.
- **Precondiciones:** El usuario tiene sesión activa.
- **Postcondiciones:** Ninguna.
- **Secuencia normal:**
  1. El usuario accede a la sección de notificaciones internas
  2. El sistema carga las notificaciones del usuario.
  3. El sistema muestra el listado con iconos y mensajes diferenciados por tipo y un indicador de no leídas.
- **Extensiones o flujos alternativos:** Si no hay notificaciones, se muestra un mensaje de estado vacío.
- **Frecuencia esperada:** Media.
- **Temas abiertos:** Ninguno

**UC21. Gestionar notificaciones (marcar leídas, eliminar):**

- **Código:** UC21
- **Nombre:** Gestionar notificaciones (marcar leídas, eliminar)
- **Requisito:** RQF25
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario marca como leídas o elimina notificaciones de forma individual o masiva.
- **Precondiciones:** El usuario tiene al menos una notificación.
- **Postcondiciones:** Las notificaciones seleccionadas quedan marcadas como leídas o eliminadas. El contador de no leídas se actualiza.
- **Secuencia normal:**
  1. El usuario selecciona una o varias notificaciones con la casilla de cada fila, o pulsa "Todo" para operar sobre todas.
  2. El usuario elige Marcar como leídas o Eliminar.
  3. El sistema aplica la acción y refresca la lista.
- **Extensiones o flujos alternativos:** Si la acción afecta a notificaciones ya eliminadas en otra sesión, el sistema ignora la operación y sincroniza el estado.
- **Frecuencia esperada:** Media.

- **Temas abiertos:** Ninguno

#### UC22. Editar el perfil personal:

- **Código:** UC22
- **Nombre:** Editar el perfil personal
- **Requisito:** RQF20
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario actualiza su foto, banner, biografía y ubicación.
- **Precondiciones:** El usuario tiene sesión activa.
- **Postcondiciones:** Los datos actualizados quedan persistidos y se reflejan en el perfil público del usuario.
- **Secuencia normal:**
  1. El usuario abre la edición de perfil desde su pestaña Perfil.
  2. El usuario modifica los campos deseados; las imágenes se suben al servicio externo y se obtiene una URL pública.
  3. El usuario selecciona la ubicación de una lista de capitales de provincia o la escribe manualmente.
  4. El usuario pulsa Guardar. 5. El sistema persiste los cambios y retorna al perfil.
- **Extensiones o flujos alternativos:** Si la subida de imagen falla, se mantiene la anterior. Si se introduce una ubicación libre sin coordenadas, las funciones meteorológicas pueden no operar para ese usuario.
- **Frecuencia esperada:** Baja.
- **Temas abiertos:** Ninguno

#### UC23. Cambiar la contraseña:

- **Código:** UC23
- **Nombre:** Cambiar la contraseña
- **Requisito:** RQF21
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario reemplaza su contraseña por una nueva.
- **Precondiciones:** El usuario conoce su contraseña actual.
- **Postcondiciones:** La contraseña queda actualizada en la base de datos con su nuevo hash.
- **Secuencia normal:**
  1. El usuario accede a la pantalla de gestión de cuenta.
  2. El usuario introduce la contraseña actual y la nueva contraseña (con confirmación).
  3. El sistema verifica la contraseña actual frente al hash almacenado.
  4. El sistema reemplaza el hash con el de la nueva contraseña y devuelve confirmación.
  5. El sistema muestra una confirmación visual al usuario.
- **Extensiones o flujos alternativos:** Si la contraseña actual es incorrecta, el sistema responde con un error y no aplica el cambio. Si la nueva contraseña no cumple los requisitos mínimos, se rechaza.

- **Frecuencia esperada:** Baja.
- **Temas abiertos:** Ninguno

#### UC24. Exportar los datos personales:

- **Código:** UC24
- **Nombre:** Exportar los datos personales
- **Requisito:** RQF22
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario descarga un archivo con todos sus datos personales para ejercer su derecho a la portabilidad.
- **Precondiciones:** El usuario tiene sesión activa.
- **Postcondiciones:** Se entrega al usuario un archivo JSON con su perfil, sus plantas y los metadatos asociados.
- **Secuencia normal:**
  1. El usuario pulsa "Exportar datos" en la gestión de cuenta.
  2. El sistema recopila el perfil del usuario y todas sus plantas activas.
  3. El sistema entrega los datos como descarga en el navegador o en el sistema operativo.
- **Extensiones o flujos alternativos:** Si la entrega falla por motivos de plataforma, se muestra un mensaje al usuario.
- **Frecuencia esperada:** Baja.
- **Temas abiertos:** Ninguno

#### UC25. Eliminar la cuenta:

- **Código:** UC25
- **Nombre:** Eliminar la cuenta
- **Requisito:** RQF23
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario solicita la eliminación de su cuenta.
- **Precondiciones:** El usuario conoce su contraseña actual.
- **Postcondiciones:** La cuenta queda marcada como eliminada (borrado lógico). Si el usuario eligió no preservar contenido, sus publicaciones y comentarios también quedan marcados como eliminados; en caso contrario, permanecen visibles pero atribuidos a un usuario eliminado.
- **Secuencia normal:**
  1. El usuario accede a la gestión de cuenta y pulsa "Eliminar cuenta".
  2. El usuario decide si desea preservar su contenido público y confirma con su contraseña.
  3. El sistema valida la contraseña y aplica la baja en cascada.
  4. El cliente cierra la sesión y redirige al inicio de sesión.
- **Extensiones o flujos alternativos:** Si la contraseña es incorrecta, la operación se cancela.
- **Frecuencia esperada:** Muy baja.
- **Temas abiertos:** Ninguno

**UC26. Configurar las preferencias:**

- **Código:** UC26
- **Nombre:** Configurar las preferencias
- **Requisito:** RQNF11
- **Versión:** 1.0
- **Actor principal:** USU2 – Usuario registrado
- **Personal involucrado e intereses:** USU2
- **Descripción:** El usuario ajusta sus preferencias de notificaciones push y privacidad.
- **Precondiciones:** El usuario tiene sesión activa.
- **Postcondiciones:** Las preferencias quedan persistidas y se aplican de inmediato (los perfiles privados desaparecen del feed público y no aceptan nuevas conversaciones; las notificaciones push se omiten cuando el usuario las ha desactivado).
- **Secuencia normal:**
  1. El usuario abre la pantalla de ajustes.
  2. El usuario activa o desactiva los interruptores de notificaciones push y perfil público.
  3. El sistema persiste el cambio y refresca el estado global del usuario.
- **Extensiones o flujos alternativos:** Si la actualización falla, el cambio visual se revierte.
- **Frecuencia esperada:** Baja.
- **Temas abiertos:** Ninguno

**UC27. Acceder al panel de administración:**

- **Código:** UC27
- **Nombre:** Acceder al panel de administración
- **Requisito:** RQF26
- **Versión:** 1.0
- **Actor principal:** USU3 – Administrador
- **Personal involucrado e intereses:** USU3
- **Descripción:** El administrador entra en el panel administrativo desde su perfil o desde los ajustes.
- **Precondiciones:** El usuario tiene rol administrador.
- **Postcondiciones:** Se muestra el panel principal de administración con accesos a las distintas funciones.
- **Secuencia normal:**
  1. El administrador pulsa "Panel de administración" desde Perfil o Ajustes.
  2. El sistema verifica el rol y carga el panel con los accesos disponibles, el contador de reportes pendientes y la información de diagnóstico del sistema.
- **Extensiones o flujos alternativos:** Si el usuario perdió privilegios entre sesiones, se muestra un error de permiso.
- **Frecuencia esperada:** Baja.
- **Temas abiertos:** Ninguno

**UC28. Buscar usuarios desde el panel administrativo:**

- **Código:** UC28
- **Nombre:** Buscar usuarios desde el panel administrativo

- **Requisito:** RQF19, RQF26
- **Versión:** 1.0
- **Actor principal:** USU3 – Administrador
- **Personal involucrado e intereses:** USU3
- **Descripción:** El administrador localiza usuarios por nombre o correo, sin la restricción de privacidad que afecta a los usuarios estándar.
- **Precondiciones:** El administrador tiene sesión activa.
- **Postcondiciones:** Se muestra una lista de hasta 20 usuarios coincidentes con la consulta.
- **Secuencia normal:**
  1. El administrador introduce la consulta en el buscador del panel.
  2. El sistema aplica el filtro sobre todos los usuarios (incluyendo perfiles privados).
  3. El sistema devuelve los resultados con foto, nombre y datos básicos.
- **Extensiones o flujos alternativos:** Si no hay coincidencias, se muestra un listado vacío.
- **Frecuencia esperada:** Baja.
- **Temas abiertos:** Ninguno

#### UC29. Gestionar reportes de incidencia:

- **Código:** UC29
- **Nombre:** Gestionar reportes de incidencia
- **Requisito:** RQF27
- **Versión:** 1.0
- **Actor principal:** USU3 – Administrador
- **Personal involucrado e intereses:** USU3
- **Descripción:** El administrador consulta y actualiza el estado de los reportes recibidos.
- **Precondiciones:** Existe al menos un reporte registrado.
- **Postcondiciones:** El reporte queda con estado actualizado (resuelto, desestimado o pendiente) y registra el administrador que lo resolvió.
- **Secuencia normal:**
  1. El administrador abre la lista de reportes.
  2. El administrador filtra por número de ticket, estado o rango de fechas.
  3. El administrador selecciona un reporte y consulta el contenido reportado.
  4. El administrador elige Resolver, Desestimar o Reabrir y el sistema actualiza el estado.
- **Extensiones o flujos alternativos:** Si el contenido reportado ya no existe, se muestra un aviso pero el reporte sigue siendo gestionable.
- **Frecuencia esperada:** Media.
- **Temas abiertos:** Ninguno

#### UC30. Eliminar contenido como moderador:

- **Código:** UC30
- **Nombre:** Eliminar contenido como moderador
- **Requisito:** RQF26
- **Versión:** 1.0
- **Actor principal:** USU3 – Administrador
- **Personal involucrado e intereses:** USU3

- **Descripción:** El administrador elimina una publicación o un comentario como moderador.
- **Precondiciones:** El contenido existe.
- **Postcondiciones:** El contenido queda marcado como eliminado por moderación. El sistema notifica al autor original y, si el elemento es un comentario, decrementa el contador de la publicación padre.
- **Secuencia normal:**
  1. El administrador abre el contenido (por ejemplo, desde un reporte).
  2. El administrador pulsa "Eliminar" en el menú contextual.
  3. El sistema confirma la acción, aplica el borrado lógico, genera una notificación interna al autor y emite la actualización del feed.
- **Extensiones o flujos alternativos:** Ninguno relevante.
- **Frecuencia esperada:** Baja.
- **Temas abiertos:** Ninguno

#### UC31. Banear temporalmente a un usuario:

- **Código:** UC31
- **Nombre:** Banear temporalmente a un usuario
- **Requisito:** RQF26
- **Versión:** 1.0
- **Actor principal:** USU3 – Administrador
- **Personal involucrado e intereses:** USU3
- **Descripción:** El administrador aplica un baño temporal a un usuario por una duración determinada.
- **Precondiciones:** El usuario destino existe.
- **Postcondiciones:** El usuario destino queda baneado hasta la fecha calculada. Mientras tanto, no puede publicar, comentar ni enviar mensajes, pero sí puede leer.
- **Secuencia normal:**
  1. El administrador abre el menú administrativo en el perfil del usuario destino.
  2. El administrador elige "Banear temporalmente" y selecciona la duración (1, 3, 7 o 30 días).
  3. El sistema calcula la fecha de fin del baño y persiste el cambio.
  4. El sistema genera una notificación de aviso administrativo para el usuario afectado.
- **Extensiones o flujos alternativos:** El administrador no puede banearse a sí mismo.
- **Frecuencia esperada:** Baja.
- **Temas abiertos:** Ninguno

#### UC32. Enviar un aviso formal a un usuario:

- **Código:** UC32
- **Nombre:** Enviar un aviso formal a un usuario
- **Requisito:** RQF26
- **Versión:** 1.0
- **Actor principal:** USU3 – Administrador
- **Personal involucrado e intereses:** USU3
- **Descripción:** El administrador envía un mensaje formal al usuario destino sin aplicar un baño.
- **Precondiciones:** El usuario destino existe.
- **Postcondiciones:** Se genera una notificación de tipo aviso administrativo para el usuario destino.
- **Secuencia normal:**

1. El administrador abre el menú administrativo en el perfil del usuario destino.
  2. El administrador escribe el mensaje del aviso.
  3. El sistema persiste la notificación y la entrega al usuario destino por todos los canales activos.
- **Extensiones o flujos alternativos:** Si el mensaje está vacío, el sistema lo rechaza.
  - **Frecuencia esperada:** Baja.
  - **Temas abiertos:** Ninguno

#### UC33. Consultar y restaurar elementos eliminados:

- **Código:** UC33
- **Nombre:** Consultar y restaurar elementos eliminados
- **Requisito:** RQF28
- **Versión:** 1.0
- **Actor principal:** USU3 – Administrador
- **Personal involucrado e intereses:** USU3
- **Descripción:** El administrador consulta los elementos eliminados por moderación y restaura los que considere apropiados.
- **Precondiciones:** Existe al menos un elemento eliminado por moderación.
- **Postcondiciones:** El elemento restaurado vuelve a ser visible y, en el caso de comentarios, el contador de la publicación padre se reincrementa.
- **Secuencia normal:**
  1. El administrador abre la pantalla de elementos eliminados.
  2. El sistema lista las plantas, publicaciones, comentarios y usuarios eliminados por moderación, agrupados por tipo.
  3. El administrador pulsa "Restaurar" en el elemento deseado.
  4. El sistema revierte el borrado y notifica al propietario original.
- **Extensiones o flujos alternativos:** Los elementos eliminados como consecuencia de la baja de cuenta del propio usuario no son restaurables y se indica explícitamente.
- **Frecuencia esperada:** Baja.
- **Temas abiertos:** Ninguno

#### UC34. Ejecutar manualmente el proceso de recordatorios:

- **Código:** UC34
- **Nombre:** Ejecutar manualmente el proceso de recordatorios
- **Requisito:** RQF29
- **Versión:** 1.0
- **Actor principal:** USU3 – Administrador
- **Personal involucrado e intereses:** USU3
- **Descripción:** El administrador dispara el proceso de recordatorios sin esperar a la ejecución programada de medianoche.
- **Precondiciones:** El administrador tiene sesión activa.
- **Postcondiciones:** El proceso ejecuta los seis subprocesos (clima, lluvia confirmada, poda, cosecha, recordatorios pendientes, "todo al día") y genera las notificaciones correspondientes.
- **Secuencia normal:**

1. El administrador pulsa "Ejecutar cron" en el panel.
  2. El sistema solicita confirmación.
  3. El administrador confirma y el sistema lanza el proceso.
  4. El sistema devuelve un resumen con número de notificaciones generadas por subproceso.
- **Extensiones o flujos alternativos:** Si el cron ya se está ejecutando (lock activo), la nueva invocación se omite sin error.
  - **Frecuencia esperada:** Baja.
  - **Temas abiertos:** Ninguno

#### UC35. Simular eventos climáticos:

- **Código:** UC35
- **Nombre:** Simular eventos climáticos
- **Requisito:** RQF29
- **Versión:** 1.0
- **Actor principal:** USU3 – Administrador
- **Personal involucrado e intereses:** USU3
- **Descripción:** El administrador genera notificaciones simuladas de lluvia o tormenta para todas las plantas exteriores.
- **Precondiciones:** Existe al menos una planta exterior con coordenadas.
- **Postcondiciones:** Se generan notificaciones simuladas para cada planta exterior y se envían por los canales activos.
- **Secuencia normal:**
  1. El administrador pulsa "Simular lluvia" o "Simular tormenta" en el panel.
  2. El sistema solicita confirmación.
  3. El administrador confirma.
  4. El sistema itera las plantas exteriores y crea las notificaciones correspondientes.
- **Extensiones o flujos alternativos:** Si no hay plantas exteriores, la operación no genera notificaciones y se informa al administrador.
- **Frecuencia esperada:** Muy baja.
- **Temas abiertos:** Ninguno

### 3.5. Modelo de clases de análisis

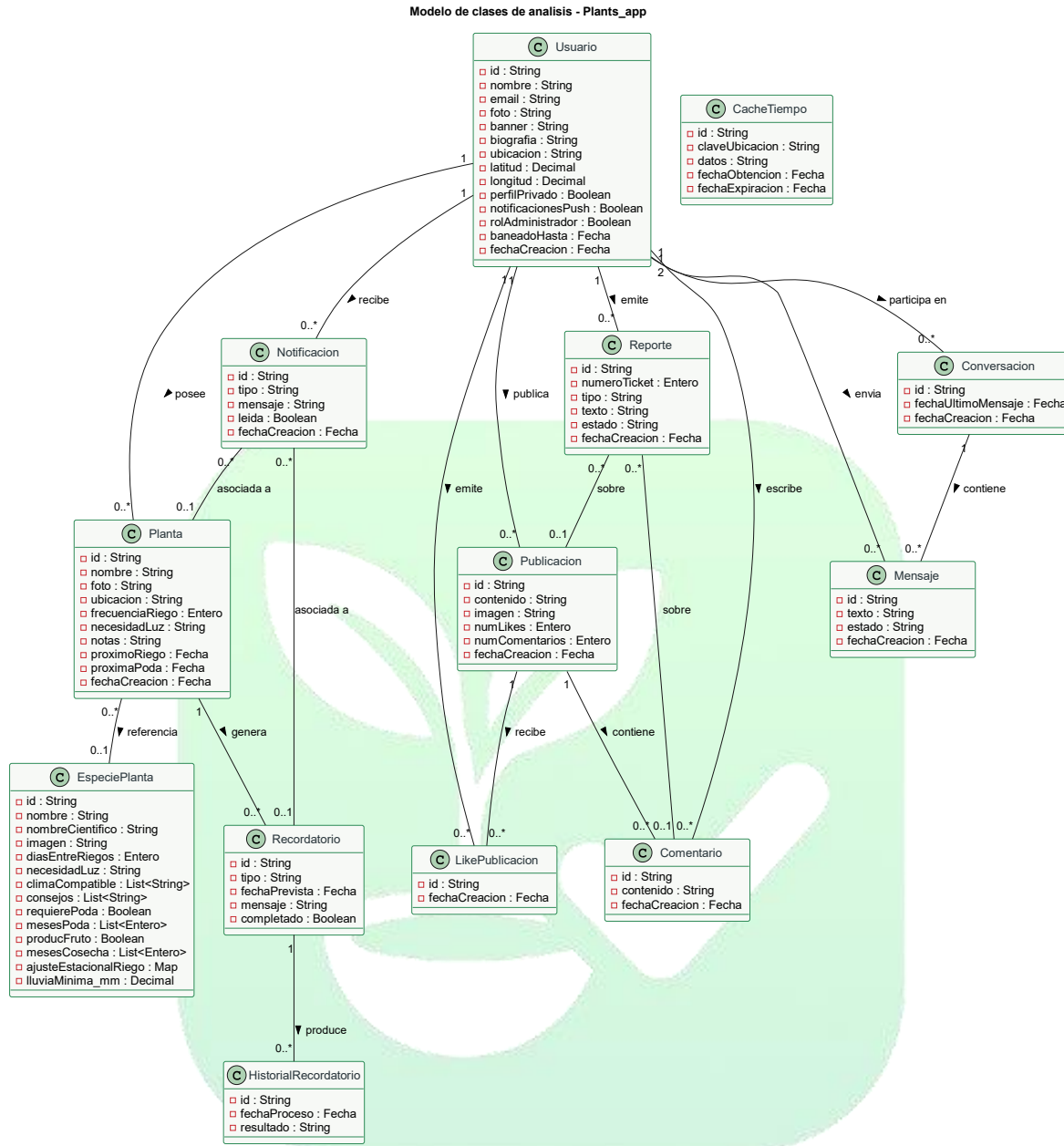
El modelo de clases de análisis representa los conceptos del dominio que el sistema gestiona, con sus atributos esenciales y sus relaciones, sin entrar todavía en detalles técnicos como el motor de persistencia, los métodos de serialización o la organización por capas (esos aspectos se abordan en el diseño, sección 4.2). El propósito de este modelo es transmitir, con vocabulario neutro respecto a la tecnología, qué información maneja la aplicación y cómo se vinculan los conceptos entre sí.

Las clases identificadas son:

- **Usuario:** representa a una persona registrada en la aplicación. Contiene los datos de perfil, las preferencias y la información de baneo cuando aplique.
- **Planta:** cada planta registrada por un usuario. Pertenecce a un usuario y, opcionalmente, a una especie del catálogo.
- **EspeciePlanta:** catálogo de referencia con las características de cuidado de cada especie de planta (frecuencia base de riego, necesidad de luz, ajustes estacionales, meses de poda y cosecha).
- **Recordatorio:** tarea futura asociada a una planta (riego, poda o cosecha).
- **HistorialRecordatorio:** registro de los recordatorios efectivamente procesados, usado para garantizar la idempotencia y evitar notificar dos veces el mismo evento.
- **Publicación:** contenido publicado en el feed de la comunidad.
- **LikePublicacion:** marca de "me gusta" emitida por un usuario sobre una publicación.
- **Comentario:** respuesta textual de un usuario a una publicación.
- **Conversación:** canal privado entre dos usuarios.
- **Mensaje:** cada uno de los mensajes intercambiados dentro de una conversación.
- **Notificación:** aviso interno mostrado al usuario en la pestaña de avisos. Puede provenir del procesamiento de recordatorios, de una alerta climática, de una acción administrativa o de un mensaje informativo.
- **Reporte:** ticket de incidencia generado por un usuario al denunciar contenido inadecuado.
- **CacheTiempo:** estructura de almacenamiento temporal de los datos meteorológicos por ubicación, con el fin de reducir el número de peticiones a servicios externos.

Las relaciones más relevantes son:

- Un Usuario tiene cero o más Plantas; cada Planta pertenece a un único Usuario.
- Una Planta hace referencia opcionalmente a una EspeciePlanta del catálogo.
- Una Planta tiene cero o más Recordatorios; cada Recordatorio genera registros en HistorialRecordatorio cuando se procesa.
- Un Usuario puede crear muchas Publicaciones, recibir y emitir muchos LikePublicacion y publicar muchos Comentarios.
- Una Publicacion tiene cero o más Comentarios y cero o más LikePublicacion.
- Dos Usuarios participan en una Conversacion común; una Conversacion contiene muchos Mensajes ordenados cronológicamente.
- Un Usuario recibe muchas Notificaciones, opcionalmente vinculadas a una Planta o a un Recordatorio.
- Un Usuario puede emitir muchos Reportes; un Reporte se vincula opcionalmente a una Publicacion o a un Comentario.



### 3.6. Interfaces de usuario

Este apartado describe la organización general de la interfaz de usuario de la aplicación. Se divide en dos sub-apartados: el primero recoge los aspectos comunes que aparecen en la mayoría de las pantallas (estructura de navegación, paleta corporativa, comportamiento responsive, elementos transversales como banners de conexión y mensajes de error) y el segundo enumera el conjunto completo de pantallas con una descripción funcional de cada una. Los mockups visuales con capturas reales de la aplicación se presentan más adelante, en el apartado 4.4 del DSI.

---

### 3.6.1. Aspectos comunes de la interfaz de usuario

La interfaz de usuario se construye con un sistema de diseño común que se aplica a todas las pantallas. Sus pilares principales son:

**Paleta corporativa:** La aplicación utiliza una paleta de colores centrada en el verde (verde bosque #2E8B57 como primario, verde claro #7BC47F como secundario y amarillo cálido #FFD166 como acento) sobre fondos neutros suaves (blanco roto #F7F9F6 y gris suave verdoso #E8EDE6 para superficies). El texto se renderiza siempre en gris antracita #263238 para el contenido principal y gris medio #607D6B para subtítulos y metadatos, garantizando un contraste suficiente para cumplir el nivel AA de las pautas WCAG. Los colores semánticos están reservados a estados concretos: verde brillante #38C172 para confirmaciones, ámbar #F0A030 para precaución, azul claro #4AA3FF para información y rojo suave #E04F5F para errores. Ningún color se utiliza de forma decorativa fuera de su rol semántico.

**Navegación principal adaptativa:** Tras la autenticación, la aplicación organiza su contenido en seis pestañas: Mis Plantas, Comunidad, Mensajes, Avisos, Calendario y Perfil. En pantallas anchas (navegador web o tabletas en orientación horizontal, lado más corto mayor o igual a 600 píxeles independientes de densidad) se muestra una barra lateral con iconos y etiquetas alineadas a la izquierda. En móviles en orientación vertical se sustituye por una barra inferior con iconos sin texto, optimizando el espacio vertical para el contenido. En móviles en orientación horizontal se utiliza una barra lateral compacta con solo iconos.

**Indicadores transversales:** En la parte superior de las pantallas autenticadas se muestra un indicador discreto cuando el dispositivo pierde la conexión a internet, junto con un mensaje breve. Las pestañas de Mensajes y Avisos pueden mostrar un indicador rojo cuando contienen elementos no leídos, calculado de forma reactiva a partir del estado actual.

**Tipografía y componentes:** La aplicación utiliza la tipografía predeterminada del sistema (Material 3 / Cupertino según la plataforma) con tamaños 12 a 18 puntos para el cuerpo y hasta 24 puntos para los encabezados. Los componentes habituales (botones, campos de entrada, tarjetas, listas, diálogos de confirmación) se basan en los widgets estándar de Material 3 personalizados con la paleta corporativa.

**Estados de la aplicación:** Las pantallas que cargan datos muestran un indicador de progreso circular o esqueletos de contenido mientras esperan la respuesta del servidor. Los errores se centralizan en tres patrones: mensajes transitorios en la parte inferior para errores recuperables, banners incrustados con botón de reintento para errores que afectan a una sección concreta y diálogos modales bloqueantes para decisiones irreversibles (eliminar cuenta, salir sin guardar). Durante el arranque inicial, si el primer contacto con el servidor tarda más de cinco segundos, la pantalla de inicio muestra un mensaje contextual que explica que el servicio se está activando.

---

### 3.6.2. Especificación de pantallas y ventanas

A continuación, se describen una a una todas las pantallas de la aplicación, agrupadas por sección funcional. Cada descripción enumera de forma narrativa qué contiene la pantalla y qué acciones puede realizar el usuario en ella.

**Stack no autenticado:**

**Pantalla de inicio (Splash):** Muestra el logotipo de la aplicación y un indicador circular de carga. Mientras se carga, comprueba el estado de la sesión local (token guardado en el almacén seguro), refresca el token si está

próximo a caducar y deriva al usuario a la pantalla principal si la sesión es válida o a la pantalla de inicio de sesión en caso contrario. Si la primera petición al servidor tarda más de cinco segundos, se complementa con un mensaje contextual; a los 25 segundos el mensaje se amplía explicando que el servicio se está despertando.

**Inicio de sesión (Login):** Contiene el logotipo, dos campos (correo electrónico y contraseña), un botón de iniciar sesión y un enlace para crear una cuenta. Valida en cliente el formato y muestra los errores devueltos por el servidor mediante mensajes claros.

**Registro:** Presenta un formulario con cuatro campos (nombre, correo, contraseña y confirmación de contraseña), un botón de registro y un enlace para volver al inicio de sesión si el usuario ya dispone de cuenta. Aplica validaciones de longitud y formato en cliente y muestra los errores del servidor (correo ya en uso) en línea.

**Bienvenida (Welcome):** Carrusel de cuatro pantallas de onboarding que se muestra una sola vez tras el primer registro. Cada pantalla describe una funcionalidad principal con un icono y un texto breve. Concluye con un botón para acceder a la pantalla principal.

#### **Pestaña Mis Plantas:**

**Lista de plantas (PlantsListPage):** Pantalla principal del tab. Encabezada por una barra superior con el título "Mis Plantas" y varios iconos de acción: acceso directo al calendario, buscador en línea, filtro por especie y filtro por ciudad (cuando hay plantas con ubicación). Bajo la barra, una rejilla de tarjetas con la foto, el nombre y el próximo riego de cada planta. Cuando no hay plantas se muestra un estado vacío con instrucciones. Un botón flotante en la esquina inferior abre el formulario de nueva planta.

**Detalle de planta (PlantDetailPage):** Muestra la foto a tamaño completo y, en el cuerpo, el nombre personalizado, la especie (con un enlace que abre una ventana emergente con información ampliada), la ubicación (interior o exterior), los próximos recordatorios de riego y poda, las notas del usuario y los botones de Editar, Eliminar y "Regar hoy". Si la especie tiene definido un ajuste estacional y la estación actual aplica un factor distinto a 1.0, se muestra un desglose explicativo del cálculo de riego.

**Formulario de planta (PlantFormPage):** Sirve tanto para crear como para editar. Incluye el campo de nombre, un autocompletado de especie sobre el catálogo, un selector de foto, un selector de ubicación (interior o exterior) y un autocompletado de ciudad sobre el catálogo de capitales de provincia. Solo en modo edición se exponen los campos de frecuencia de riego y necesidad de luz, ya que en modo creación esos valores se rellenan automáticamente al seleccionar la especie. La barra superior incluye el botón de guardar.

**Información de la especie (ventana emergente SpeciesInfoSheet):** Aparece como hoja deslizante desde el detalle de planta. Muestra el nombre científico, los requisitos de cuidado (riego, luz, temperatura), los chips de clima compatible, los consejos de cultivo y, cuando aplique, las indicaciones de poda y cosecha.

#### **Pestaña Comunidad:**

**Feed de la comunidad (CommunityFeedPage):** Encabezada por una barra superior con el título "Comunidad" y un icono de búsqueda de usuarios. Bajo la barra, un listado de publicaciones con desplazamiento infinito; cada tarjeta muestra la foto y el nombre del autor, la fecha, la imagen opcional, el texto, el contador de "me gusta" y el contador de comentarios, junto con un menú contextual con tres puntos para editar, eliminar o reportar. Un botón flotante abre el formulario de nueva publicación.

**Detalle de publicación (PostDetailPage):** Muestra la publicación completa, la lista de comentarios bajo ella y un campo de entrada al final para añadir un nuevo comentario. Cada comentario muestra el avatar y el nombre del autor (clicables) y, si llega desde un reporte, un identificador del ticket asociado.

**Crear publicación (CreatePostPage):** Formulario con un selector de foto, una previsualización de la imagen seleccionada, un campo de texto multilínea para el contenido y los botones de Publicar y Cancelar.

**Perfil de otro usuario (UserProfilePage):** Cabecera con banner, avatar superpuesto, nombre del usuario, contador de publicaciones y, si no es el propio perfil, un botón de "Enviar mensaje". Si el perfil es privado, se muestra un mensaje explícito y se oculta el botón. Bajo la cabecera, dos pestañas separadas por publicaciones con foto y publicaciones de solo texto, con sus respectivos listados.

#### **Pestaña Mensajes:**

**Lista de conversaciones (ConversationsListPage):** Listado de las conversaciones del usuario con la foto y el nombre del interlocutor, una previsualización del último mensaje, su marca de tiempo y un indicador si la conversación contiene mensajes sin leer.

**Conversación (ChatPage):** Cabecera con la foto, el nombre del interlocutor y un indicador de presencia. Cuerpo con las burbujas de los mensajes alineadas a izquierda (del interlocutor) o derecha (propias), con iconos diferenciados según el estado del mensaje (pendiente, enviado, entregado o leído). Al pie, un campo de entrada multilínea con botón de enviar; en móviles, la tecla Intro envía el mensaje y Ctrl+Intro inserta un salto de línea. Si el usuario está baneado, el campo se sustituye por un mensaje informativo sin posibilidad de escritura.

#### **Pestaña Avisos:**

**Lista de avisos (NotificationsPage):** Encabezada por una barra superior con el título "Avisos" y dos acciones globales (marcar todo como leído, eliminar todo). Lista las notificaciones internas con un icono diferenciado según el tipo, el texto del mensaje y una casilla de selección al final de cada fila. Al seleccionar una o varias notificaciones, la barra superior cambia a un modo contextual que muestra el número de elementos seleccionados y dos acciones (marcar seleccionados como leídos, eliminar seleccionados). Las notificaciones sin leer se distinguen visualmente.

#### **Pestaña Calendario:**

**Calendario de tareas (CalendarPage):** Muestra un calendario mensual con los días del mes ocupando la mayor parte de la pantalla. Cada día con tareas previstas exhibe puntos de color según el tipo (azul para riego, ámbar para poda, verde para cosecha). Al seleccionar un día, se despliega bajo el calendario una lista con todos los eventos planificados, cada uno con su icono, su descripción y la planta a la que se refiere. Una leyenda inferior identifica el significado de cada color.

#### **Pestaña Perfil:**

**Mi perfil (MyProfilePage):** Cabecera con banner, avatar, nombre, correo, biografía, ubicación y contador de publicaciones. Incluye los botones de editar perfil, de acceso al panel administrativo (solo administradores) y de cierre de sesión. Bajo la cabecera, dos pestañas con las publicaciones propias agrupadas por presencia de foto.

**Editar perfil (EditProfilePage):** Formulario con selectores de banner y avatar (que se suben al servicio externo de imágenes), un campo de nombre, un campo multilínea de biografía y un autocompletado de ciudad. Botón de guardar en la barra superior.

**Ajustes (SettingsPage):** Agrupa interruptores para las notificaciones push y para la privacidad del perfil (público o privado), accesos a la pantalla de gestión de cuenta y a la pantalla de reporte de incidencias, información sobre la versión de la aplicación y, para administradores, un acceso al panel de administración.

**Gestión de cuenta (AccountManagementPage):** Tres bloques: cambio de contraseña (contraseña actual, nueva contraseña, confirmación), exportación de datos personales en formato JSON y eliminación de cuenta (con un interruptor para preservar el contenido público de forma anónima y un diálogo de confirmación con introducción de contraseña).

**Formulario de reporte (ReportFormPage):** Pantalla accesible desde el menú contextual de publicaciones y comentarios o desde los ajustes. Contiene un selector de tipo (general, publicación, comentario), un campo de texto multilínea para describir la incidencia y un selector opcional de imagen.

#### Subsección de administración:

**Panel de administración (AdminDashboardPage):** Lista de tarjetas con accesos al panel de reportes de incidencias (incluyendo un indicador con el número de reportes pendientes) y a los elementos eliminados. Muestra una tarjeta de diagnóstico con información del sistema y una sección de acciones con los botones para ejecutar manualmente el proceso de recordatorios y simular eventos climáticos.

**Reportes de incidencias (IncidentReportsPage):** Listado de reportes con una barra superior de filtros (número de ticket, rango de fechas, estado mediante chips), cada uno mostrando el número de ticket en formato INC-XXX, el tipo, el texto, la fecha, la insignia de estado, el administrador que lo resolvió cuando aplique y los botones de Resolver, Desestimar o Reabrir según el estado.

**Elementos eliminados (DeletedItemsPage):** Listado de los elementos eliminados por moderación, agrupados por tipo (publicaciones, comentarios, plantas, usuarios). Cada elemento muestra su tipo, su identificador, su propietario, la fecha de eliminación y un botón de Restaurar. Los elementos eliminados como consecuencia de la baja voluntaria de cuenta no son restaurables.

## 4. Diseño de los sistemas de información

### 4.1. Diseño de la arquitectura del sistema

El diseño de la arquitectura del sistema parte de los requisitos identificados en el EVS y de las alternativas tecnológicas justificadas en el apartado 1.5. Su objetivo es definir cómo se estructuran las capas del sistema, qué tecnologías cubren cada capa y qué reglas de diseño se aplican para garantizar la mantenibilidad, la testabilidad y la separación de responsabilidades.

Plants\_app sigue un modelo cliente-servidor de tres capas (frontend, backend, base de datos) complementado con tres servicios externos integrados (meteorología, almacenamiento de imágenes, notificaciones push). El frontend adopta el patrón MVVM con inyección de dependencias mediante un contenedor; el backend organiza el código siguiendo Clean Architecture estricta, con dependencias dirigidas siempre hacia el dominio y con interfaces explícitas para cada caso de uso y para cada mapeador entre capas.

Los apartados siguientes desarrollan el detalle del entorno tecnológico y el catálogo de requisitos de diseño que han guiado las decisiones técnicas concretas.

---

#### 4.1.1. Descripción general del entorno tecnológico del sistema

A continuación se desarrolla con mayor profundidad el stack tecnológico empleado, indicando para cada capa el lenguaje, los frameworks principales, las librerías auxiliares y el rol que desempeñan dentro del sistema.

##### Frontend (cliente multiplataforma):

- **Lenguaje:** Dart 3.x.
- **Framework:** Flutter 3.x, con compilación a tres objetivos a partir del mismo código fuente: Android, iOS y aplicación web.
- **Patrón arquitectónico:** MVVM (Model-View-ViewModel) con separación estricta entre dominio (entidades, casos de uso, interfaces de repositorio y mapeadores), datos (implementaciones de repositorios, fuentes de datos locales y remotas, modelos de serialización JSON) y presentación (ViewModels que extienden ChangeNotifier, páginas y widgets).
- **Gestión de estado:** Provider 6.x sobre ChangeNotifier.
- **Inyección de dependencias:** GetIt 8.x, con un contenedor centralizado que registra singletons, factorías y servicios.
- **Cliente HTTP:** Dio 5.x con interceptores propios para autenticación, reintentos y trazas.
- **Comunicación en tiempo real:** socket\_io\_client 3.x.
- **Almacenamiento local:** flutter\_secure\_storage para el token de autenticación y datos sensibles, Hive como caché ligera y SharedPreferences para preferencias simples.
- **Notificaciones push:** firebase\_messaging 15.x complementado con flutter\_local\_notifications 19.x para mostrar las notificaciones en primer plano.
- **Imágenes:** image\_picker para selección de archivos, flutter\_image\_compress para compresión y cached\_network\_image para la entrega cacheada.
- **Otras utilidades:** intl para formato de fechas y locales, connectivity\_plus para detectar cambios de red, uuid para identificadores temporales.

##### Backend (servidor de aplicación):

- **Lenguaje:** TypeScript 5.x.
- **Runtime:** Node.js 20.x LTS.
- **Framework:** NestJS 11.x (apoyado internamente en Express).
- **Patrón arquitectónico:** Clean Architecture estricta. La capa de dominio contiene las entidades del negocio, las interfaces de los repositorios, los DTOs y las interfaces de los casos de uso. La capa de datos contiene las implementaciones de los repositorios (sobre el driver oficial de MongoDB), los esquemas y modelos de las colecciones, las fuentes externas y los mapeadores con sus interfaces. La capa de presentación contiene los controladores HTTP, el gateway de Socket.IO, los servicios técnicos, los validadores y los trabajos programados.
- **Inyección de dependencias:** InversifyJS 7.x, con un contenedor central en `core/container.ts`.
- **Persistencia:** driver oficial de MongoDB (sin Mongoose), conectado a un cluster gestionado en MongoDB Atlas. Los esquemas y los índices se definen explícitamente al arrancar la aplicación.
- **Autenticación:** JSON Web Tokens (librería `jsonwebtoken`) con expiración de 30 días y endpoint de refresco silencioso. Las contraseñas se cifran con `bcryptjs` (10 rondas de sal).

- **Validación:** class-validator y class-transformer aplicados sobre los DTOs de la capa de dominio.
- **Comunicación en tiempo real:** Socket.IO 4.x, instanciado sobre el mismo servidor HTTP que Express, sin adaptador externo (single-instance).
- **Tareas programadas:** node-cron 4.x, con un proceso diario a las 00:00 hora Europe/Madrid responsable de la generación de recordatorios, alertas climáticas y notificaciones de "todo al día".
- **Logging:** winston, con formato legible en desarrollo y JSON en producción para facilitar la indexación en la consola del proveedor de hosting.
- **Seguridad HTTP:** helmet para cabeceras seguras, cors permisivo en prototipo, express-rate-limit con límites específicos para los endpoints de autenticación, compresión y límites de tamaño de carga.

#### Base de datos:

- **Motor:** MongoDB 7.x.
- **Hosting:** MongoDB Atlas cluster M0 (Free Tier), con acceso autenticado por URI segura.
- **Operación:** lectura y escritura mediante el driver oficial; los esquemas se enforzan con la cláusula `\$jsonSchema` de MongoDB para las colecciones más críticas.

#### Servicios externos integrados:

- **WeatherAPI.com:** proxy interno del backend (endpoint `/weather`) que consulta el clima actual y el pronóstico a 48 horas a partir de unas coordenadas, con caché por clave de ubicación.
- **Cloudinary:** almacenamiento y entrega de imágenes. El backend expone el endpoint `/upload/image` que recibe el archivo, lo sube a Cloudinary y devuelve la URL pública que se persiste en la base de datos.
- **Firebase Cloud Messaging:** servicio de envío de notificaciones push. El backend invoca el SDK de Firebase Admin cada vez que necesita entregar una notificación al sistema operativo del usuario.
- **cron-job.org:** programador externo que invoca el endpoint `/health` a las 23:52 hora Europe/Madrid cada día, manteniendo el contenedor de Render activo justo antes de la ejecución del cron interno.

#### Despliegue:

- **Backend:** Render.com Free Tier (Frankfurt, Europa Central). Auto-deploy a partir de la rama principal del repositorio Git. La aplicación se compila con `npm install --include=dev && npm run build` y se arranca con `node dist/main`. La zona horaria del contenedor se fija a Europe/Madrid mediante variable de entorno.
- **Base de datos:** MongoDB Atlas cluster M0, con acceso de red abierto a las IP dinámicas del proveedor de hosting.
- **Frontend web:** servido desde la propia infraestructura del backend en su versión final o como build estático en cualquier hosting compatible.
- **Aplicación móvil:** distribuida como APK firmada para Android con keystore propio (validez 10000 días). Para iOS se generan únicamente las instrucciones de build, sin distribución en App Store.

### 4.1.2. Catálogo de requisitos de diseño

El catálogo de requisitos de diseño recoge las decisiones técnicas concretas que guían la implementación. Se divide en dos sub-tablas: requisitos del modelo de datos (RD) y requisitos de vistas (RV).

#### Requisitos del modelo de datos

| Código | Descripción                                                                                                                                                                     | Fecha       |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| RD1    | El modelo de datos será no relacional, orientado a documentos                                                                                                                   | 27 feb 2026 |
| RD2    | El motor de base de datos será MongoDB en su versión 7.                                                                                                                         | 27 feb 2026 |
| RD3    | Las contraseñas se almacenarán mediante un algoritmo de hash con sal (bcrypt, 10 rondas), nunca en claro.                                                                       | 27 feb 2026 |
| RD4    | Las imágenes (perfiles, banners, plantas, publicaciones) se almacenarán en el servicio externo Cloudinary; en la base de datos solo se persistirá la URL pública.               | 1 mar 2026  |
| RD5    | El acceso a los datos siempre se realizará a través de la API del backend; el frontend no consultará la base de datos directamente.                                             | 27 feb 2026 |
| RD6    | El borrado de los registros principales (usuarios, plantas, publicaciones, comentarios) será lógico mediante un campo `deletedAt`, no físico.                                   | 1 mar 2026  |
| RD7    | El histórico de procesamiento de recordatorios garantizará la idempotencia mediante una clave de idempotencia diaria por planta y tipo, evitando duplicación de notificaciones. | 10 mar 2026 |
| RD8    | Las colecciones críticas (usuarios, plantas, mensajes) tendrán esquemas validados con `JsonSchema` y los índices necesarios para soportar las consultas frecuentes.             | 1 mar 2026  |
| RD9    | Los datos meteorológicos se cachearán por clave de ubicación con un TTL configurable para reducir el consumo de la cuota gratuita del servicio externo.                         | 14 mar 2026 |

#### Requisitos de vistas

| Código | Descripción                                                                                                                                                                                                   | Fecha       |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| RV1    | Los cambios en datos compartidos (mensajes, feed, notificaciones) se propagarán al cliente mediante Socket.IO en tiempo real.                                                                                 | 28 mar 2026 |
| RV2    | Las acciones de usuario que modifican estado local (likes, contadores, eliminaciones) se aplicarán de forma optimista en la interfaz, revirtiéndose si la confirmación del servidor llega con error.          | 28 mar 2026 |
| RV3    | La validación de los formularios se realizará en dos niveles: cliente (UX inmediata) y servidor (seguridad efectiva). El cliente nunca confía en sus propias validaciones para decisiones de autorización.    | 28 mar 2026 |
| RV4    | La interfaz adaptará su barra de navegación al tamaño y orientación del dispositivo: barra lateral en pantallas anchas, barra inferior en móviles verticales, barra lateral compacta en móviles horizontales. | 1 may 2026  |
| RV5    | Las imágenes externas se entregarán con caché en cliente para reducir el coste de red y se redimensionarán a un ancho máximo razonable antes de subirse al servicio externo.                                  | 1 mar 2026  |

| Código | Descripción                                                                                                                                                                        | Fecha       |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| RV6    | Las notificaciones internas se mostrarán también en la barra del sistema cuando el usuario tenga la aplicación cerrada o en segundo plano y haya consentido recibirlas.            | 15 abr 2026 |
| RV7    | El estado de pérdida de conexión se notificará al usuario mediante un indicador persistente en la parte superior de la aplicación, sin bloquear la lectura del contenido cacheado. | 22 abr 2026 |
| RV8    | El tema visual respetará las pautas de contraste WCAG en nivel AA, utilizando colores semánticos diferenciados para éxito, advertencia, información y error.                       | 25 abr 2026 |

## 4.2. Modelo de clases de diseño

El modelo de clases de diseño refleja la implementación técnica real del sistema, incluyendo las capas arquitectónicas, los tipos exactos de los atributos, los métodos públicos relevantes y las dependencias entre los componentes. A diferencia del modelo de clases de análisis (3.5), aquí se documentan ya las clases que existen en el código fuente: entidades, modelos de serialización, repositorios y sus interfaces, casos de uso y sus interfaces, mapeadores, ViewModels (en el frontend), controladores (en el backend), servicios y trabajos programados.

Dada la diferencia estructural entre frontend y backend, el modelo de clases de diseño se presenta en dos diagramas separados.

### 4.2.1. Modelo de clases de diseño (Frontend Flutter)

El frontend organiza el código en tres capas (dominio, datos, presentación) con dependencias dirigidas siempre hacia el dominio. Las entidades del dominio son inmutables y carecen de lógica de negocio compleja. Los modelos de la capa de datos son responsables exclusivamente de la serialización JSON, sin operar transformaciones de dominio. Los mapeadores se encargan de convertir entre modelos y entidades. Los casos de uso son clases con un único método público `execute()` y se invocan desde los ViewModels. Los repositorios son interfaces de dominio con una implementación concreta en la capa de datos que orquesta la fuente remota (a través del cliente HTTP centralizado) y el almacenamiento local.

[DIAGRAMA: Diagramas/Clases\_Diseño\_Frontend.png]

### 4.2.2. Modelo de clases de diseño (Backend NestJS)

El backend aplica Clean Architecture de forma estricta: la capa de dominio define las entidades, las interfaces de los repositorios, los DTOs de petición y respuesta y las interfaces de los casos de uso; la capa de datos contiene las implementaciones de los repositorios sobre el driver de MongoDB, los modelos de las colecciones, los datasources externos y los mapeadores; la capa de presentación contiene los controladores HTTP, los servicios técnicos, el gateway de Socket.IO y los trabajos programados. Las dependencias se inyectan mediante un contenedor InversifyJS.

Por su tamaño, el diagrama de clases del backend se divide en dos archivos. El primero presenta las capas de dominio y datos (entidades, interfaces de repositorios, interfaces de casos de uso, modelos MongoDB, mapeadores e implementaciones de repositorios). El segundo presenta la capa de presentación y los servicios

técnicos (controladores HTTP, gateway de Socket.IO, servicios de hash/JWT/notificaciones, trabajos programados, middlewares).

[DIAGRAMA: Diagramas/Clases\_Disenio\_Backend\_1.png]

[DIAGRAMA: Diagramas/Clases\_Disenio\_Backend\_2.png]

### 4.3. Modelo físico de datos

La base de datos del sistema es MongoDB, un motor no relacional orientado a documentos. La estructura física se compone de un conjunto de colecciones, cada una con un esquema validado por la propia base de datos y con los índices necesarios para soportar las consultas frecuentes. A continuación se documentan, una por una, todas las colecciones del sistema indicando, para cada campo, su tipo, su descripción y los índices definidos sobre la colección. El sistema utiliza el tipo `ObjectId` propio de MongoDB para los identificadores primarios y para las referencias entre colecciones.





| Campo | Tipo     | Descripción                      |
|-------|----------|----------------------------------|
| _id   | ObjectId | Identificador único del usuario. |
| name  | string   | Nombre visible del usuario.      |

| Campo | Tipo     | Descripción                      |
|-------|----------|----------------------------------|
| _id   | ObjectId | Identificador único del usuario. |
| name  | string   | Nombre visible del usuario.      |

| Campo             | Tipo              | Descripción                                                                   |
|-------------------|-------------------|-------------------------------------------------------------------------------|
| email             | string            | Correo electrónico (único).                                                   |
| passwordHash      | string            | Hash bcrypt de la contraseña.                                                 |
| photo             | string            | URL de la foto de perfil en Cloudinary (opcional).                            |
| bannerPhoto       | string            | URL del banner de perfil (opcional).                                          |
| bio               | string            | Biografía libre (opcional).                                                   |
| location          | string            | Ubicación libre del usuario (opcional).                                       |
| locationLat       | number            | Latitud asociada a la ubicación (opcional).                                   |
| locationLon       | number            | Longitud asociada a la ubicación (opcional).                                  |
| fcmToken          | string            | Token de Firebase Cloud Messaging del dispositivo (opcional).                 |
| lastChatPushTitle | string            | Último título de push de chat enviado al usuario.                             |
| preferences       | embedded document | Subdocumento con las preferencias del usuario (pushNotifications, isPrivate). |
| bannedUntil       | Date              | Fecha hasta la que el usuario está baneado (opcional).                        |
| role              | string            | Rol del usuario; los administradores tienen el valor 'admin'.                 |
| createdAt         | Date              | Marca de creación.                                                            |
| updatedAt         | Date              | Última actualización.                                                         |
| deletedAt         | Date              | Marca de borrado lógico (opcional).                                           |

Índices: índice único sobre `email`; índice sobre `deletedAt` para excluir eficientemente los usuarios eliminados.

#### Colección `plants`

| Campo  | Tipo     | Descripción                               |
|--------|----------|-------------------------------------------|
| _id    | ObjectId | Identificador único de la planta.         |
| userId | ObjectId | Referencia al usuario propietario.        |
| name   | string   | Nombre personalizado dado por el usuario. |

| Campo                      | Tipo              | Descripción                                                                           |
|----------------------------|-------------------|---------------------------------------------------------------------------------------|
| speciesId                  | ObjectId          | Referencia a la especie del catálogo.                                                 |
| photo                      | string            | URL de la foto (opcional).                                                            |
| location                   | string            | Ubicación de la planta ('Interior' o 'Exterior').                                     |
| plantLocation              | string            | Ciudad asociada a la planta.                                                          |
| plantLocationLat           | number            | Latitud de la ubicación de la planta.                                                 |
| plantLocationLon           | number            | Longitud de la ubicación de la planta.                                                |
| wateringFrequency          | number            | Frecuencia de riego en días.                                                          |
| lightNeed                  | string            | Necesidad de luz ('Low' / 'Medium' / 'High').                                         |
| pruningFrequency           | number            | Frecuencia de poda en días (opcional).                                                |
| notes                      | string            | Notas personalizadas (opcional).                                                      |
| nextWatering               | Date              | Próxima fecha de riego prevista (opcional).                                           |
| nextPruning                | Date              | Próxima fecha de poda prevista (opcional).                                            |
| considerWeatherForWatering | boolean           | Si se aplica el ajuste por previsión de lluvia.                                       |
| pendingRainAdjustment      | embedded document | Estado intermedio cuando se ha pospuesto el riego por previsión de lluvia (opcional). |
| createdAt                  | Date              | Marca de creación.                                                                    |
| updatedAt                  | Date              | Última actualización.                                                                 |
| deletedAt                  | Date              | Marca de borrado lógico (opcional).                                                   |

Índices: `userId + deletedAt`; `speciesId`; `plantLocation`.

#### Colección `plant\_species`

| Campo          | Tipo     | Descripción                                 |
|----------------|----------|---------------------------------------------|
| _id            | ObjectId | Identificador único de la especie.          |
| name           | string   | Nombre común.                               |
| scientificName | string   | Nombre científico.                          |
| image          | string   | URL de la imagen representativa (opcional). |

| Campo                      | Tipo              | Descripción                                                                       |
|----------------------------|-------------------|-----------------------------------------------------------------------------------|
| careRequirements           | embedded document | Subdocumento con `wateringDays`, `lightNeed`, `temperatureRange`.                 |
| climateCompatibility       | array<string>     | Lista de climas compatibles.                                                      |
| tips                       | array<string>     | Consejos de cultivo.                                                              |
| createdBy                  | ObjectId          | Usuario que creó la especie (si no es del seed).                                  |
| isPublic                   | boolean           | Si la especie está disponible públicamente.                                       |
| requiresPruning            | boolean           | Si la especie requiere poda.                                                      |
| pruningMonths              | array<number>     | Meses recomendados para la poda (1 a 12).                                         |
| produceFruit               | boolean           | Si la especie produce fruto.                                                      |
| harvestMonths              | array<number>     | Meses de cosecha (1 a 12).                                                        |
| seasonalWateringAdjustment | embedded document | Subdocumento con factores 'summer' y 'winter' aplicados a la frecuencia de riego. |
| minRainFallMm              | number            | Umbral en milímetros para considerar la planta regada por lluvia.                 |
| watersLitersPerWatering    | number            | Litros de agua por riego (informativo).                                           |
| createdAt                  | Date              | Marca de creación.                                                                |
| updatedAt                  | Date              | Última actualización                                                              |
| deletedAt                  | Date              | Marca de borrado lógico (opcional)                                                |

Índices: índice de texto sobre `name` y `scientificName`; índice sobre `produceFruit + harvestMonths` y sobre `requiresPruning + pruningMonths`.

#### Colección `reminders`

| Campo         | Tipo     | Descripción                             |
|---------------|----------|-----------------------------------------|
| _id           | ObjectId | Identificador único del recordatorio.   |
| plantId       | ObjectId | Referencia a la planta.                 |
| userId        | ObjectId | Referencia al usuario propietario.      |
| type          | string   | Tipo ('watering', 'pruning', 'custom'). |
| scheduledDate | Date     | Fecha planificada.                      |

| Campo       | Tipo    | Descripción                                |
|-------------|---------|--------------------------------------------|
| message     | string  | Mensaje del recordatorio.                  |
| isCompleted | boolean | Si el usuario ya lo marcó como completado. |
| suspended   | boolean | Si está temporalmente suspendido.          |
| attempts    | number  | Número de intentos de procesamiento.       |
| createdAt   | Date    | Marca de creación.                         |

Índices: `userId`; `plantId`; `scheduledDate + isCompleted` para localizar pendientes.

#### Colección `reminder\_history`

| Campo          | Tipo     | Descripción                              |
|----------------|----------|------------------------------------------|
| _id            | ObjectId | Identificador del registro de historial. |
| reminderId     | ObjectId | Referencia al recordatorio procesado.    |
| processedAt    | Date     | Marca temporal del procesamiento.        |
| result         | string   | Resultado ('sent', 'skipped', 'failed'). |
| details        | string   | Detalle textual (opcional).              |
| idempotencyKey | string   | Clave que evita el doble procesamiento.  |

Índices: índice único sobre `idempotencyKey`; índice sobre `reminderId + processedAt`.

#### Colección `posts`

| Campo         | Tipo     | Descripción                            |
|---------------|----------|----------------------------------------|
| _id           | ObjectId | Identificador único de la publicación. |
| userId        | ObjectId | Autor.                                 |
| image         | string   | URL de la imagen (opcional).           |
| likesCount    | number   | Contador denormalizado de "me gusta".  |
| commentsCount | number   | Contador denormalizado de comentarios. |

| Campo          | Tipo            | Descripción                                                   |
|----------------|-----------------|---------------------------------------------------------------|
| seenBy         | array<ObjectId> | Usuarios que han visualizado la publicación.                  |
| createdAt      | Date            | Marca de creación.                                            |
| updatedAt      | Date            | Última actualización.                                         |
| deletedAt      | Date            | Marca de borrado lógico (opcional).                           |
| deletedByAdmin | boolean         | Si la eliminación fue resultado de moderación administrativa. |

Índices: `createdAt` descendente para el feed; `userId + deletedAt` para el perfil; `deletedByAdmin` para la pantalla de restauración.

#### Colección `post\_likes`

| Campo     | Tipo     | Descripción                    |
|-----------|----------|--------------------------------|
| _id       | ObjectId | Identificador único.           |
| postId    | ObjectId | Referencia a la publicación.   |
| userId    | ObjectId | Usuario que dio el "me gusta". |
| createdAt | Date     | Marca de creación.             |

Índices: índice único compuesto sobre `postId + userId` (evita likes duplicados).

#### Colección `comments`

| Campo          | Tipo     | Descripción                           |
|----------------|----------|---------------------------------------|
| _id            | ObjectId | Identificador único.                  |
| postId         | ObjectId | Referencia a la publicación.          |
| userId         | ObjectId | Autor del comentario.                 |
| content        | string   | Texto del comentario.                 |
| createdAt      | Date     | Marca de creación.                    |
| deletedAt      | Date     | Marca de borrado lógico (opcional).   |
| deletedByAdmin | boolean  | Si la eliminación fue por moderación. |

Índices: `postId + createdAt` para paginar comentarios; `userId` para el perfil del autor.

#### Colección `conversations`

| Campo         | Tipo            | Descripción                                        |
|---------------|-----------------|----------------------------------------------------|
| _id           | ObjectId        | Identificador único de la conversación.            |
| participants  | array<ObjectId> | Identificadores de los dos usuarios participantes. |
| lastMessageAt | Date            | Marca del último mensaje (opcional).               |
| createdAt     | Date            | Marca de creación.                                 |
| updatedAt     | Date            | Última actualización.                              |
| deletedAt     | Date            | Marca de borrado lógico (opcional).                |

Índices: índice sobre `participants` para localizar conversaciones de un usuario; índice sobre `lastMessageAt` descendente para ordenar la lista.

#### Colección `messages`

| Campo          | Tipo              | Descripción                                                               |
|----------------|-------------------|---------------------------------------------------------------------------|
| _id            | ObjectId          | Identificador único del mensaje.                                          |
| conversationId | ObjectId          | Referencia a la conversación.                                             |
| senderId       | ObjectId          | Emisor del mensaje.                                                       |
| receiverId     | ObjectId          | Receptor (opcional, redundante para acelerar consultas).                  |
| text           | string            | Contenido del mensaje (opcional).                                         |
| contentMeta    | embedded document | Metadatos adicionales (opcional).                                         |
| status         | string            | Estado ('pending', 'sent', 'delivered', 'read').                          |
| tempId         | string            | Identificador temporal asignado por el cliente para reconciliar el envío. |
| createdAt      | Date              | Marca de creación.                                                        |
| updatedAt      | Date              | Última actualización.                                                     |

Índices: `conversationId + createdAt`; `receiverId + status` para detectar mensajes no leídos por destinatario.

#### Colección `notifications`

| Campo | Tipo     | Descripción          |
|-------|----------|----------------------|
| _id   | ObjectId | Identificador único. |

| Campo      | Tipo     | Descripción                                                                                                                                              |
|------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| userId     | ObjectId | Usuario destinatario.                                                                                                                                    |
| type       | string   | Tipo de notificación ('watering', 'pruning', 'harvest', 'fertilizing', 'repotting', 'weather_rain', 'weather_storm', 'custom', 'admin_warning', 'info'). |
| message    | string   | Texto mostrado al usuario.                                                                                                                               |
| reminderId | ObjectId | Referencia al recordatorio asociado (opcional).                                                                                                          |
| plantId    | ObjectId | Referencia a la planta asociada (opcional).                                                                                                              |
| isRead     | boolean  | Si el usuario la ha leído.                                                                                                                               |
| createdAt  | Date     | Marca de creación.                                                                                                                                       |

Índices: `userId + createdAt` descendente para listar las notificaciones del usuario; `userId + isRead` para calcular el contador de no leídas.

#### Colección `reports`

| Campo        | Tipo              | Descripción                                                        |
|--------------|-------------------|--------------------------------------------------------------------|
| _id          | ObjectId          | Identificador único.                                               |
| userId       | ObjectId          | Usuario que emite el reporte.                                      |
| type         | string            | Tipo ('general', 'post', 'comment').                               |
| targetId     | ObjectId          | Referencia al contenido reportado (opcional para 'general').       |
| text         | string            | Descripción de la incidencia.                                      |
| imageUrl     | string            | URL de evidencia (opcional).                                       |
| ticketNumber | number            | Número secuencial asignado al reporte.                             |
| status       | string            | Estado ('pending', 'resolved', 'dismissed').                       |
| resolvedBy   | embedded document | Subdocumento con 'adminId', 'adminName' y 'resolvedAt' (opcional). |
| createdAt    | Date              | Marca de creación.                                                 |

Índices: `status + createdAt`; `ticketNumber` único.

**Colección `weather\_cache`**

| Campo       | Tipo              | Descripción                                                  |
|-------------|-------------------|--------------------------------------------------------------|
| _id         | ObjectId          | Identificador único.                                         |
| locationKey | string            | Clave de la ubicación (latitud, longitud).                   |
| data        | embedded document | Datos meteorológicos en bruto devueltos por WeatherApi.      |
| fetchedAt   | Date              | Marca de la última actualización del registro.               |
| expiresAt   | Date              | Marca a partir de la cual el registro se considera caducado. |

Índices: índice único sobre `locationKey`; índice TTL sobre `expiresAt` para limpieza automática.

**Colección `counters`**

| Campo | Tipo   | Descripción                |
|-------|--------|----------------------------|
| _id   | string | Nombre del contador.       |
| seq   | number | Valor actual del contador. |

Esta colección auxiliar implementa contadores atómicos para generar números de ticket secuenciales sin colisiones.

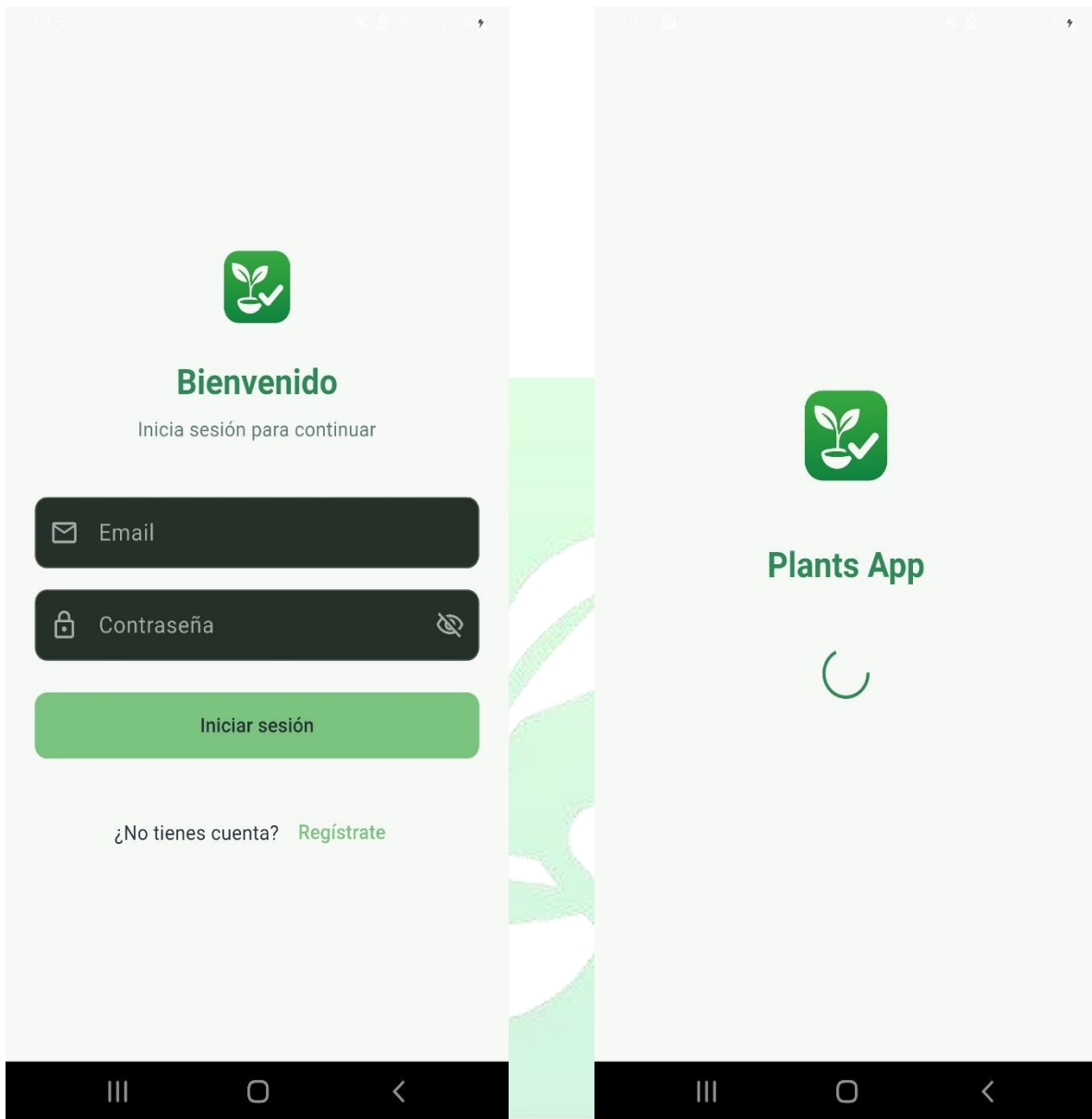
**Colección `locations`**

| Campo    | Tipo     | Descripción                        |
|----------|----------|------------------------------------|
| _id      | ObjectId | Identificador único de la entrada. |
| name     | string   | Nombre corto de la ciudad.         |
| fullname | string   | Nombre largo de la ciudad.         |
| lat      | number   | Latitud de la ciudad.              |
| lon      | number   | Longitud de la ciudad.             |

**4.4. Diseño de la interfaz de usuario**

Este apartado recopila las capturas reales de la aplicación organizadas por sección. Las capturas se han tomado tanto en navegador web (resolución de escritorio, evidenciando la adaptación con barra lateral) como en dispositivo Android físico (evidenciando la adaptación con barra inferior y la respuesta a la orientación). Cada captura va sucedida de un breve título descriptivo. Cuando una pantalla tiene variantes notables (estado vacío, estado de error, vista con datos), se incluyen capturas separadas para cada situación.

**Stack no autenticado:**



pantalla de inicio de sesión en móvil con el logotipo, los campos de correo y contraseña y el botón de iniciar sesión.

Pantalla de inicio en móvil mostrando el logotipo centrado y el indicador circular de carga sobre fondo claro.



**Bienvenido**  
Inicia sesión para continuar

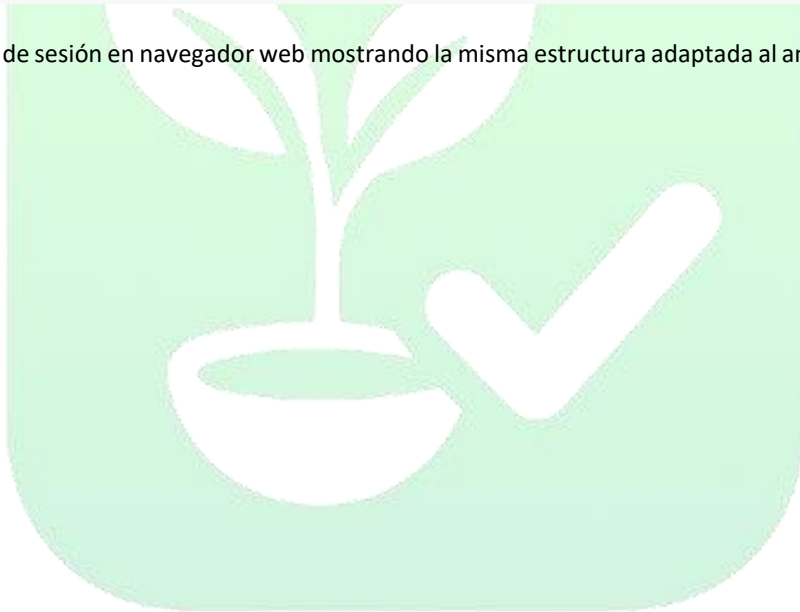
Email

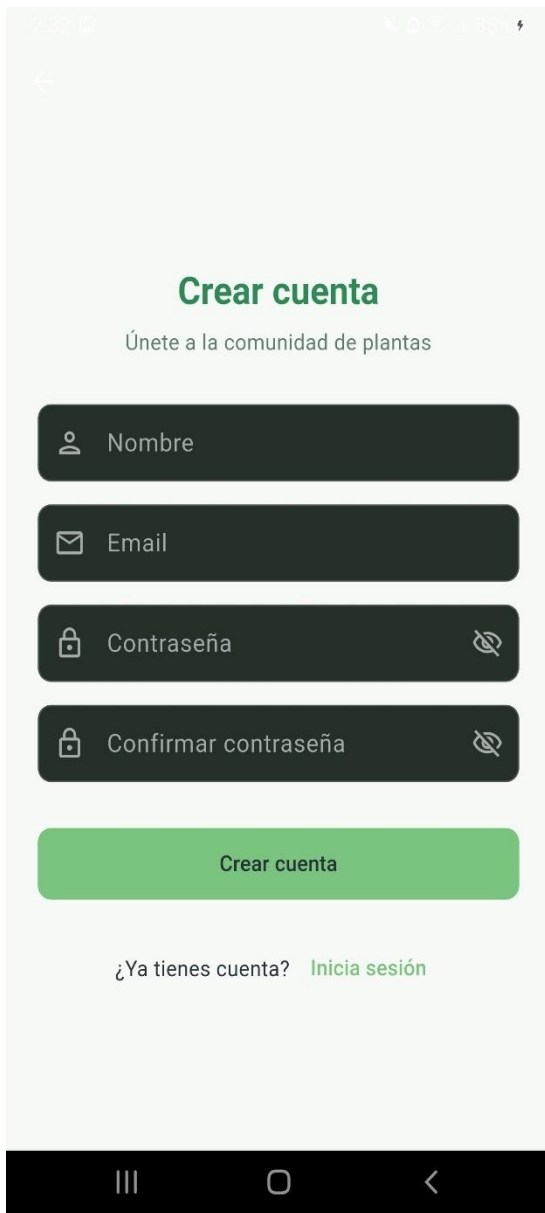
Contraseña

Iniciar sesión

[¿No tienes cuenta? Regístrate](#)

Pantalla de inicio de sesión en navegador web mostrando la misma estructura adaptada al ancho de escritorio.





2:22

←

## Crear cuenta

Únete a la comunidad de plantas

Nombre

Email

Contraseña

Confirmar contraseña

Crear cuenta

¿Ya tienes cuenta? [Inicia sesión](#)

III O <

Pantalla de registro en móvil con los cuatro campos (nombre, correo, contraseña, confirmación) y el botón de registrarse.



2:24

Omitir



## Gestiona tus plantas

Registra todas tus plantas, sus cuidados y lleva un seguimiento de su salud.

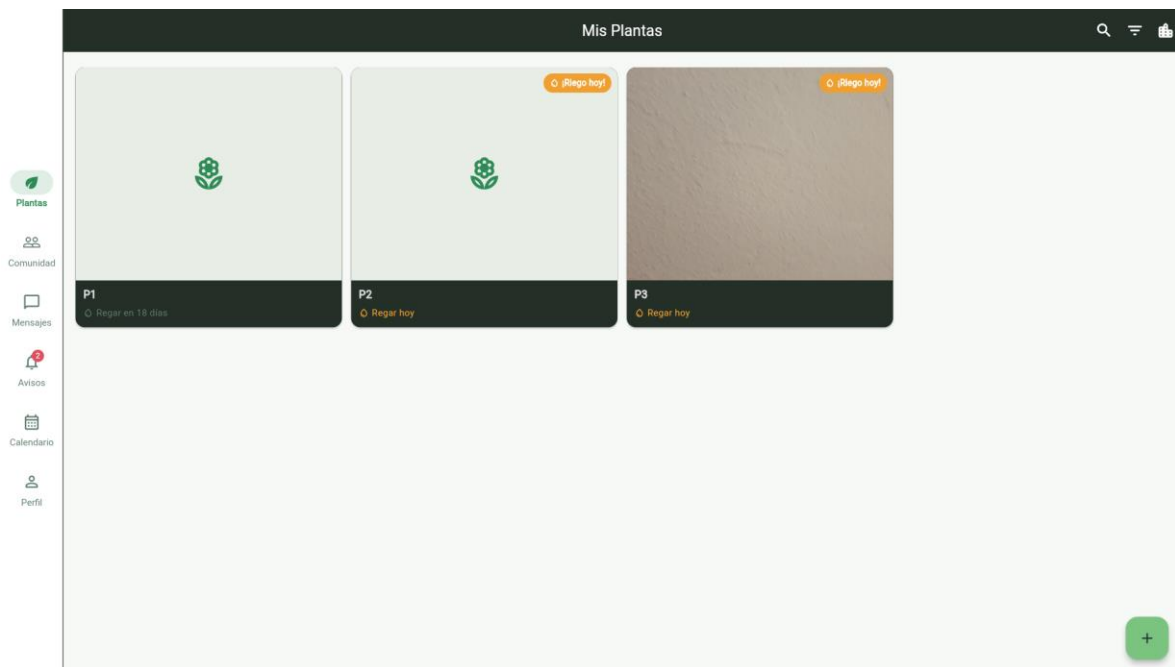
● ● ● ●

Siguiente

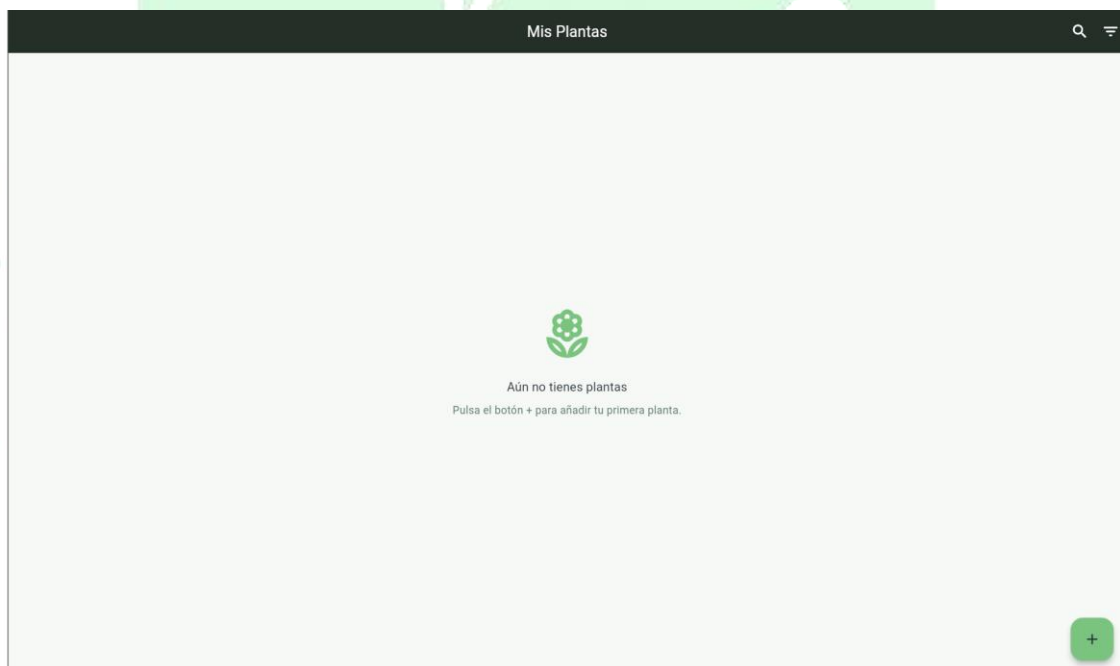
III O <

Primer slide de bienvenida en móvil con el logotipo, un título descriptivo y la indicación de deslizar para continuar.

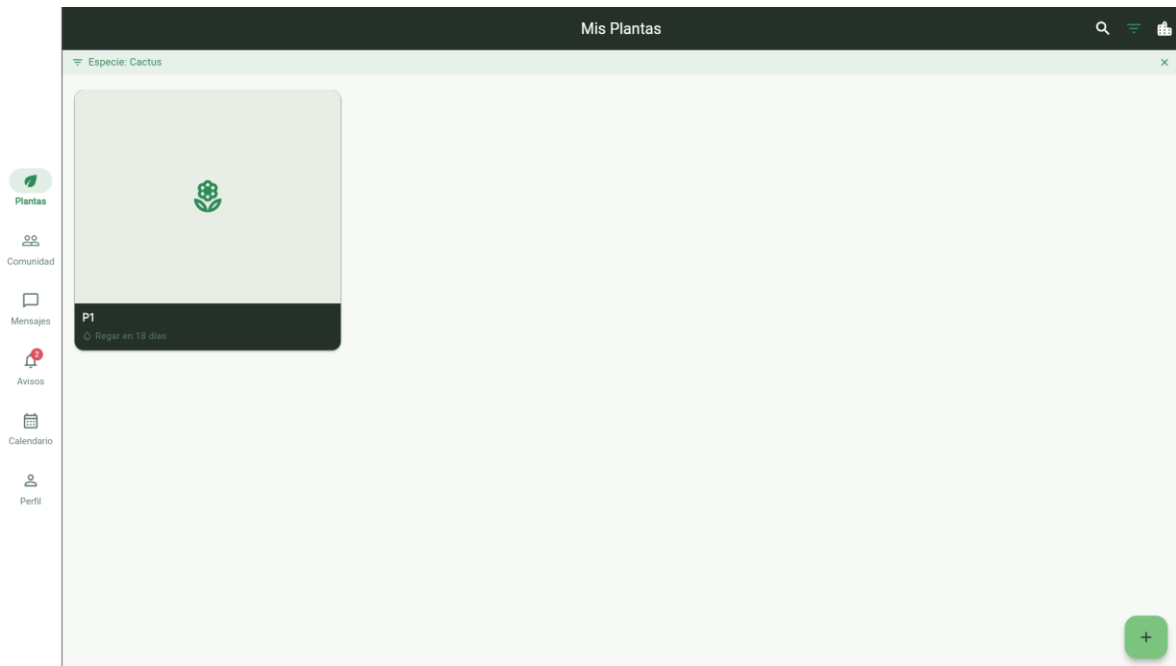
### Pestaña Mis Plantas:



Listado de plantas en navegador web con la barra lateral de navegación a la izquierda y la rejilla principal a la derecha.

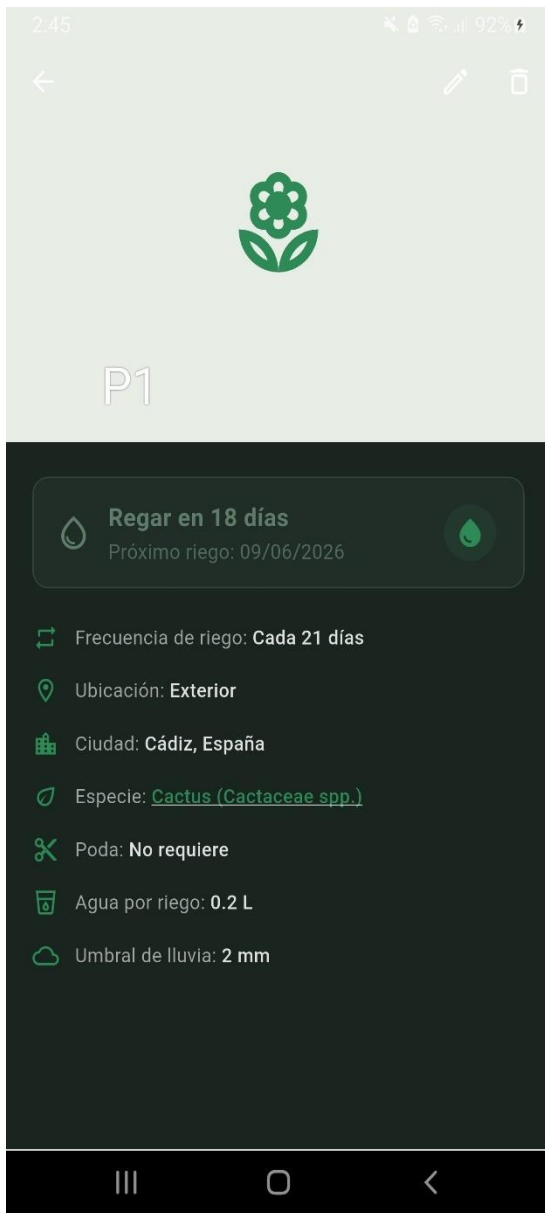


Estado vacío del listado de plantas mostrando el mensaje informativo y la sugerencia de añadir la primera planta.

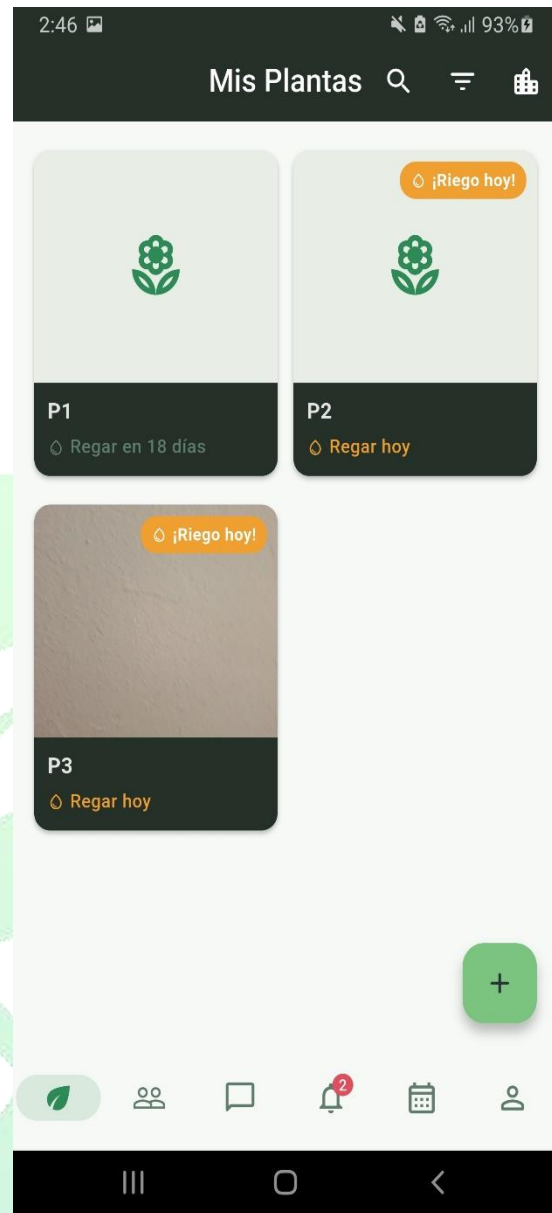


Listado de plantas con un filtro de especie activo mostrando la barra inferior de filtro activo y los resultados filtrados

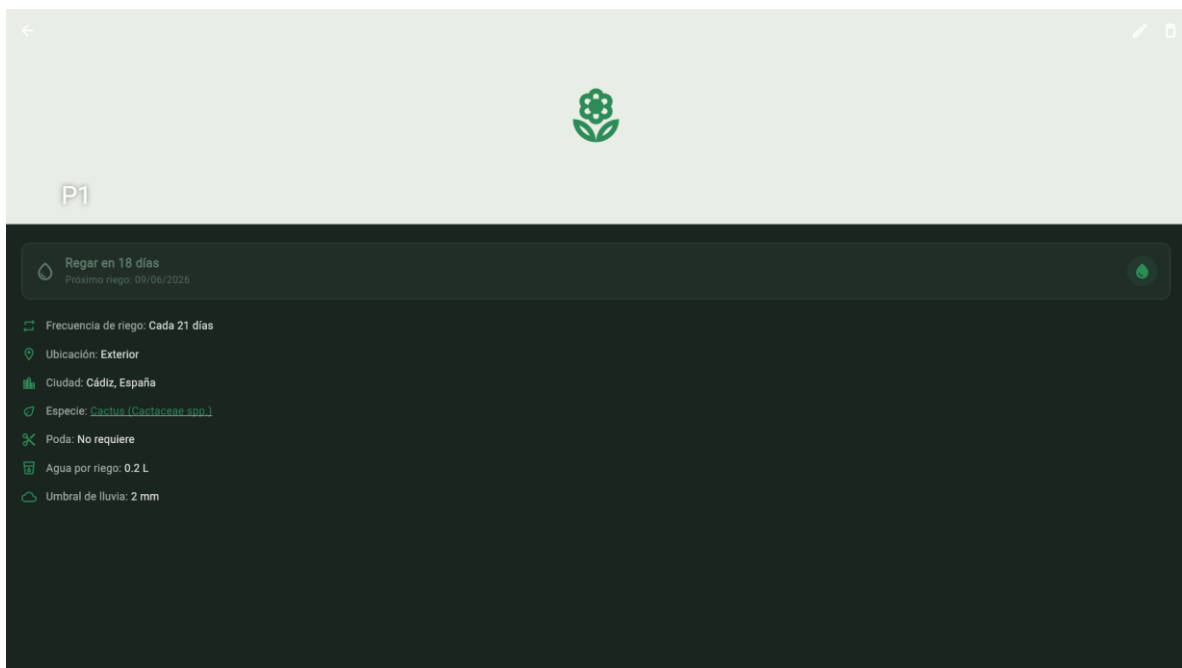




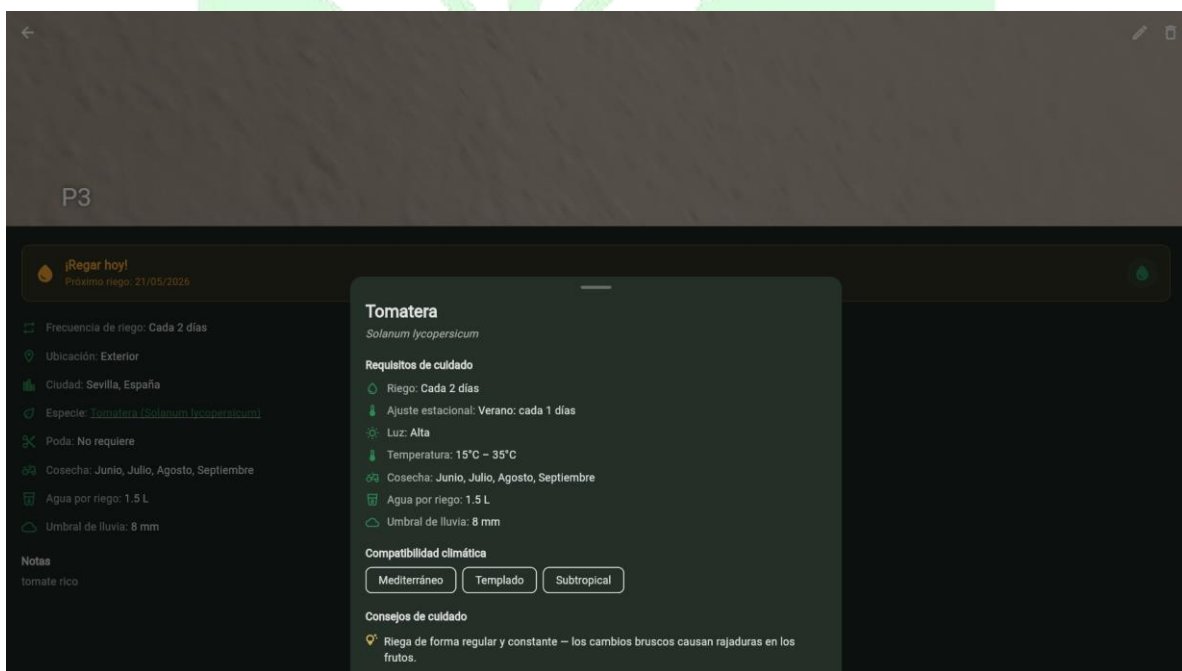
Detalle de una planta en móvil mostrando la foto a tamaño completo, el nombre, la especie con enlace a la ventana de información, los próximos recordatorios y los botones de acción.



Listado de plantas en móvil con la rejilla de tarjetas, la barra superior con iconos de calendario, búsqueda y filtros, y el botón flotante de añadir planta



Detalle de la misma planta en navegador web.



Ventana emergente con la información ampliada de una especie, mostrada como hoja deslizante sobre el detalle de la planta.

2:52 96%

← Nueva planta Crear planta

Añadir foto

Nombre de la planta \*

Buscar especie \*

Ubicación \* Interior

Ciudad de la planta \*

Notas personalizadas (opcional)

Formulario de creación de planta en móvil mostrando el autocompletado de especie y los campos visibles en modo creación.

2:54 97%

← Editar planta Guardar cambios

Añadir foto

Nombre de la planta \* P1

Buscar especie \*

Ubicación \* Exterior

Ciudad de la planta \* Cádiz, España

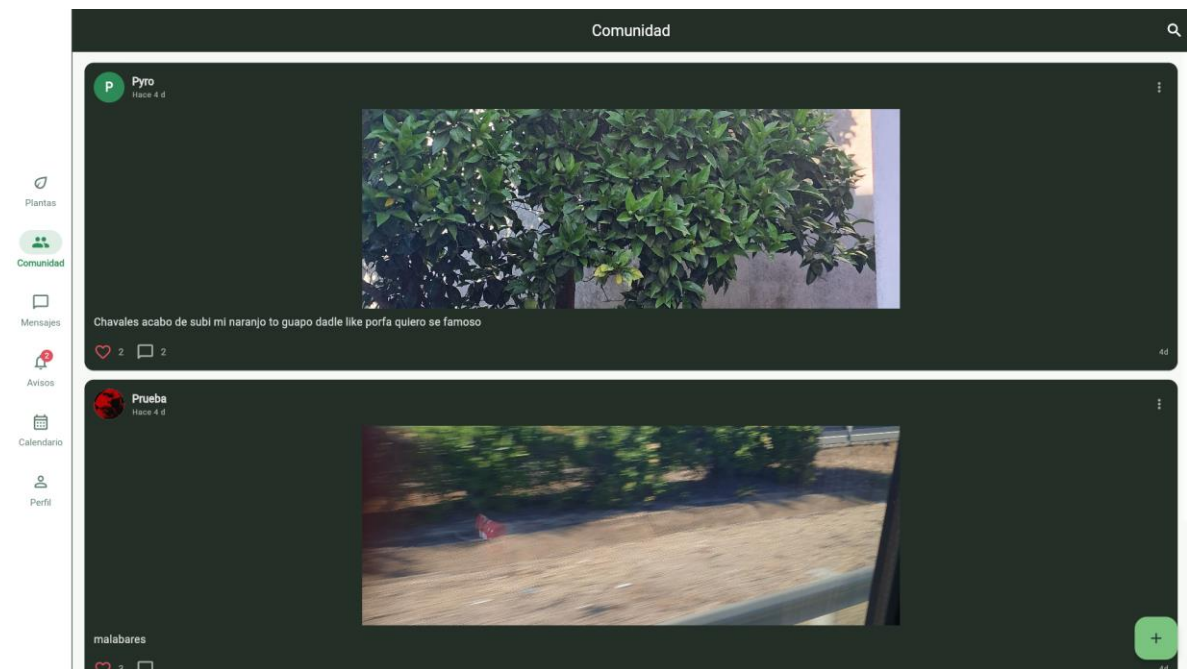
Luz necesaria \* Media

Frecuencia de riego (días) \* 21 días

Notas personalizadas (opcional)

Formulario de edición de planta en móvil mostrando los campos adicionales de frecuencia de riego y necesidad de luz, exclusivos del modo edición.

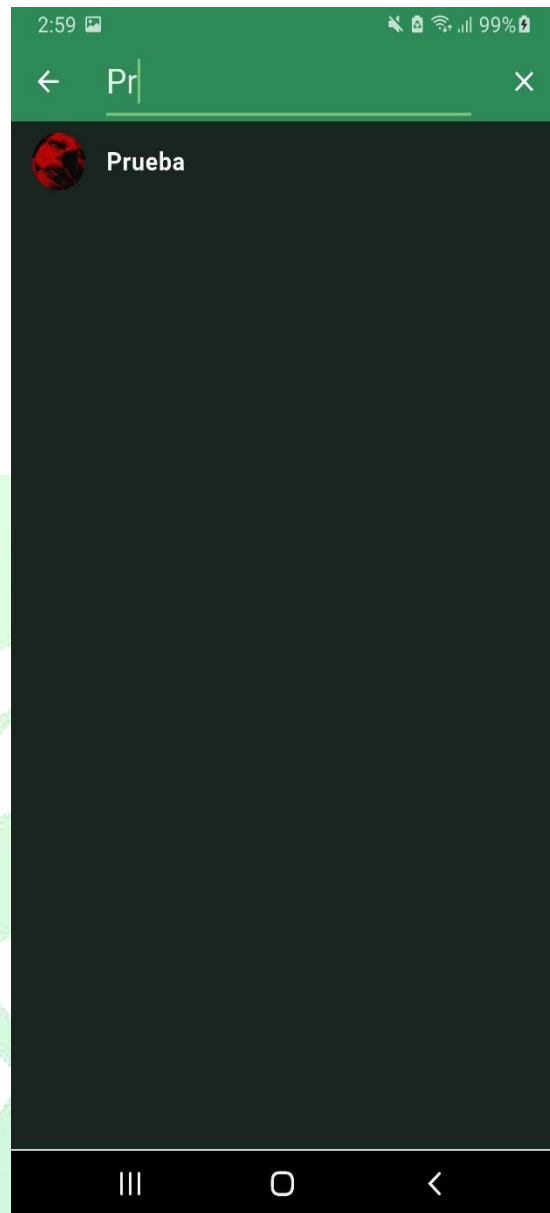
### Pestaña Comunidad:



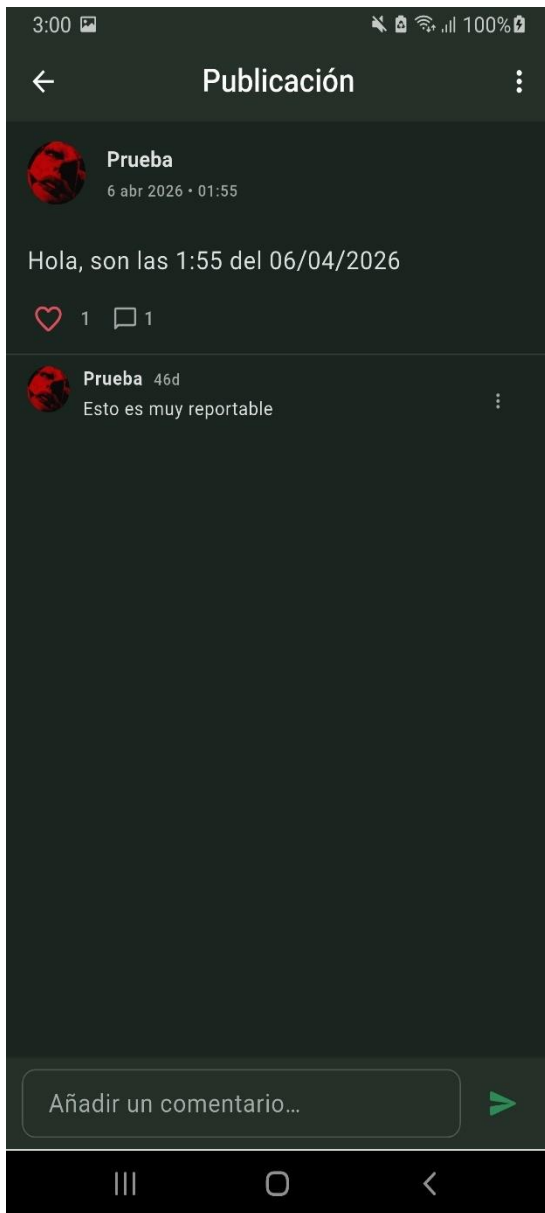
Feed de la comunidad en navegador web con el ancho de las imágenes limitado para mantener la proporción visual.



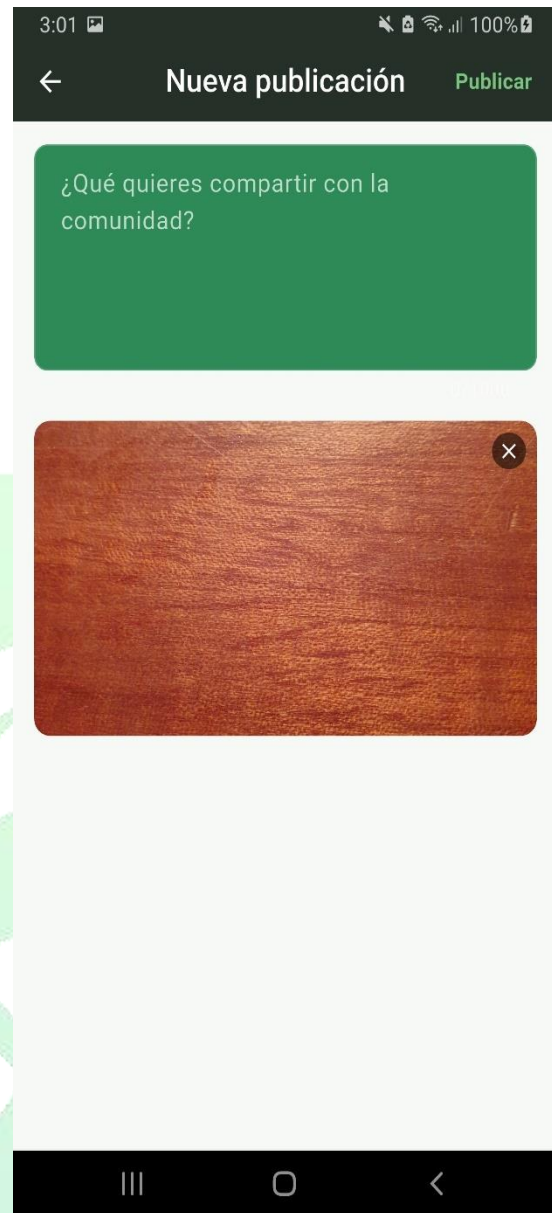
Feed de la comunidad en móvil con varias publicaciones, contadores de "me gusta" y comentarios visibles.



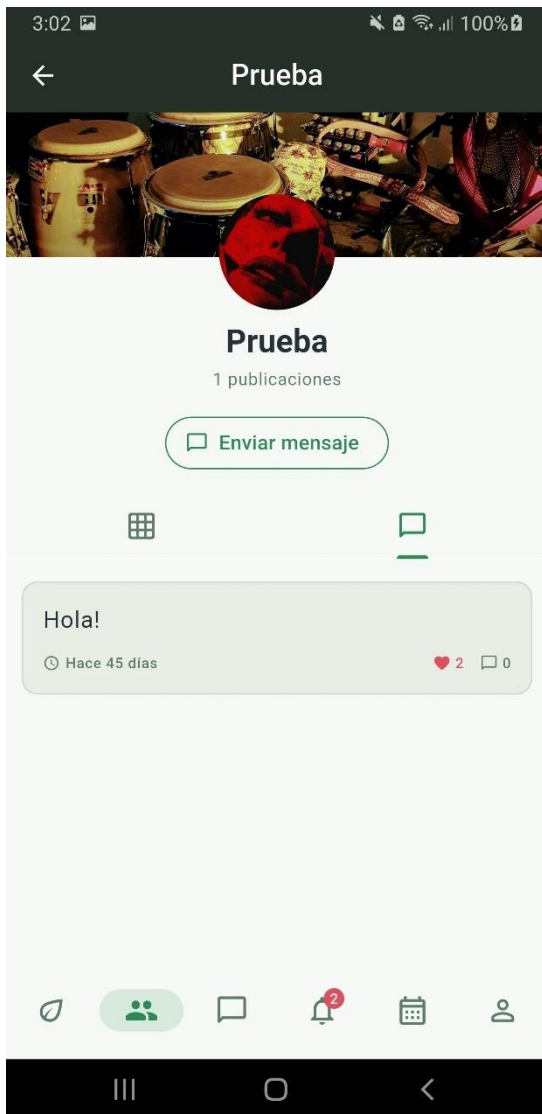
Buscador de usuarios en móvil mostrando resultados con foto y nombre.



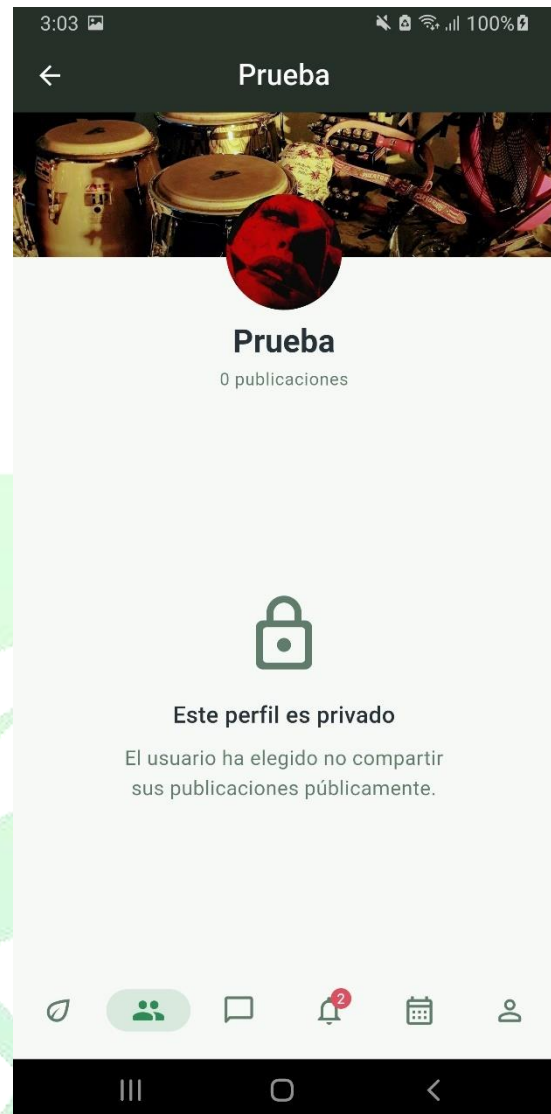
Detalle de una publicación en móvil con la lista de comentarios y el campo de entrada al pie.



Formulario de creación de publicación en móvil con previsualización de imagen seleccionada.

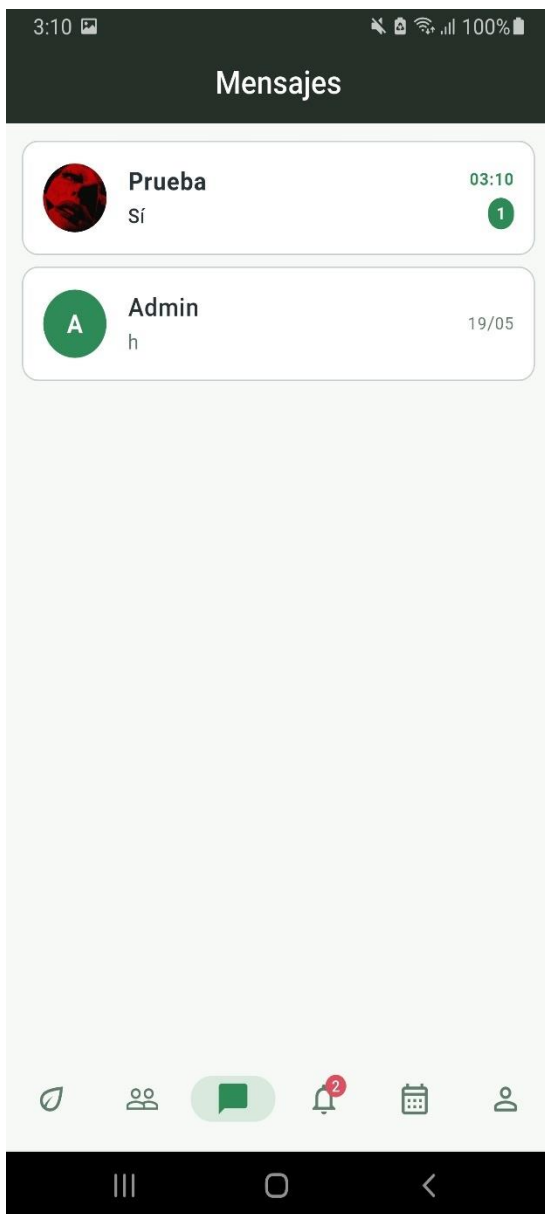


Perfil de otro usuario en móvil mostrando banner, avatar, nombre, botón de enviar mensaje y la pestaña "Con foto".

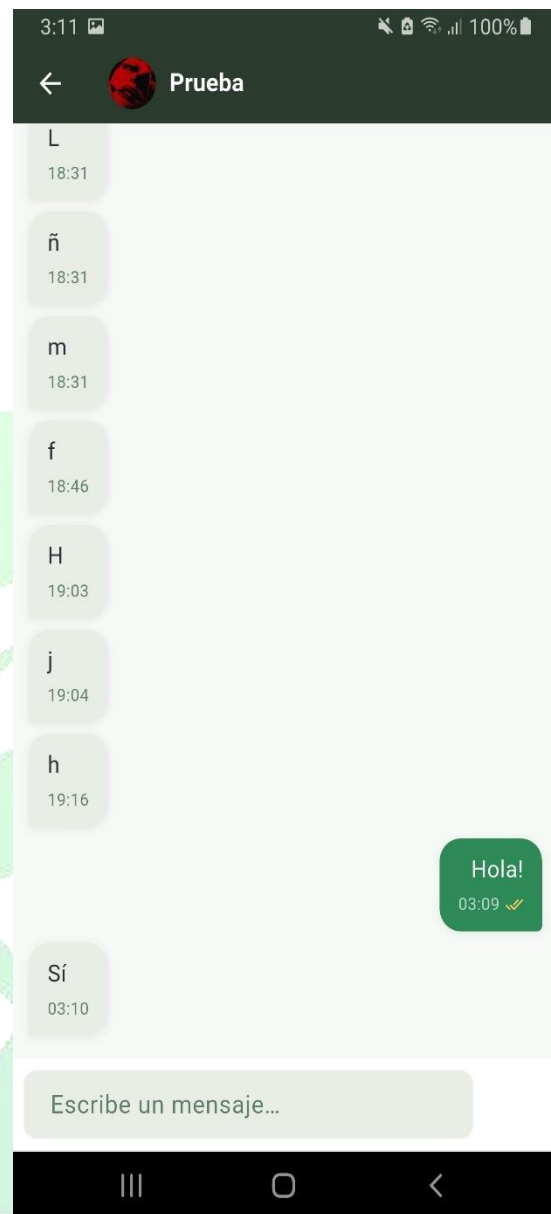


Perfil privado de otro usuario mostrando el icono de candado y el mensaje explícito sin publicaciones.

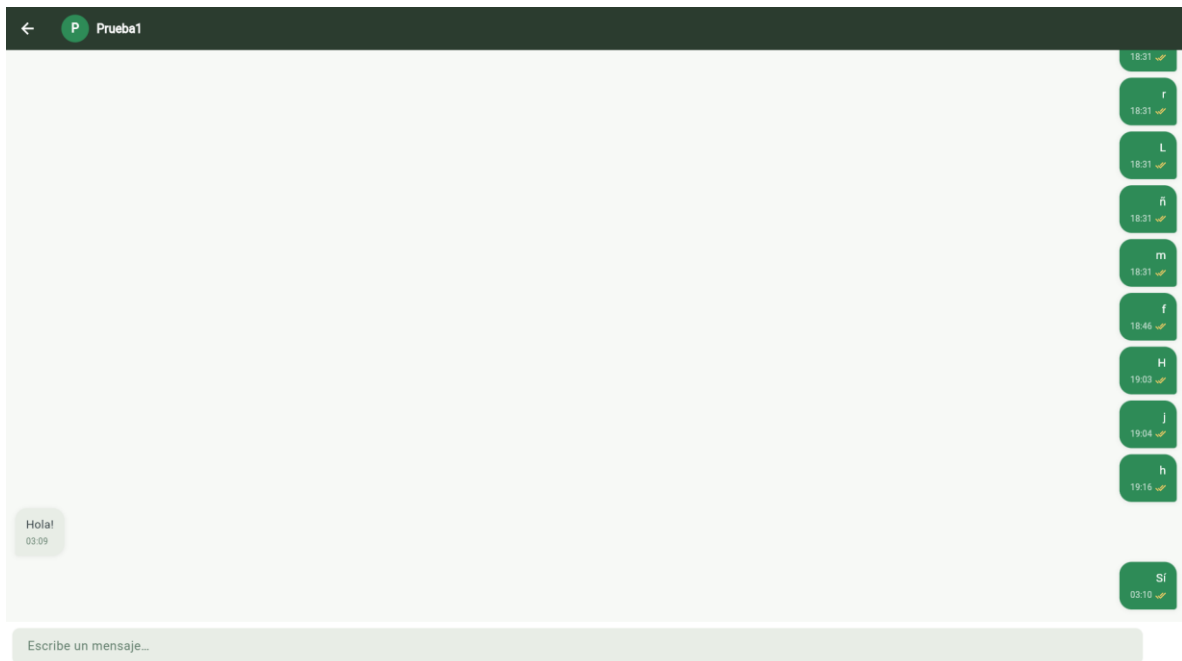
**Pestaña Mensajes:**



Lista de conversaciones en móvil con foto, nombre, último mensaje y marca de tiempo, incluyendo al menos una conversación con indicador de no leídos.



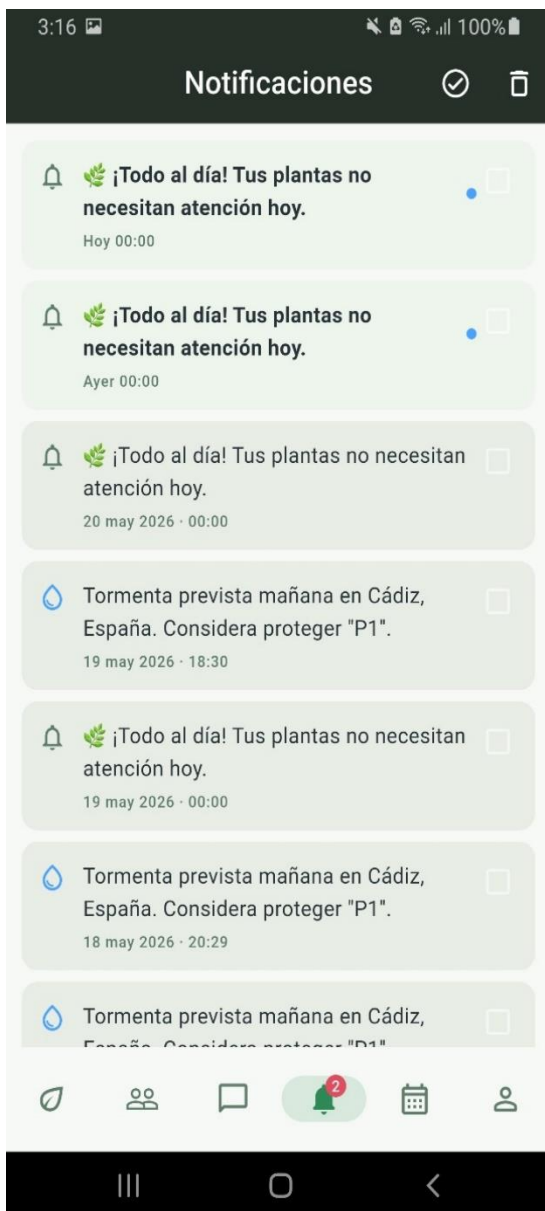
Conversación abierta en móvil mostrando burbujas de mensajes propios y ajenos con sus iconos de estado y el campo de entrada al pie.



La misma conversación en navegador web.



### Pestaña Avisos:

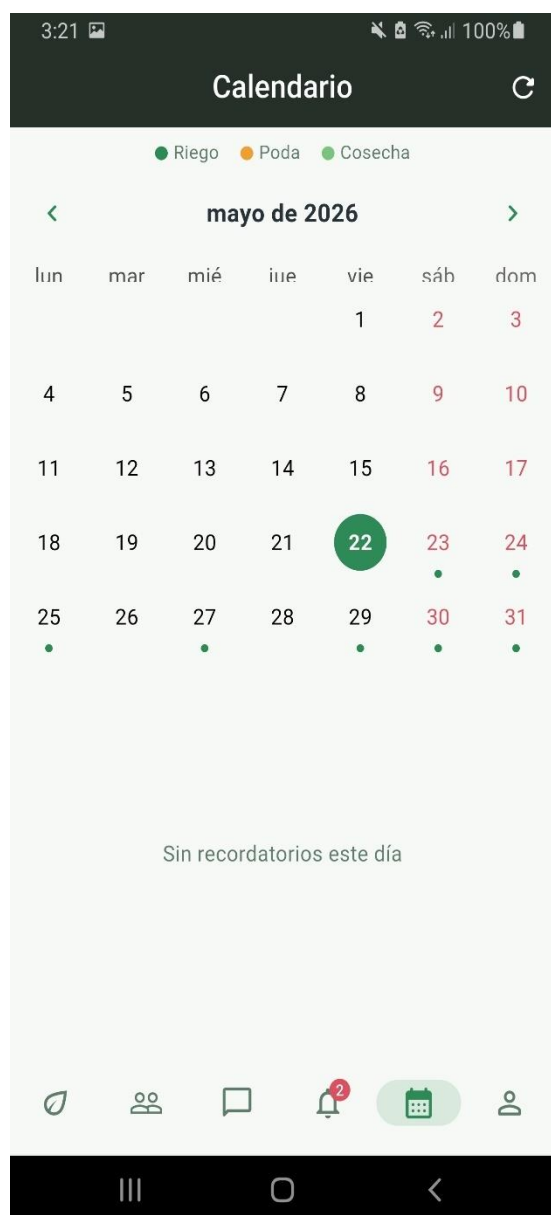


Pestaña de avisos en móvil con varias notificaciones de distintos tipos, mostrando sus iconos diferenciados.

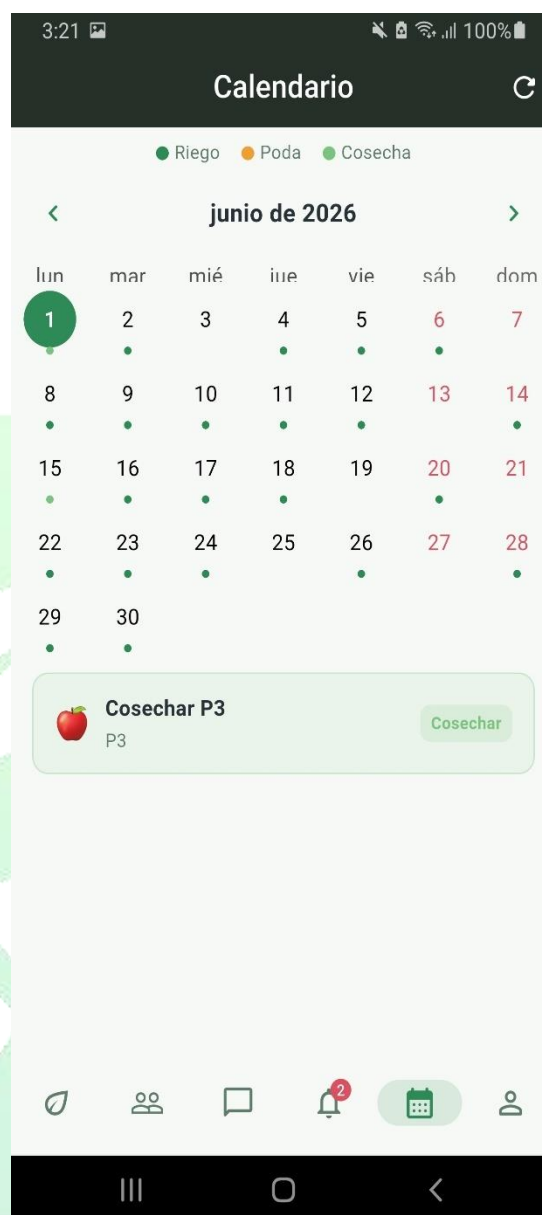


Pestaña de avisos en móvil con varias notificaciones seleccionadas y la barra superior contextual con el contador de seleccionados y las acciones de marcar/eliminar.

### Pestaña Calendario:



Calendario mensual en móvil con varios días marcados con puntos de color y la leyenda inferior visible.

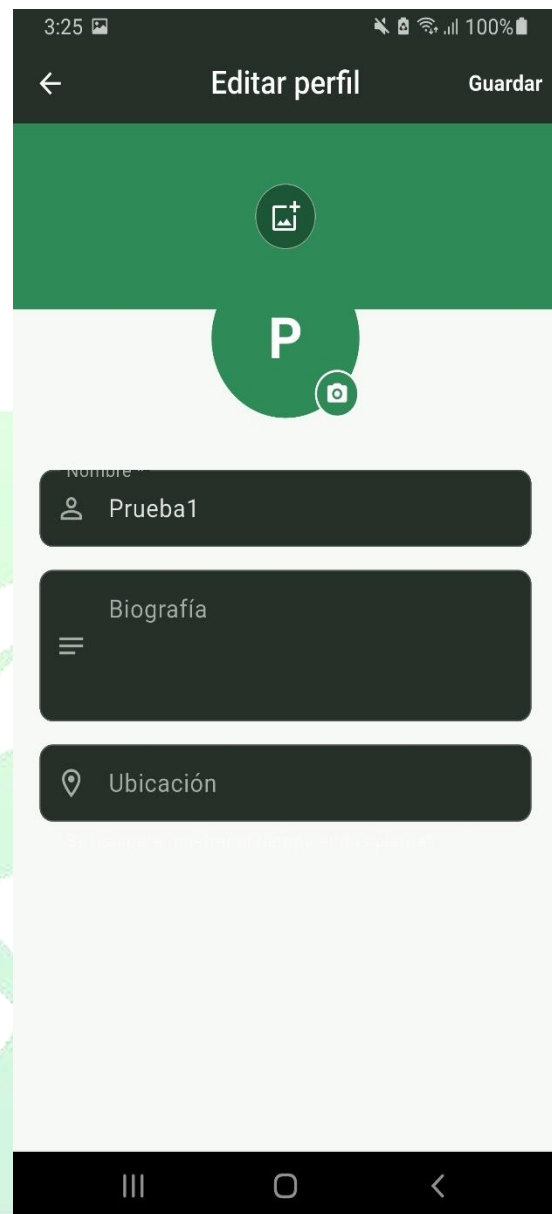


Calendario en móvil con un día seleccionado y la lista de eventos del día desplegada bajo el calendario.

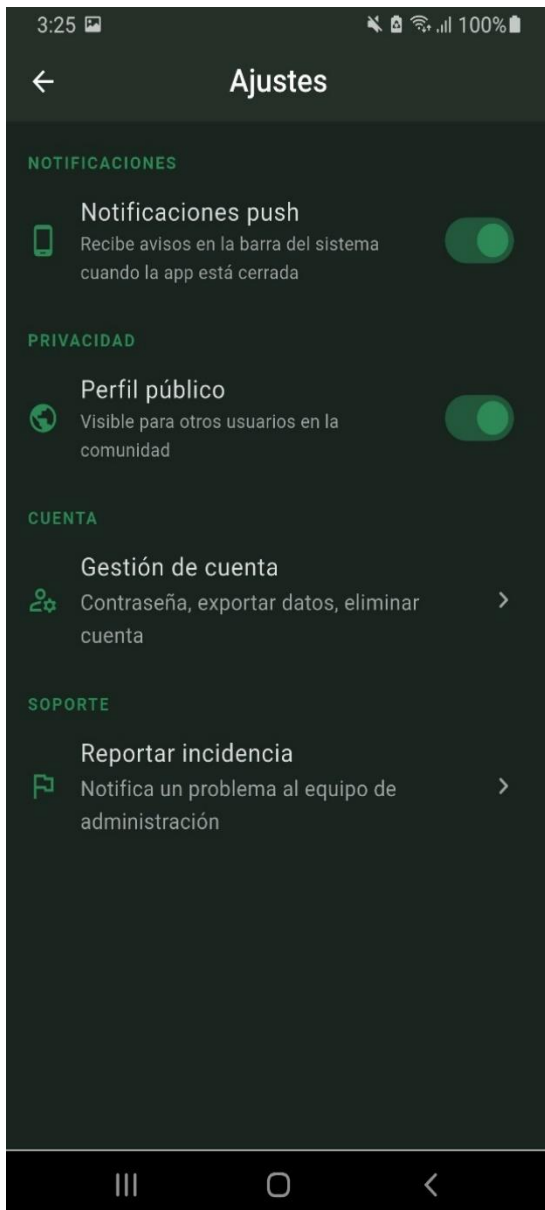
**Pestaña Perfil:**



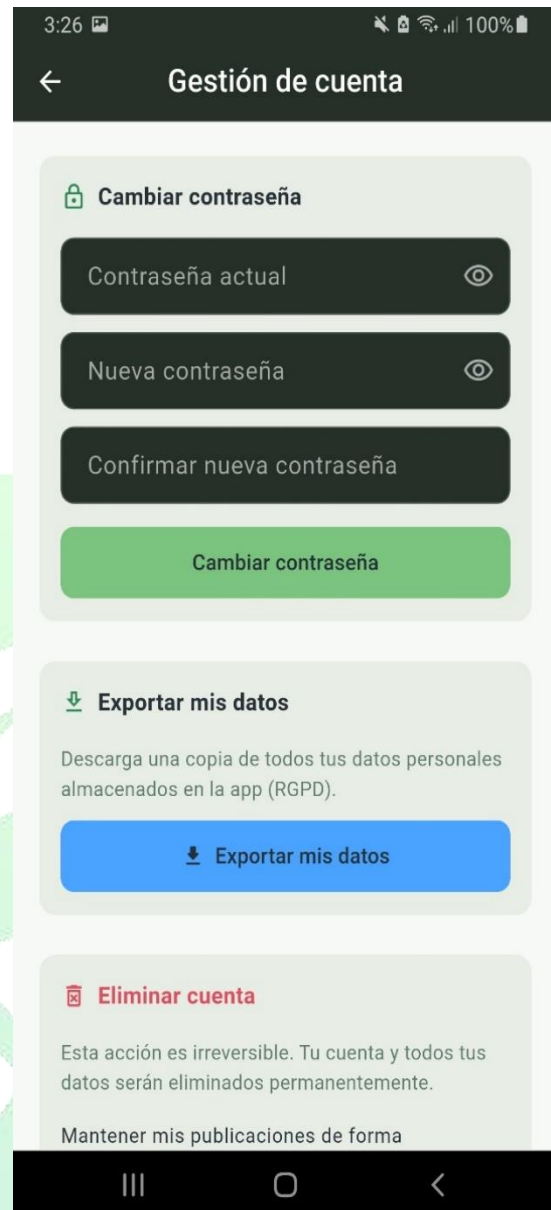
Pantalla de mi perfil en móvil mostrando banner, avatar, datos personales y las pestañas de publicaciones propias.



Pantalla de edición de perfil en móvil con el selector de banner, el avatar superpuesto y los campos de nombre, biografía y ubicación.



Pantalla de ajustes en móvil mostrando las secciones de notificaciones, privacidad, soporte y cuenta.



Pantalla de gestión de cuenta en móvil mostrando los bloques de cambio de contraseña, exportación de datos y eliminación de cuenta con el interruptor de preservación de contenido.

Reportar incidencia

Describe el problema que has encontrado

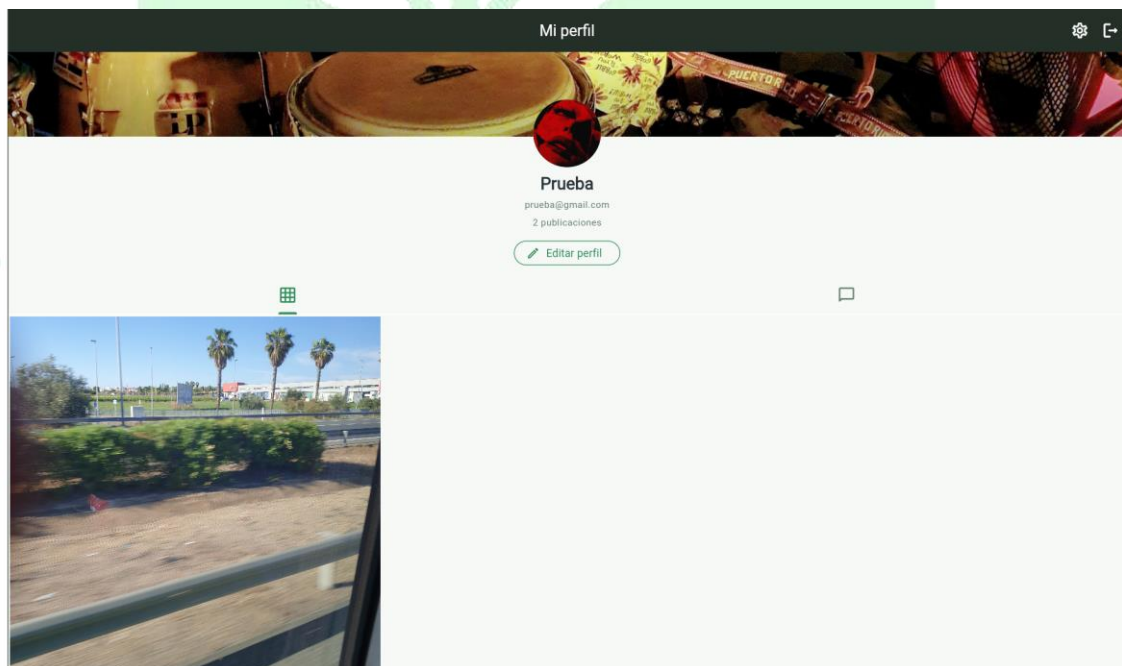
Ej. Al abrir la sección de plantas la app se cierra...

Imagen adjunta (opcional)

Añadir imagen

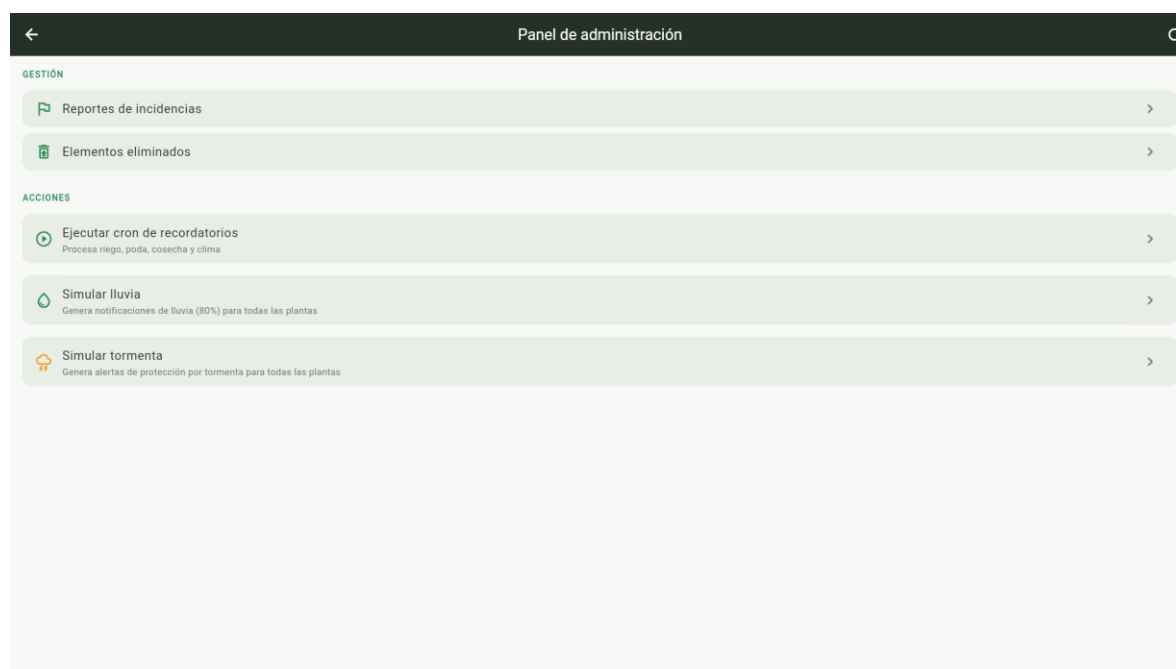
Enviar reporte

Formulario de reporte de incidencia mostrando el selector de tipo, el campo de descripción y el selector de imagen opcional.

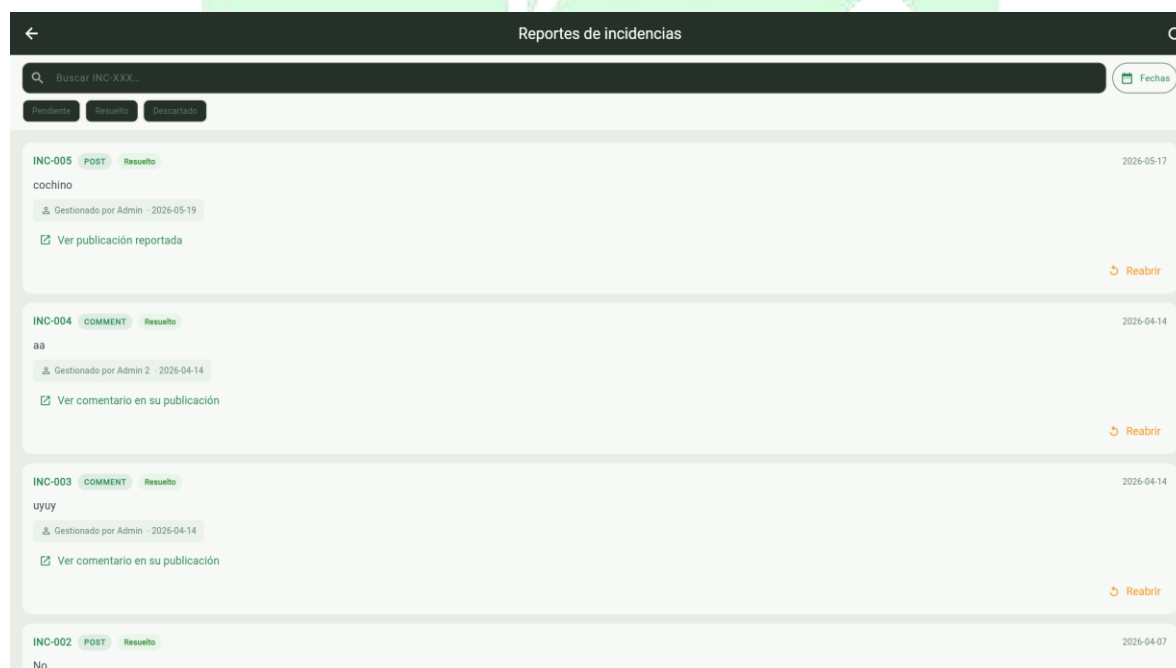


Pantalla de mi perfil en navegador web.

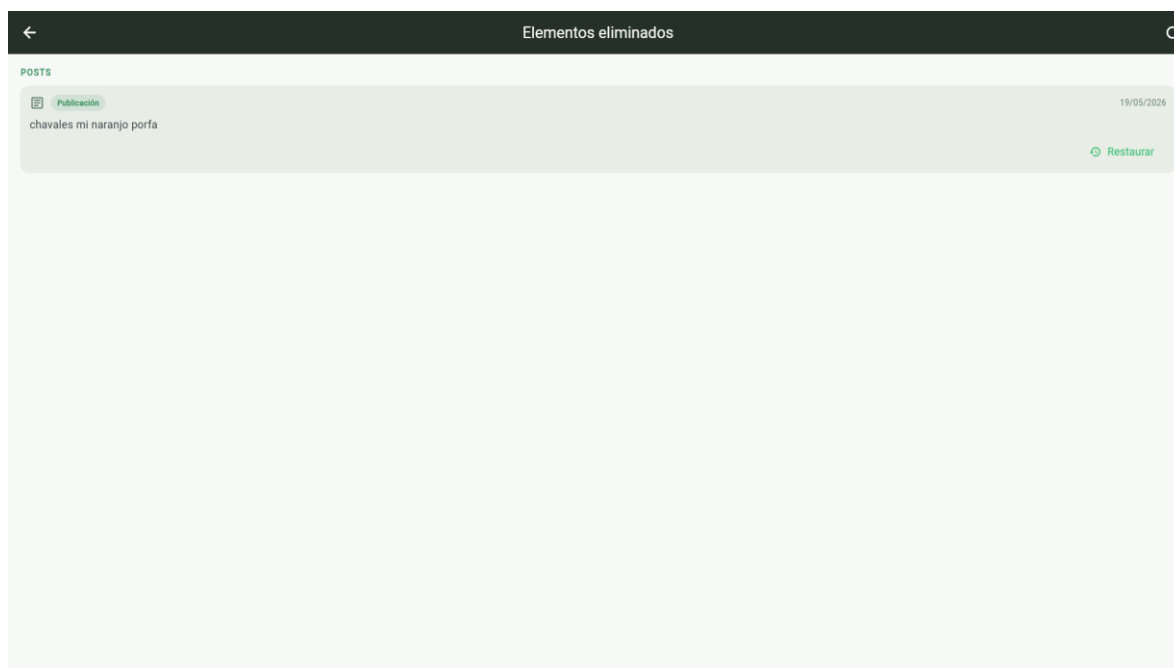
## Panel de administración:



Panel de administración mostrando las tarjetas de acceso, el indicador de reportes pendientes y la sección de acciones.



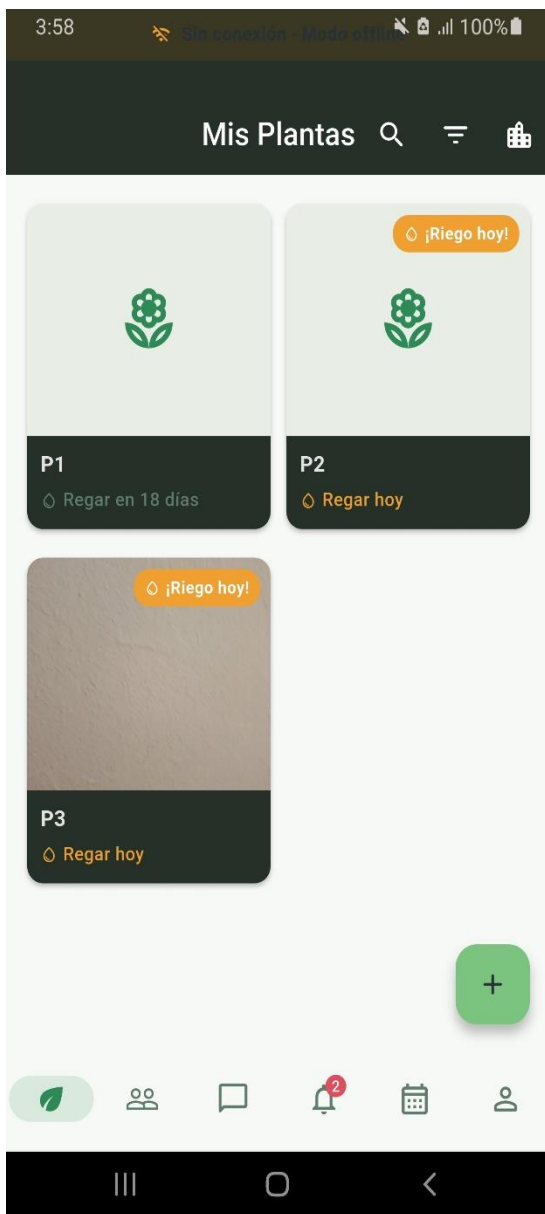
Listado de reportes de incidencias.



Listado de elementos eliminados agrupado por tipo con los botones de restaurar visibles.



**Estados transversales:**



Captura de cualquier pantalla mostrando el indicador superior de pérdida de conexión a internet.



Pantalla de inicio mostrando el mensaje contextual que aparece cuando el servidor tarda en responder más de cinco segundos.

#### 4.5. Plan de migración y carga inicial de datos

La aplicación no migra datos desde ningún sistema anterior, ya que se trata de un producto de nueva creación. No obstante, sí requiere una carga inicial mínima en la base de datos para que pueda utilizarse correctamente desde el primer arranque. Esta carga inicial está implementada en forma de varios scripts independientes invocables desde la línea de comandos del backend; los scripts son idempotentes (pueden ejecutarse varias veces sin generar duplicados) y consultan al usuario únicamente la URI de conexión a la base de datos a través de la variable de entorno correspondiente.

##### Elementos cargados por el seed:

El proceso de inicialización persiste tres conjuntos de datos en la base de datos:

**Catálogo de especies de plantas:** Un total de catorce especies cuidadosamente seleccionadas para cubrir los casos de uso del proyecto, con todas sus características de cuidado (frecuencia base de riego, necesidad de luz, climas compatibles, consejos, meses de poda y cosecha, ajustes estacionales y umbrales de lluvia). Entre las especies se incluye una especie de prueba con frecuencia de riego diaria y todos los meses marcados para poda y cosecha, que sirve para verificar el funcionamiento de los recordatorios sin esperar a las ventanas naturales del año.

**Catálogo de ubicaciones:** Las cincuenta y dos capitales de provincia españolas con su nombre corto, su nombre completo ('Ciudad, España') y sus coordenadas (latitud y longitud). Este catálogo se utiliza en el autocompletado de ubicación tanto del perfil del usuario como del formulario de planta, y permite resolver con precisión las coordenadas que se pasan al servicio meteorológico.

**Usuario administrador por defecto:** Un usuario con rol administrador cuyas credenciales se leen de variables de entorno ('ADMIN\_EMAIL' y 'ADMIN\_PASSWORD'). Si el usuario ya existe se eleva su rol a administrador si no lo tenía; si está marcado como eliminado se restaura. Este usuario permite el acceso al panel administrativo desde el primer arranque, sin necesidad de promover cuentas manualmente.

##### Ejecución de la carga inicial:

Los scripts se invocan mediante los comandos ``npm run seed:species``, ``npm run seed:locations`` y ``npm run seed:admin`` ejecutados desde el directorio del backend. Cada script lee la variable de entorno ``MONGODB_URI`` para identificar la base de datos destino y aplica las inserciones o actualizaciones necesarias. El despliegue en producción contempla la ejecución manual de estos comandos contra la base de datos remota una sola vez, antes del primer uso. Si en el futuro se incorporan nuevos elementos al catálogo (nuevas especies, nuevas capitales) basta con actualizar el script correspondiente y volver a ejecutarlo: las entradas ya presentes se respetan y las nuevas se añaden.

Al margen de estos scripts, la aplicación no requiere ningún otro proceso de migración. Las colecciones del resto del modelo ('users', 'plants', 'posts', 'comments', 'messages', 'notifications', 'reports', etc.) se pueblan de forma natural a partir de la actividad de los usuarios. Las dependencias adicionales necesarias para la ejecución (paquetes Node.js y dependencias de Flutter) se instalan automáticamente con los comandos ``npm install`` y ``flutter pub get`` la primera vez que se compila cada proyecto.

## 4.6. Plan de pruebas

El plan de pruebas se compone de pruebas funcionales y de aceptación organizadas por área temática. Cada prueba se codifica con la nomenclatura `PR-<area><número>` (por ejemplo, `PR-AUT1`), incluye una descripción concisa del escenario y se clasifica como **\*\*V\*\*** (verdadero/verde: la prueba debe pasar, comportamiento correcto esperado) o **\*\*F\*\*** (falso/rojo: la prueba debe fallar de forma controlada, esto es, el sistema debe rechazar la operación con un error claro). Adicionalmente, una segunda tabla resume el resultado de cada prueba en cada una de las plataformas soportadas (Web, Android, iOS), permitiendo evidenciar la cobertura multiplataforma.

### 4.6.1. Pruebas de autenticación y registro

| Código  | Descripción                                                                                                            | Tipo |
|---------|------------------------------------------------------------------------------------------------------------------------|------|
| PR-AUT1 | Registrar un usuario nuevo con correo y contraseña válidos.                                                            | V    |
| PR-AUT2 | Intentar registrar un correo ya existente: el sistema responde con error de conflicto.                                 | F    |
| PR-AUT3 | Intentar registrar con una contraseña demasiado corta: el sistema rechaza la petición.                                 | F    |
| PR-AUT4 | Iniciar sesión con credenciales correctas.                                                                             | V    |
| PR-AUT5 | Intentar iniciar sesión con contraseña incorrecta: el sistema responde con error de autenticación.                     | F    |
| PR-AUT6 | Reabrir la aplicación con un token aún válido y entrar directamente sin reintroducir credenciales.                     | V    |
| PR-AUT7 | Forzar más de diez intentos fallidos de inicio de sesión en un minuto: el sistema responde con error de tasa excedida. | F    |
| PR-AUT8 | Cerrar sesión manualmente y verificar que se vuelve a la pantalla de inicio de sesión.                                 | V    |

### 4.6.2. Pruebas de inventario de plantas

| Código  | Descripción                                                                              | Tipo |
|---------|------------------------------------------------------------------------------------------|------|
| PR-INV1 | Crear una planta nueva con foto, especie, ubicación y los datos derivados de la especie. | V    |
| PR-INV2 | Crear una planta sin seleccionar especie: el formulario impide enviar.                   | F    |
| PR-INV3 | Editar una planta propia modificando el nombre y verificar que se persiste.              | V    |

| Código  | Descripción                                                                                                           | Tipo |
|---------|-----------------------------------------------------------------------------------------------------------------------|------|
| PR-INV4 | Intentar editar una planta ajena (a través de un identificador manipulado): el sistema responde con error de permiso. | F    |
| PR-INV5 | Eliminar una planta propia y comprobar que desaparece del listado y deja de generar recordatorios.                    | V    |
| PR-INV6 | Filtrar el listado por nombre y obtener únicamente las plantas coincidentes.                                          | V    |
| PR-INV7 | Filtrar por especie y por ciudad simultáneamente.                                                                     | V    |
| PR-INV8 | Subir como foto un archivo de tipo no soportado: el sistema lo rechaza con un mensaje claro.                          | F    |

#### 4.6.3. Pruebas de recordatorios y calendario

| Código  | Descripción                                                                                                                   | Tipo |
|---------|-------------------------------------------------------------------------------------------------------------------------------|------|
| PR-REC1 | Marcar manualmente una planta como regada y verificar que la próxima fecha de riego se recalcula correctamente.               | V    |
| PR-REC2 | Ejecutar el proceso de recordatorios y comprobar que se genera una notificación por cada planta con riego pendiente.          | V    |
| PR-REC3 | Ejecutar el proceso de recordatorios dos veces el mismo día y comprobar que no se duplican las notificaciones (idempotencia). | V    |
| PR-REC4 | Simular una previsión de lluvia significativa y comprobar que el próximo riego se pospone.                                    | V    |
| PR-REC5 | Al día siguiente, simular que no llovió y comprobar que el sistema revierte el ajuste recuperando la fecha original.          | V    |
| PR-REC6 | Consultar el calendario mensual y verificar que los días con tareas muestran los puntos de color correspondientes.            | V    |
| PR-REC7 | Seleccionar un día con tareas y comprobar que la lista de eventos del día se carga correctamente.                             | V    |

#### 4.6.4. Pruebas del feed de la comunidad

| Código  | Descripción                                                                                    | Tipo |
|---------|------------------------------------------------------------------------------------------------|------|
| PR-COM1 | Publicar un mensaje con texto y foto y comprobar que aparece en el feed de los demás usuarios. | V    |

| Código   | Descripción                                                                                                                                      | Tipo |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------|------|
| PR-COM2  | Publicar sin texto y sin imagen: el formulario impide enviar.                                                                                    | F    |
| PR-COM3  | Dar "me gusta" a una publicación y verificar que el contador se incrementa.                                                                      | V    |
| PR-COM4  | Quitar el "me gusta" y verificar que el contador se decrementa.                                                                                  | V    |
| PR-COM5  | Pulsar repetidamente el botón de "me gusta" en rápida sucesión: el sistema impide solicitudes duplicadas mientras la anterior no haya terminado. | V    |
| PR-COM6  | Añadir un comentario a una publicación y verificar que aparece bajo ella.                                                                        | V    |
| PR-COM7  | Eliminar un comentario propio y verificar que el contador de comentarios decrementa.                                                             | V    |
| PR-COM8  | Reportar una publicación y comprobar que se genera un ticket con número secuencial único.                                                        | V    |
| PR-COM9  | Buscar un usuario por nombre y abrir su perfil.                                                                                                  | V    |
| PR-COM10 | Buscar un usuario con perfil privado siendo usuario estándar: no aparece en los resultados.                                                      | F    |

#### 4.6.5. Pruebas de chat en tiempo real

| Código  | Descripción                                                                                                                                                       | Tipo |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| PR-CHT1 | Abrir una conversación con otro usuario y enviar un mensaje. El destinatario lo recibe en tiempo real.                                                            | V    |
| PR-CHT2 | Enviar un mensaje mientras el destinatario está fuera de la aplicación: recibe una notificación push agrupada.                                                    | V    |
| PR-CHT3 | Enviar varios mensajes consecutivos al mismo destinatario con la aplicación cerrada: solo recibe una notificación con el último título relevante (deduplicación). | V    |
| PR-CHT4 | Abrir la aplicación tras estar fuera: las notificaciones de la aplicación se limpian de la barra del sistema.                                                     | V    |
| PR-CHT5 | Verificar que el estado de un mensaje pasa de pendiente a enviado, después a entregado y finalmente a leído cuando el destinatario abre la conversación.          | V    |
| PR-CHT6 | Intentar enviar un mensaje a un usuario que ha eliminado su cuenta: el campo de entrada se sustituye por un aviso de solo lectura.                                | F    |

#### 4.6.6. Pruebas de notificaciones

| Código  | Descripción                                                                                            | Tipo |
|---------|--------------------------------------------------------------------------------------------------------|------|
| PR-NOT1 | Recibir una notificación de recordatorio y verlas listada en la pestaña Avisos.                        | V    |
| PR-NOT2 | Seleccionar varias notificaciones y marcarlas como leídas: el contador de no leídas se actualiza.      | V    |
| PR-NOT3 | Seleccionar varias notificaciones y eliminarlas: desaparecen de la lista.                              | V    |
| PR-NOT4 | Marcar todas las notificaciones como leídas y comprobar que el indicador rojo del tab desaparece.      | V    |
| PR-NOT5 | Desactivar el interruptor de notificaciones push y verificar que dejan de llegar al sistema operativo. | V    |
| PR-NOT6 | Reactivar el interruptor y verificar que vuelven a llegar.                                             | V    |

#### 4.6.7. Pruebas del panel de administración

| Código  | Descripción                                                                                                                                                     | Tipo |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| PR-ADM1 | Acceder al panel de administración con un usuario administrador.                                                                                                | V    |
| PR-ADM2 | Acceder al panel de administración con un usuario estándar (a través de una URL directa): el sistema responde con error de permiso.                             | F    |
| PR-ADM3 | Resolver un reporte de incidencia y verificar que aparece el administrador que lo cerró.                                                                        | V    |
| PR-ADM4 | Reabrir un reporte cerrado y verificar que el campo "resuelto por" se elimina.                                                                                  | V    |
| PR-ADM5 | Eliminar una publicación como moderador y verificar que el autor recibe una notificación.                                                                       | V    |
| PR-ADM6 | Banear temporalmente a un usuario por tres días: el usuario afectado recibe una notificación de aviso y, al intentar publicar, el sistema rechaza la operación. | V    |
| PR-ADM7 | Restaurar una publicación eliminada por moderación: vuelve a aparecer en el feed.                                                                               | V    |
| PR-ADM8 | Ejecutar manualmente el cron de recordatorios y verificar el resumen devuelto.                                                                                  | V    |
| PR-ADM9 | Simular un evento de lluvia y comprobar que se genera una notificación por cada planta exterior.                                                                | V    |

#### 4.6.8. Pruebas de soporte fuera de línea

| Código  | Descripción                                                                                                       | Tipo |
|---------|-------------------------------------------------------------------------------------------------------------------|------|
| PR-OFF1 | Desactivar el wifi en mitad de una sesión y verificar que el indicador superior de "Sin conexión" aparece.        | V    |
| PR-OFF2 | Intentar publicar un post sin conexión: el sistema informa del error de red.                                      | F    |
| PR-OFF3 | Restaurar la conexión y verificar que el indicador desaparece y los datos cacheados se refrescan automáticamente. | V    |

#### 4.6.9. Pruebas de perfil y gestión de cuenta

| Código  | Descripción                                                                                                                                                                        | Tipo |
|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| PR-PER1 | Editar el perfil propio modificando foto, banner, biografía y ubicación, y verificar que los cambios quedan persistidos y se reflejan en el perfil público.                        | V    |
| PR-PER2 | Editar el perfil sustituyendo la ubicación libre por una capital de provincia del catálogo y verificar que las coordenadas asociadas se actualizan correctamente.                  | V    |
| PR-PER3 | Cambiar la contraseña introduciendo la contraseña actual correcta y una nueva contraseña que cumple los requisitos de longitud.                                                    | V    |
| PR-PER4 | Intentar cambiar la contraseña introduciendo una contraseña actual incorrecta: el sistema rechaza la operación con un error de autenticación.                                      | F    |
| PR-PER5 | Intentar cambiar la contraseña con una nueva que no cumple la longitud mínima exigida: el sistema rechaza la operación con un mensaje de validación.                               | F    |
| PR-PER6 | Exportar los datos personales y verificar que se descarga un archivo JSON con el perfil completo y todas las plantas activas del usuario.                                          | V    |
| PR-PER7 | Eliminar la cuenta marcando la opción de preservar el contenido público: las publicaciones y comentarios anteriores siguen visibles pero quedan atribuidos a un usuario eliminado. | V    |
| PR-PER8 | Eliminar la cuenta sin marcar la opción de preservar el contenido: las publicaciones y comentarios del usuario desaparecen del feed.                                               | V    |

| Código  | Descripción                                                                                                                                             | Tipo |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| PR-PER9 | Intentar eliminar la cuenta introduciendo una contraseña incorrecta en el diálogo de confirmación: el sistema rechaza la operación y no aplica la baja. | F    |

#### 4.6.10. Pruebas de privacidad y preferencias

| Código  | Descripción                                                                                                                                                                            | Tipo |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| PR-PRV1 | Marcar el perfil como privado y verificar que, desde otra cuenta estándar, el usuario deja de aparecer en los resultados del buscador de la comunidad.                                 | V    |
| PR-PRV2 | Marcar el perfil como privado y verificar que, desde una cuenta administradora, el usuario sigue siendo visible en el buscador del panel administrativo.                               | V    |
| PR-PRV3 | Con el perfil del destinatario marcado como privado y sin conversación previa entre ambos, intentar iniciar una conversación: el sistema rechaza la operación con un error de permiso. | F    |
| PR-PRV4 | Reactivar el perfil como público y verificar que vuelve a aparecer en las búsquedas de la comunidad y permite recibir nuevas conversaciones.                                           | V    |
| PR-PRV5 | Mantener una conversación previa con un usuario que posteriormente marca su perfil como privado: la conversación existente sigue siendo funcional para ambas partes.                   | V    |

#### 4.6.11. Resumen multiplataforma

La siguiente tabla resume el resultado de cada prueba en cada una de las plataformas soportadas. El símbolo ✓ indica que la prueba se ejecutó con el resultado esperado; el símbolo — indica que la prueba no es aplicable a la plataforma. Las notificaciones push del sistema operativo solo se verifican en Android y en iOS, no en la versión web.

| Código  | Web | Android | iOS |
|---------|-----|---------|-----|
| PR-AUT1 | ✓   | ✓       | ✓   |
| PR-AUT2 | ✓   | ✓       | ✓   |
| PR-AUT3 | ✓   | ✓       | ✓   |
| PR-AUT4 | ✓   | ✓       | ✓   |
| PR-AUT5 | ✓   | ✓       | ✓   |

| Código   | Web | Android | iOS |
|----------|-----|---------|-----|
| PR-AUT6  | ✓   | ✓       | ✓   |
| PR-AUT7  | ✓   | ✓       | ✓   |
| PR-AUT8  | ✓   | ✓       | ✓   |
| PR-INV1  | ✓   | ✓       | ✓   |
| PR-INV2  | ✓   | ✓       | ✓   |
| PR-INV3  | ✓   | ✓       | ✓   |
| PR-INV4  | ✓   | ✓       | ✓   |
| PR-INV5  | ✓   | ✓       | ✓   |
| PR-INV6  | ✓   | ✓       | ✓   |
| PR-INV7  | ✓   | ✓       | ✓   |
| PR-INV8  | ✓   | ✓       | ✓   |
| PR-REC1  | ✓   | ✓       | ✓   |
| PR-REC2  | ✓   | ✓       | ✓   |
| PR-REC3  | ✓   | ✓       | ✓   |
| PR-REC4  | ✓   | ✓       | ✓   |
| PR-REC5  | ✓   | ✓       | ✓   |
| PR-REC6  | ✓   | ✓       | ✓   |
| PR-REC7  | ✓   | ✓       | ✓   |
| PR-COM1  | ✓   | ✓       | ✓   |
| PR-COM2  | ✓   | ✓       | ✓   |
| PR-COM3  | ✓   | ✓       | ✓   |
| PR-COM4  | ✓   | ✓       | ✓   |
| PR-COM5  | ✓   | ✓       | ✓   |
| PR-COM6  | ✓   | ✓       | ✓   |
| PR-COM7  | ✓   | ✓       | ✓   |
| PR-COM8  | ✓   | ✓       | ✓   |
| PR-COM9  | ✓   | ✓       | ✓   |
| PR-COM10 | ✓   | ✓       | ✓   |
| PR-CHT1  | ✓   | ✓       | ✓   |
| PR-CHT2  | —   | ✓       | ✓   |
| PR-CHT3  | —   | ✓       | ✓   |
| PR-CHT4  | —   | ✓       | ✓   |
| PR-CHT5  | ✓   | ✓       | ✓   |

| Código  | Web | Android | iOS |
|---------|-----|---------|-----|
| PR-CHT6 | ✓   | ✓       | ✓   |
| PR-NOT1 | ✓   | ✓       | ✓   |
| PR-NOT2 | ✓   | ✓       | ✓   |
| PR-NOT3 | ✓   | ✓       | ✓   |
| PR-NOT4 | ✓   | ✓       | ✓   |
| PR-NOT5 | ✓   | ✓       | ✓   |
| PR-NOT6 | ✓   | ✓       | ✓   |
| PR-ADM1 | ✓   | ✓       | ✓   |
| PR-ADM2 | ✓   | ✓       | ✓   |
| PR-ADM3 | ✓   | ✓       | ✓   |
| PR-ADM4 | ✓   | ✓       | ✓   |
| PR-ADM5 | ✓   | ✓       | ✓   |
| PR-ADM6 | ✓   | ✓       | ✓   |
| PR-ADM7 | ✓   | ✓       | ✓   |
| PR-ADM8 | ✓   | ✓       | ✓   |
| PR-ADM9 | ✓   | ✓       | ✓   |
| PR-OFF1 | ✓   | ✓       | ✓   |
| PR-OFF2 | ✓   | ✓       | ✓   |
| PR-OFF3 | ✓   | ✓       | ✓   |
| PR-PER1 | ✓   | ✓       | ✓   |
| PR-PER2 | ✓   | ✓       | ✓   |
| PR-PER3 | ✓   | ✓       | ✓   |
| PR-PER4 | ✓   | ✓       | ✓   |
| PR-PER5 | ✓   | ✓       | ✓   |
| PR-PER6 | ✓   | ✓       | ✓   |
| PR-PER7 | ✓   | ✓       | ✓   |
| PR-PER8 | ✓   | ✓       | ✓   |
| PR-PER9 | ✓   | ✓       | ✓   |
| PR-PRV1 | ✓   | ✓       | ✓   |
| PR-PRV2 | ✓   | ✓       | ✓   |
| PR-PRV3 | ✓   | ✓       | ✓   |
| PR-PRV4 | ✓   | ✓       | ✓   |
| PR-PRV5 | ✓   | ✓       | ✓   |

**Pruebas automatizadas complementarias:**

Al margen de las pruebas funcionales descritas en este plan, el proyecto cuenta con una batería de pruebas automatizadas ejecutadas tanto en el desarrollo local como antes de cada despliegue:

- Pruebas unitarias y de servicio del backend: 267 casos repartidos en 28 suites, ejecutados con Jest. Cubren la lógica de los casos de uso, los servicios técnicos (autenticación, notificaciones, meteorología) y los mapeadores.
- Pruebas de extremo a extremo del backend: 3 suites adicionales ejecutadas con Supertest sobre la aplicación arrancada (autenticación completa, ciclo de vida de una planta y moderación administrativa).
- Pruebas unitarias y de widget del frontend: 340 casos ejecutados con `flutter test`. Cubren los ViewModels, los mapeadores y los widgets más relevantes.

La cobertura de líneas medida con las herramientas estándar de cada plataforma se sitúa en torno al 29% en el backend y al 23% en el frontend, con estrategia piramidal que prioriza la lógica de dominio sobre la capa de presentación y de infraestructura (cubierta esta última principalmente por las pruebas funcionales).

